



सर्व विद्या अधिपतये

Faculty of Engineering (Diploma)

Semester – 3

Subject Name – Data Structure and Algorithm

Subject Code - DE103303

Faculty Name – Ashwini Walake

## UNIT -1: Basic Concept of Data Structure

### What is a Data Structure?

A data structure is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently.

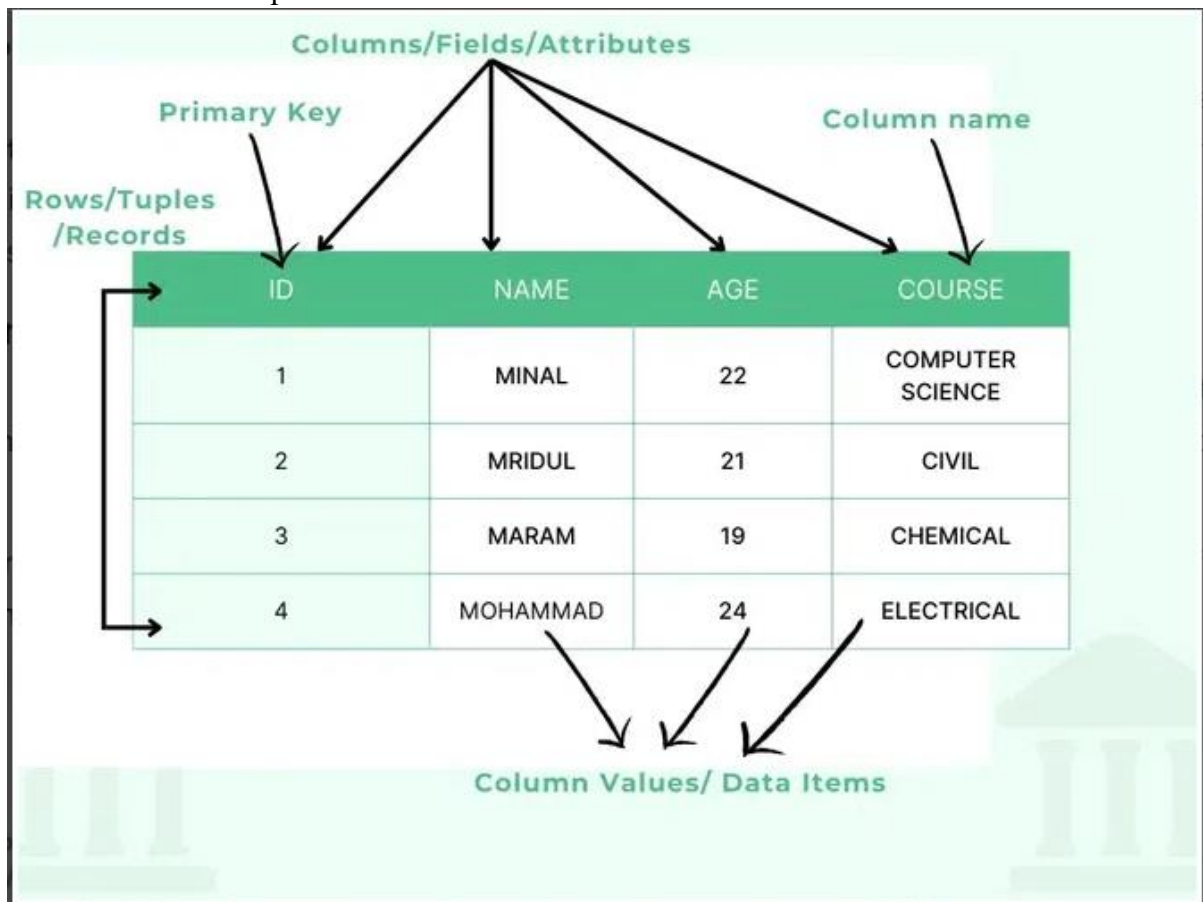
Data structures are specialized formats for **organizing, processing, storing, and retrieving data** efficiently in a computer's memory.

It defines the relationship between the data and the operations that can be performed on it (such as insertion, deletion, search, sorting, etc.).

### Information:

It is processed, organised and structured data in a meaningful way.

Information is more precise form of data. Information is created from data.



### Cell:

It is a basic structural and functional unit which represents an entity.

### Field:

The smallest piece of information that can be stored.

OR simply 'single piece of information'.

### Record:

*It is a collection of fields.*

In a record, all the fields are logically related.

### File:

*It is a collection of records.*

A file is a large list that stores the data & information and retrieves it.

Files can be accessed in a sequential or random access manner.

## Purpose of Data Structures

Data structures are the basis of programming tools and the choice of data structures should provide the following:

1. The data structures should satisfactorily represent the **relationship between data elements**.
2. The data structures should be **easy** so that the **programmer can easily process the data**.
3. The study of Data Structure covers the following points:
4. Amount of **memory required to store** data
5. Amount of **time required to process** data
6. Representation of data in memory
7. **Operation performed** on data

## Advantages of DS (Data Structures)

- Data structures (DS) allow **information storage on hard disks**.
- It also provides **efficient and effective data manipulation**.
- We can access data anytime, anywhere.
- DS provide **models to real-life problems**.
- DS can be decomposed into their component elements, so **processing of data becomes easy**.
- DS used in **efficient algorithm design**.

## Applications of DS

Data structures are used to organize and manage data efficiently in computer systems. They improve the speed of searching, sorting, inserting, deleting, and processing data. Some important applications are:

### 1. Database Management Systems (DBMS)

- Databases use **B-Trees** and **Hash Tables** to store and retrieve records quickly.
- **Example:** Searching a student's record using a roll number.

### 2. Operating Systems

- Operating systems use **Queues** for CPU scheduling and **Stacks** for function calls.
- **Example:** Managing multiple processes running on a computer.

### 3. Compiler Design

- Compilers use **Stacks** and **Trees** for syntax analysis and expression evaluation.
- **Example:** Evaluating mathematical expressions like  $(A+B) \times C$ .

### 4. Computer Networks

- **Graphs** are used to find the shortest path for data transmission.
- **Example:** Internet routing and GPS navigation.

### 5. File Management Systems

- File systems use **Tree** structures to organize folders and files.
- **Example:** Windows File Explorer directory structure.

## 6. Web Browsers

- Browsers use **Stacks** to store browsing history.
- **Example:** The Back and Forward buttons.

## 7. Search Engines

- Search engines use **Tries** and **Hash Tables** for fast searching and auto-complete suggestions.
- **Example:** Google search suggestions while typing.

## 8. Social Media

- **Graphs** represent users and their connections.
- **Example:** Friend recommendations on social networking sites.

## 9. E-Commerce Websites

- Data structures help manage products, customer details, and orders.
- **Example:** Searching products on Amazon or Flipkart.

## 10. Artificial Intelligence (AI)

- AI uses **Trees** and **Graphs** for decision-making and problem-solving.
- **Example:** Decision trees used in machine learning.

## 1.2 TYPES OF DATA STRUCTURE

Data structures are specialized formats for **organizing, processing, storing, and retrieving data** efficiently in a computer's memory. They are broadly categorized into **Primitive** and **Non-Primitive** structures, which are further divided based on how the elements are arranged and accessed.

### 1. Primitive Data Structures

These are the most fundamental, predefined data types provided directly by a programming language. They hold a single value and serve as the building blocks for more complex structures

- **Integer:** Stores whole numbers without decimal points (e.g., 5, -12).
- **Float / Double:** Stores fractional numbers requiring decimal precision (e.g., 3.14, 0.001).
- **Character:** Holds a single text symbol or letter (e.g., 'A', '\$').
- **Boolean:** Represents logical truth values, strictly restricted to True or False.
- **Pointer / Reference:** Stores the memory address of another variable to allow direct memory handling.

### 2. Non-Primitive Data Structures

These are complex architectures created by combining primitive data types. They emphasize the relationships between grouped items and are divided into two main categories

#### A. Linear Data Structures

Elements in a linear structure are arranged sequentially or linearly, meaning each item is connected directly to its previous and next adjacent elements. They can be traversed fully in a single run

- **Arrays:** A collection of elements of the same type stored in contiguous memory blocks, allowing direct access via index numbers.
- **Linked Lists:** A sequence of nodes where each node contains its data and a memory pointer linking it to the next node.
- **Stacks:** A structure operating strictly on the **Last-In, First-Out (LIFO)** principle, where additions (push) and deletions (pop) occur only at the top end.
- **Queues:** A structure operating on the **First-In, First-Out (FIFO)** principle, where elements are added at the back (enqueue) and removed from the front (dequeue)

### B. Non-Linear Data Structures

Elements are arranged in a hierarchical or interconnected network rather than a straight sequence. You cannot traverse all elements in a single sequential pass.

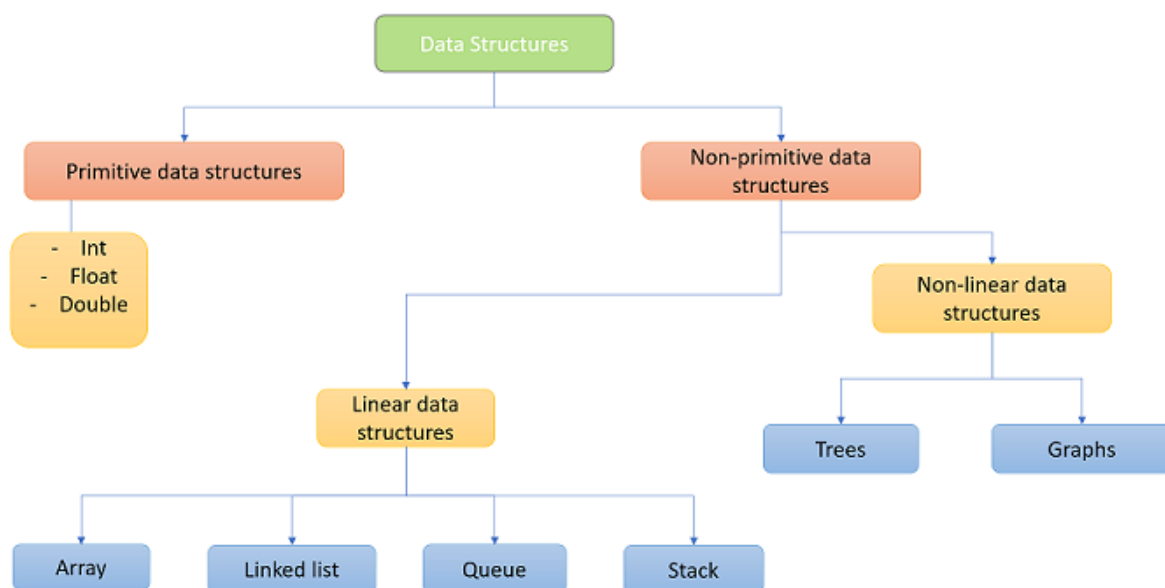
- **Trees:** A multi-level, hierarchical structure featuring a top "root" node that branches down into multiple child nodes and ends at "leaf" nodes.
- **Graphs:** A network of nodes (vertices) connected non-sequentially by links (edges), ideal for representing maps or social connections.

### 3. Specialized Data Structures

Engineered for highly specific computational tasks, these often build upon standard types to ensure fast processing

- **Hash Tables:** Formats that use a mathematical formula (hash function) to map unique keys to specific values for rapid retrieval.
- **Heaps:** Specialized tree variants designed so that the highest or lowest value remains pinned at the root for priority access.
- **Tries:** Digital search trees used to store and rapidly retrieve strings, frequently running auto-complete and spell-check programs

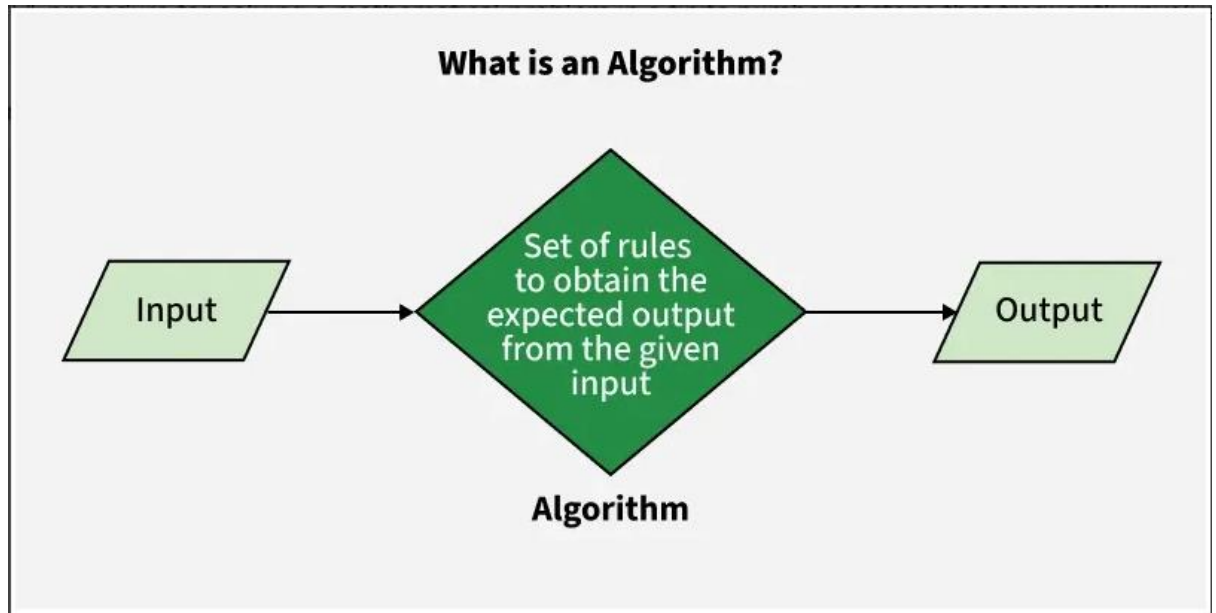
## ◆ 1.3 PRIMITIVE AND NON-PRIMITIVE DATA STRUCTURE



## INTRODUCTION TO ALGORITHMS

- An algorithm (*pronounced Al-go-rith-um*) is a procedure or formula for solving a problem.

- An **algorithm** is a sequence of finite steps to complete a task.
- **Algorithm** is defined as steps followed to solve particular problem which is written in common language which non-professionals can understand.
- Algorithm is not the complete code or program; it is just the core logic (solution) of a problem, which can be expressed using pseudo code.
- An algorithm can be implemented in more than one programming language.



### KEY FEATURES (CHARACTERISTICS) OF AN ALGORITHM

- **Input:** an algorithm must have either zero or more finite number of inputs.
- **Output:** at least one output must be generated by every algorithm.
- **Definiteness:** every step in the algorithm must be clear and unambiguous.
- **Finiteness:** an algorithm must be completed in finite number of steps.
- **Effectiveness:** Every step of an algorithm must be effective or have some meaning, i.e., useless or extra steps should be omitted.
- **Correctness:** For every input, an algorithm should produce correct output.
- **Feasibility:** An algorithm must be feasible, i.e., possible to implement.
- **Independent:** An algorithm should have step-by-step directions which should be independent of any programming code.

### Advantages of using algorithm

- As algorithm is a stepwise representation of a solution to a given problem, it is easy to understand.
- It is not dependent on any programming language.
- It can work as a blueprint for program development.
- By using algorithm, the problem is broken down into smaller parts, hence it is easy for the programmer to convert it into an actual program.

### Disadvantages of algorithm

- It is time consuming.
- It is difficult to show branching and looping in algorithm.
- Big problems are difficult to solve using algorithm.

### Algorithm to find out sum of two numbers

#### Step1. [Initialization]

Take two input numbers: x and y

Int x="10"; int y="20";

#### Step2. [Compute sum]

SUM = x + y;

#### Step3. [Print Result]

Print "SUM"

#### Step4. [Finish]

Exit

### Output

SUM = 30

## ALGORITHM ANALYSIS

Algorithm analysis is the study of an algorithm's efficiency and correctness. It helps answer questions such as:

- **How fast does the algorithm run?** (Time complexity)
- **How much memory does it use?** (Space complexity)
- **Will it always produce the correct result?** (Correctness)

### 1. Time Complexity

Time complexity measures how the running time grows as the input size (n) increases. It is commonly expressed using **Big-O notation**.

Some common complexities are:

| Complexity           | Name         | Example                          |
|----------------------|--------------|----------------------------------|
| (O(1))               | Constant     | Accessing an array element       |
| (O( $\log n$ ))      | Logarithmic  | Binary search                    |
| (O(n))               | Linear       | Linear search                    |
| (O(n $\log$ n))      | Linearithmic | Merge sort, Heap sort            |
| (O(n <sup>2</sup> )) | Quadratic    | Bubble sort, Selection sort      |
| (O(2 <sup>n</sup> )) | Exponential  | Recursive Fibonacci (naive)      |
| (O(n!))              | Factorial    | Traveling Salesman (brute force) |

### 2. Space Complexity

Space complexity measures the amount of extra memory an algorithm requires.

Example:

- Iterative factorial:  $O(1)$  extra space.
- Recursive factorial:  $O(n)$  space due to the recursion stack.

### 3. Best, Average, and Worst Cases

- **Best case:** Minimum running time.
- **Average case:** Expected running time over typical inputs.
- **Worst case:** Maximum running time.

Example (Linear Search):

- Best case:  $O(1)$  (target is first element)
- Average case:  $O(n)$
- Worst case:  $O(n)$  (target is last or absent)

### 4. Asymptotic Notations

- **Big-O ( $O$ ):** Upper bound (worst-case growth).
- **Big-Omega ( $\Omega$ ):** Lower bound (best-case growth).
- **Big-Theta ( $\Theta$ ):** Tight bound (exact asymptotic growth).

### 5. Example Analysis

Algorithm:

```
for i = 1 to n
  print(i)
```

The loop executes **n** times.

- Time complexity:  **$O(n)$**
- Space complexity:  **$O(1)$**

Another example:

```
for i = 1 to n
  for j = 1 to n
    print(i, j)
```

The inner loop runs (n) times for each of the (n) iterations of the outer loop.

- Time complexity:  **$O(n^2)$**
- Space complexity:  **$O(1)$**

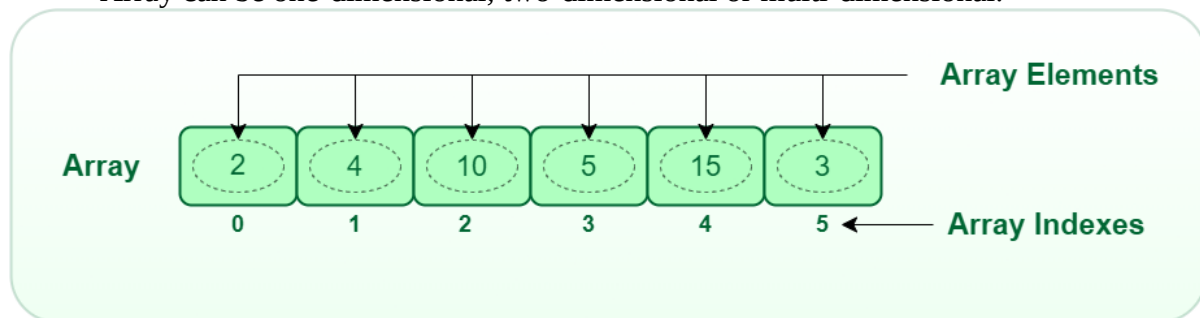
### Why Algorithm Analysis Matters

- Compares different algorithms objectively.
- Predicts performance for large inputs.
- Helps optimize software for speed and memory.
- Guides the selection of the most appropriate algorithm for a given problem.

In summary, algorithm analysis evaluates an algorithm's efficiency in terms of **time** and **space**, primarily using asymptotic notations such as (**O**), (**\Omega**), and (**\Theta**).

### Array

- Array is a **sequential collection** of variables with **same data types**.
- Array is a linear data structure which stores the set of similar type values under one variable name. I.e. **Index**
- It is an ordered set of elements consists of fixed number of data items.
- It uses **static and sequential storage** allocation.
- Array can be one-dimensional, two-dimensional or multi-dimensional.



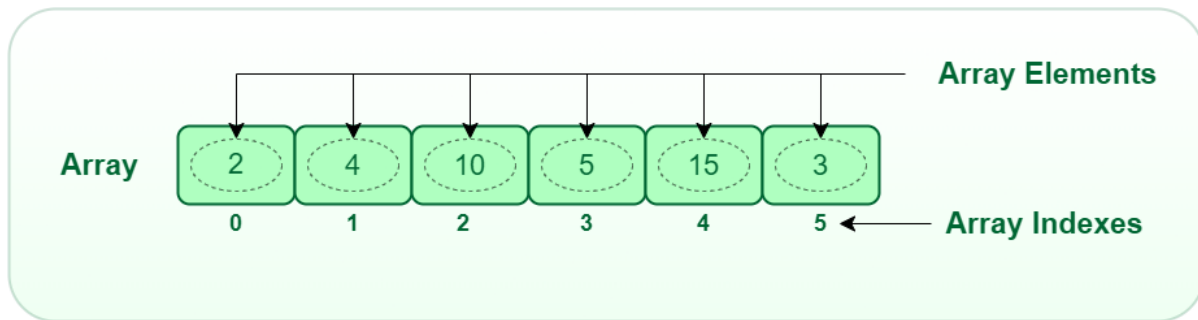
### Characteristics of an Array

- Array has a number of memory cells which are called elements, and each memory cell will occupy bytes depending on the type of array.
- The array elements are always stored **sequentially or in linear** fashion.
- Every element has an **assigned unique number**, which is called the **address of that particular element**. I.e. **Index**
- Element number is assigned by the name of the array and index number. Arrays always start with **0 index numbers** and end with one less than the size of the array.
- **Example:** If we define an array A with integer type and 3 elements like:  
`int A[3];`
- The amount of **memory occupied** by the array depends on the data type and number of elements of the array.
- In arrays, the lower bound cannot be changed, but the upper bound may be expanded or changed.
- In arrays, insertion and deletion operations are slower and time-consuming, but searching and sorting operations are faster.
- When an array is declared and not initialized, it contains **garbage values**. If it is declared as **static**, all elements are initialized to **zero by default**.

### One-Dimensional Array with Example

- The array represents a group of similar data type values under a single name.
- The C programming language allows users to define one-dimensional arrays as follows:  
**Syntax:** `data_type array_name[size];`  
**For example:** `int A[10];`
- Where A is the name of the array of 10 integers. 10 is the size of the array.
- It defines 10 integer variables, all known by the variable name A.
- In C language, array index or subscript starts from 0. So, for example, we have different integer variables as:
  - `A[0], A[1], A[2], A[3], A[4], A[5], A[6], A[7], A[8], A[9]`

- Where A[0] is the first element of an array, while A[9] is the last element.



### Program example of one-dimensional array

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[5], i;

    clrscr(); // Clears the screen

    printf("Enter five array elements:\n");

    for(i = 0; i < 5; i++)
    {
        scanf("%d", &a[i]);
    }

    printf("\nPrinting the data:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d", a[i]);
        printf("\n");
    }

    getch(); // Waits for a key press
}
```

yaml

Enter five array elements: 1 2 3 4 5

Printing data:

1

2

3

4

5

## Two Dimensional Array with Example

- Sometimes we need to store the data where more than one dimension is needed — like sales information of a company, or for mathematical calculations we need a matrix or to represent data in a tabular manner.
- In general, a two-dimensional array is used to store data in table form (i.e., **Row** and **Column**).

Syntax:

```
data_type variable_name[row size][column size];
```

- Where **variable\_name** represents the name of an array,  
**row size** indicates the number of rows in the table,  
**column size** indicates the number of columns in the array.
-

For example:

Two-dimensional array of integers can be represented as:

C

[Copy code](#)

```
int Mat[2][2];
```

So elements are:

CSS

[Copy code](#)

```
Mat[0][0]  Mat[0][1]  
Mat[1][0]  Mat[1][1]
```

If array has size  $3 \times 3$ , then it is declared as:

C

[Copy code](#)

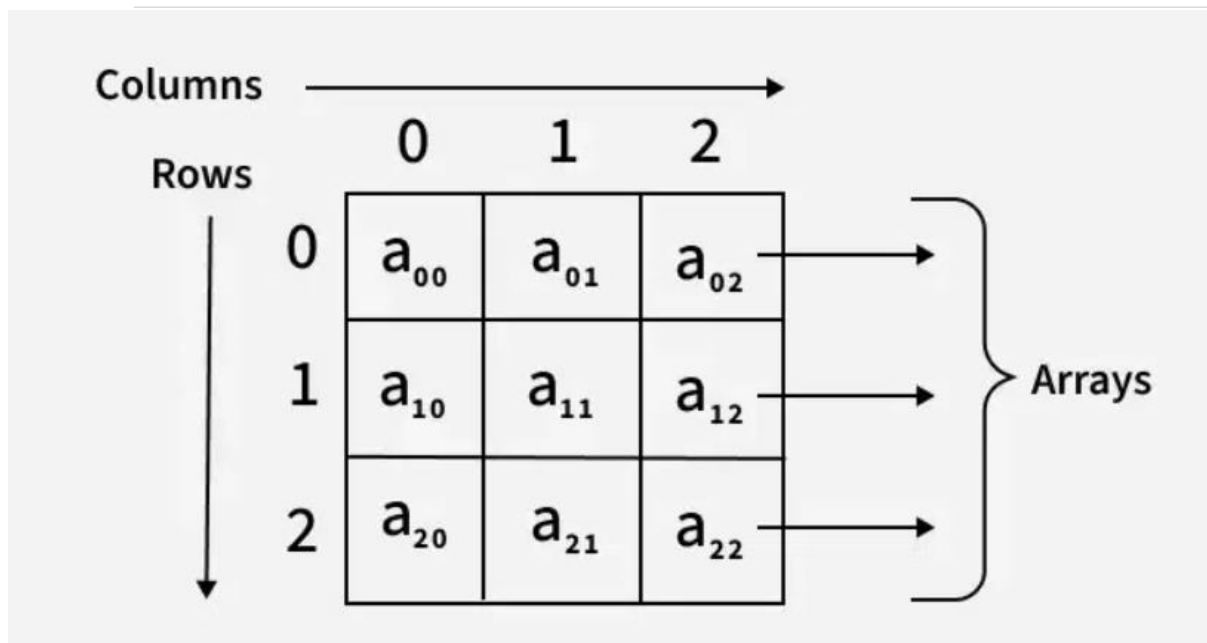
```
int Mat[3][3];
```

Which contains 9 elements like:

CSS

[Copy code](#)

```
Mat[0][0]  Mat[0][1]  Mat[0][2]  
Mat[1][0]  Mat[1][1]  Mat[1][2]  
Mat[2][0]  Mat[2][1]  Mat[2][2]
```



## UNIT 2-Strings

### STRING REPRESENTATION: READING AND WRITING STRINGS

- A string is a sequence or collection of characters. String is a one-dimensional array of characters and it is terminated by '\0' (NULL).
- It is a group of zero or more characters between double quotation marks ("").  
Example: "HELLO" is a string containing 5 characters.
- A string should contain: Alphabets (lowercase a–z, uppercase A–Z), Numeric values (0–9), Special characters (+, –, etc.), NULL
- The NULL character is automatically added at the end of a string. NULL is also written as '\0' in programming.
- Example: If we write a string "HELLO", then in memory it will be stored as:

mathematica

H E L L O \0

### Declaring the string (character array)

- C does not support a string data type. But as a string is a collection of characters, we have to take a character array to declare a string.
- The general syntax to declare a string is:

arduino

```
char string_name[size];
```

```
char str1[10];
```

- The above statement will create a character array str1 of size 10, and we can take a string of maximum 9 characters (one reserved for NULL).
- We can also declare a string using a pointer. The syntax is:

cpp

```
char *string_name;
```

Example:

cpp

```
char *S;
```

This will declare a string `S` using a pointer.

### Initialization of String

- Initialization means assigning the values to the string at the time of declaration.
- It can be done in many different ways.

## String Initialization Examples

c

```
char s[] = "hello";  
or  
char s[10] = "hello";  
or  
char s[] = {'h', 'e', 'l', 'l', 'o', '\0'};  
or  
char s[6] = {'h', 'e', 'l', 'l', 'o', '\0'};  
or  
char *s = "hello";    // using pointer
```

### Reading String Using scanf() function

- This is the most popular and standard input function. Using it we can read strings from the keyboard.

c

```
char str[10];  
scanf("%s", str);
```

But scanf() terminates input on the first whitespace. For example, if we type "Computer Engineering", it will only take "Computer".

### Using getchar() function

- Used to get a **single character** from the terminal (keyboard).
- This function can be used repeatedly to read a sequence of characters.

c

```
char ch;  
ch = getchar();    // reads a single character
```

### Using gets() function

- This is the most convenient way to read a **string with spaces**.
- It reads an entire line of text until the **newline character (Enter key)**.
- Available in <stdio.h>.
- General form:

c

```
gets(str);    // where str is a string variable
```

Example:

c

```
char name[10];  
gets(name);           // will get maximum of 9 characters  
printf("%s", name);    // %s is used to print the string
```

---

```
#include <stdio.h>
```

```
int main() {  
  
    // Small buffer size  
    char buff[10];  
    printf("Enter a string: ");  
  
    // No size limitation on input  
    gets(buff);  
    printf("You entered: %s\n", buff);  
    return 0;  
}
```

#### Output

```
Enter a string: This is a larger string (entered by the user)You entered: This is a larger string
```

## Writing String

### Using printf() function

- This is the most common method to print a string on the screen.
- %s format specifier is used to display the string.

C

```
printf("%s", str);
```

### Using putchar() function

- This function is used to put or print a **single character** on the screen.
- We can use this function repeatedly to print a sequence of characters using any loop.

C

```
int i;  
char name[10] = "AARUSH";  
for (i = 1; i < 10; i++) {  
    putchar(name[i]);    // it will print AARUSH on the screen  
}
```

### Using puts() function

- This is the most convenient way to print or write a string on the screen.
- It is available in <stdio.h> header file.
- General form of puts() function:

C

```
puts(str);    // where str is a string variable
```

### Example:

C

```
char name[10] = "NAKSH";  
puts(name);    // it will print NAKSH on the screen
```

### Example 1:

```
// C program to demonstrate putchar() method
```

```
#include <stdio.h>  
  
int main()  
{  
  
    // Get the character to be written  
    char ch = 'G';  
  
    // Write the Character to stdout  
    putchar(ch);  
  
    return (0);  
}
```

### Output

G

## STRING OPERATIONS

Different string operations are explained below:

- **Read string:** Used to read a string from the terminal.
- **Write string:** Used to write a string on the terminal.
- **Count the length:** Used to count the number of characters in the string.
- **Conversion:** Used to convert the case of the given string (i.e. upper to lower and vice versa).
- **String concatenate:** Used to combine two strings and store them into a third string.
- **String copy:** Used to copy the content of one string into another.
- **String compare:** Used to compare two strings for equality.
- **Substring:** Used to find a substring from the given string.

- **Reverse:** Used to reverse the content of the given string.
- **Append:** Used to add new string at the end of the given string.
- **Insertion:** Used to insert a text (word) in the given string.
- **Deletion:** Used to delete a text (word) from the given string.

### Finding Length of a String

#### Algorithm: STR\_LEN

*str is the given string.*

##### Step 1: [Initialize]

- Read string str from user
- count = 0

##### Step 2: [Process until end of the string]

- Repeat while (str[count] != NULL)
  - count = count + 1

##### Step 3: [Finish]

- Return (count)

### Hint:

- Character by character check each string until the end.
- Increment the count variable for each character.
- The value of count will be the **length of the string**.

## C Program for Finding the Length of a Given String

```
c
#include <stdio.h>
#include <conio.h>

void main() {
    char str[10];
    int count = 0;

    clrscr();
    printf("Enter your string: ");
    gets(str);

    while (str[count] != NULL) {
        count++;
    }

    printf("\nLength of string is: %d", count);
    getch();
}
```

 Copy code



## Output:

```
csharp
```

```
Enter your string: AARUSH
```

```
Length of string is: 6
```

## Converting Characters of a String into Upper Case and Lower Case

This algorithm converts the entered string into **lower case**.

### Algorithm: STR\_TOLOWER (S1)

*S1 is the string that we want to convert.*

#### Step 1: [Initialize]

- Read string `S1` from user
- `i = 0`

#### Step 2: [Convert upper to lower case]

- Repeat while `(S1[i] != NULL)`
  - If `(S1[i] >= 'A' AND S1[i] <= 'Z')`
    - Then `S2[i] = S1[i] + 32`
    - `i = i + 1`
  - Else
    - `S2[i] = S1[i]`

#### Step 3: [Finish]

- `S2[i] = NULL`
- Return `(S2)`

## C Program that Converts the Given String in Uppercase into Lowercase

```
c Copy code

#include <stdio.h>
#include <conio.h>

void main() {
    int i = 0;
    char s1[10], s2[10];

    clrscr();
    printf("Enter your string: ");
    gets(s1);

    while (s1[i] != NULL) {
        if (s1[i] >= 'A' && s1[i] <= 'Z') {
            s2[i] = s1[i] + 32; // convert to lowercase
        } else {
            s2[i] = s1[i];      // keep same if already lowercase
        }
        i++;
    }

    s2[i] = NULL; // terminate string

    printf("\nConverted string in lowercase: %s", s2);
    getch();
}
```

### Example Output

go

```
Enter a string in UPPERCASE: HELLO WORLD
Converted string (lowercase): hello world
```

### Copying a String (S1)

This algorithm copies the content of the given string **S1** into another string **S2**.

## Algorithm: STR\_COPY

### Step 1: [Initialize]

- Read string `s1` from user
- `count = 0`

### Step 2: [Process until end of the string and do copying]

- Repeat while `(S1[count] != NULL)`
  - `S2[count] = S1[count]`
  - `count = count + 1`

### Step 3: [Terminate copied string]

- `S2[count] = NULL`

### Step 4: [Finish]

- Return `(S2)`

## C Program for Copying the Given String

```
c
#include <stdio.h>
#include <conio.h>

void main() {
    int count = 0;
    char s1[10], s2[10];

    clrscr();
    printf("Enter string to copy: ");
    gets(s1);

    while (s1[count] != NULL) {
        s2[count] = s1[count];
        count = count + 1;
    }

    s2[count] = NULL;

    printf("\nCopied string is: %s", s2);
    getch();
}
```

 Copy code

## Output:

vbnet

Enter string to copy: india

Copied string is: india

## Concatenation of Two Strings to Form a Third New String

This algorithm concatenates (or merges) two strings **S1** and **S2** into a third string **S3**.

### Algorithm: STR\_CAT

#### Step 1: [Initialize]

- Read two strings `s1` and `s2` from user
- `count1 = 0`
- `count2 = 0`
- `count3 = 0`

### Step 2: [Process until end of the first string]

- Repeat while (`S1[count1] != NULL`)
  - `S3[count3] = S1[count1]`
  - `count1 = count1 + 1`
  - `count3 = count3 + 1`

### Step 3: [Process until end of the second string]

- Repeat while (`S2[count2] != NULL`)
  - `S3[count3] = S2[count2]`
  - `count3 = count3 + 1`
  - `count2 = count2 + 1`

### Step 4: [Finish]

- `S3[count3] = NULL`
- Return (`S3`)

### Hint:

- In Step 2, the program copies string **S1** into **S3** character by character.
- In Step 3, it copies string **S2** into **S3** character by character.
- Finally, the resulting string **S3** will contain concatenated `S1 + S2`.

c

```
#include <stdio.h>
#include <conio.h>

void main() {
    char s1[10], s2[10], s3[20];
    int count1 = 0, count2 = 0, count3 = 0;

    clrscr();
    printf("Enter two strings:\n\n");
    gets(s1);
    gets(s2);

    while (s1[count1] != NULL) {
        s3[count3] = s1[count1];
        count1++;
        count3++;
    }

    s3[count3] = ' '; // add space between strings
    count3++;

    while (s2[count2] != NULL) {
        s3[count3] = s2[count2];
        count2++;
        count3++;
    }

    s3[count3] = NULL;

    printf("\nConcatenated string is: %s", s3);
    getch();
}
```

 Copy code



## Output

csharp

```
Enter two strings
Computer
Engineering
Concatenated string is: ComputerEngineering
```

## String Appending

- This algorithm appends the string **S1** with the content of string **S2**.
- Means the string **S2** will be added at the end of the string **S1**.

### Algorithm: STR\_APPEND

#### Step 1. [initialize]

- Read two strings `S1` and `S2` from user
  - `count1 = 0`
  - `count2 = 0`
- 

#### Step 2. [process until end of the string]

- Repeat while `(S1[count1] != NULL)`
    - `count1 = count1 + 1`
- 

#### Step 3. [process until end of second string and append it to string one]

- Repeat while `(S2[count2] != NULL)`
    - `S1[count1] = S2[count2]`
    - `count1 = count1 + 1`
    - `count2 = count2 + 1`
- 

#### Step 4. [terminate string one and finish]

- `S1[count1] = NULL`
- Return `(S1)`



### Hint

- Step 2 moves to the end of string **S1**.
- Step 3 then copies characters of string **S2** one by one and appends them to **S1**.
- At the end, **S2** will be appended to **S1**.

## C program that append the content of string2 to string1

```
c Copy code

#include <stdio.h>
#include <conio.h>

void main()
{
    char S1[20], S2[10];
    int count1 = 0, count2 = 0;

    clrscr();
    printf("Enter String: ");
    gets(S1);
    printf("\nEnter String to append: ");
    gets(S2);

    while (S1[count1] != NULL)
        count1 = count1 + 1;

    while (S2[count2] != NULL)
    {
        S1[count1] = S2[count2];
        count1 = count1 + 1;
        count2 = count2 + 1;
    }

    S1[count1] = NULL;
    printf("\nAppended String is: %s", S1);

    getch();
}
```

## Output

```
vbnet

Enter String: Data
Enter String to append: Structure
Appended String is: DataStructure
```

## Reversing a string

This algorithm reverses the given string (org) and stores it into new string (rev).

## Algorithm: STR\_REVERSE

This algorithm reverses the given string ( `org` ) and stores it into a new string ( `rev` ).

### Step 1. [find length of given string]

- Read string `org` from user
- `len = strlen(org)`

### Step 2. [initialize]

- `count = 0`
- `len = len - 1`

### Step 3. [process string and copy character by character]

- While ( `len >= 0` )
  - `rev[count] = org[len]`
  - `count = count + 1`
  - `len = len - 1`

### Step 4. [finish]

- `rev[count] = NULL`
- return ( `rev` )

.

```

#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char org[10], rev[10];
    int len, count = 0;

    clrscr();
    printf("Enter your string: ");
    gets(org);

    len = strlen(org);
    len = len - 1;

    while (len >= 0)
    {
        rev[count] = org[len];
        count++;
        len--;
    }

    rev[count] = NULL;
    printf("\nReversed string is: %s", rev);

    getch();
}

```



## Output

csharp

Enter your string: abc  
Reversed string is: cba

## Comparing strings

This algorithm compares two strings S1 and S2 and finds whether they are equal or not.

### Algorithm: STR\_COMPARE

#### Step 1. [initialize]

- Read two strings `S1` and `S2` from user
- `i = 0`

#### Step 2. [find length of two strings]

- `L1 = strlen(S1)`
- `L2 = strlen(S2)`

#### Step 3. [compare length of two strings]

- If ( `L1 != L2` )
  - Then write "STRINGS ARE NOT EQUAL"
  - `return()`

#### Step 4. [process strings and compare character by character]

- Repeat while ( `i < L1` )
  - If `S1[i] != S2[i]`
    - Then write "STRINGS ARE NOT EQUAL"
    - `return()`
  - `i = i + 1`

#### Step 5. [return equal string]

- Write "STRINGS ARE EQUAL"
- `return()`



#### Hint:

- In Step 2, store the string lengths of S1 and S2 in `L1` and `L2`.
- In Step 3, compare the lengths. If lengths differ, strings are not equal.
- If lengths are equal, then check each character one by one in Step 4.
- If even one character is different, strings are not equal.
- If all characters match in Step 4, strings are equal.

## C program to compare two strings

c

 Copy code

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char S1[10], S2[10];
    int i = 0, L1, L2;

    clrscr();
    printf("Enter two strings\n\n");
    gets(S1);
    gets(S2);

    L1 = strlen(S1);
    L2 = strlen(S2);

    if (L1 != L2)
    {
        printf("\nStrings are not equal");
        getch();
        exit(0); // this function forcefully exits the program
    }

    while (i < L1)
    {
        if (S1[i] != S2[i])
        {
            printf("\nString are not equal");
            getch();
            exit(0);
        }
        i = i + 1;
    }

    printf("\n\nStrings are Equal");
    getch();
}
```

## Output

sql

 Copy code

```
Enter two Strings
hello
heoll
Strings are not equal
```

✅ This program checks:

1. If the **lengths are different** → immediately prints *not equal*.
2. If lengths are same → compares character by character.
  - If any character mismatches → prints *not equal*.
3. If all match → prints *Strings are Equal*.

## Find a substring from the given string

This algorithm finds the substring SUB in the given string S, from the position POS followed by NUM characters.

### Algorithm: STR\_SUBSTR

#### Step 1. [initialize]

- Read string `S` from user
- `i = 0`
- `POS = POS - 1`

#### Step 2. [process the string]

- Repeat while ( `NUM > 0` )
  - `SUB[i] = S[POS]`
  - `i = i + 1`
  - `POS = POS + 1`
  - `NUM = NUM - 1`

#### Step 3. [finish]

- `SUB[i] = NULL`
- return ( `SUB` )

## Hint

- Step 1: Declare variable `POS`.
- Step 2: From particular position `POS`, copy `NUM` number of characters into string `SUB`.
- Finally, we get the substring in `SUB`.

c

```
#include <stdio.h>
#include <conio.h>

void main()
{
    char S[20], SUB[20];
    int i = 0, NUM, POS;

    clrscr();
    printf("Enter your String: ");
    gets(S);

    printf("\nEnter the starting position from which you want substring: ");
    scanf("%d", &POS);

    printf("\nEnter number of characters for substring: ");
    scanf("%d", &NUM);

    POS = POS - 1;

    while (NUM > 0)
    {
        SUB[i] = S[POS];
        i++;
        POS++;
        NUM--;
    }

    SUB[i] = '\0'; // terminate substring
    printf("\nSubstring is: %s", SUB);

    getch();
}
```

yaml

Enter your String: computer

Enter the starting position from which you want substring: 3

Enter number of characters of substring: 3

Substring is: put

### String insertion

This algorithm inserts the string S3 into string S1 at the specified position and forms a new string S3.

#### Algorithm: STR\_INSERT

- S1 is the given string in which you want to insert new string.
- S2 is the string which is to be inserted.
- POS describes the position after which we have to insert the string.
- S3 is the resulting string.

#### Step 1. [initialize]

- Read two strings S1 and S2 from user
- i = 0
- j = 0
- k = 0

---

#### Step 2. [copy the content of string S1 into string S3 before insertion position]

- Repeat while ( i < POS )
  - S3[k] = S1[i]
  - i = i + 1

### Step 3. [initialize and set position of j]

```
ini  
  
j = 0
```

### Step 4. [insert the string]

```
r  
  
repeat while (S2[j] ≠ NULL)  
    S3[k] = S2[j]  
    j = j + 1  
    k = k + 1
```

### Step 5. [add remaining part of string S1 into S3]

```
r  
  
repeat while (S1[i] ≠ NULL)  
    S3[k] = S1[i]  
    i = i + 1  
    k = k + 1
```

### Step 6. [terminate the string S3 and return]

```
kotlin  
  
S3[k] = NULL  
return (S3)
```

---

#### Hint:

- In **Step 1**, initialize variables `i, j, k`.
- In **Step 2**, copy S1 into S3 till insertion position.
- In **Step 4**, insert string S2 into S3.
- In **Step 5**, copy remaining characters of S1 into S3.
- Finally, in **Step 6**, terminate string with `NULL` character → result is new string S3.

c

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char s1[10], s2[10], s3[20];
    int i=0, k=0, j=0, pos;
    clrscr();
    printf("Enter String:");
    gets(s1);
    printf("\nEnter string to be inserted: ");
    gets(s2);
    printf("\nEnter position after which string to be inserted:");
    scanf("%d", &pos);

    while(i < pos)
    {
        s3[k] = s1[i];
        i++;
        k++;
    }
}
```

```

while(s2[j] != NULL)
{
    s3[k] = s2[j];
    j++;
    k++;
}

while(s1[i] != NULL)
{
    s3[k] = s1[i];
    i++;
    k++;
}

s3[k] = NULL;
printf("\nNew string is: %s", s3);
getch();
}

```

## Output:

vbnet

Enter string: aarush

Enter string to be inserted: parikh

Enter position after which string to be inserted: 1

Modified String is:

aparikharush

## String deletion

### Algorithm: STR\_DEL

- S1 is the string from which we have to delete the string.
- POS describes the position after which we have to delete the string containing NUM characters.
- S2 is the result string.

### Step 1. [initialize]

```
pgsql

read string 'S1' from user
i = 0
j = 0
```

---

### Step 2. [copy the content of the S1 upto position POS]

```
lua

repeat while (i < POS)
    S2[j] = S1[i]
    i = i + 1
    j = j + 1
```

### Step 3. [skip the string (text) that we want to delete]

```
lua

repeat while (NUM > 0)
    i = i + 1
    NUM = NUM - 1
```

#### Step 4. [copy the remaining part of S1 into S2]

```
r

repeat while (S1[i] ≠ NULL)
    S2[j] = S1[i]
    i = i + 1
    j = j + 1
```

#### Step 5. [terminate the String S2 and return]

```
kotlin

S2[j] = NULL
return (S2)
```

C program that delete the substring of number of characters from the given string at particular position

```
c

#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char s1[10], s2[10];
    int i = 0, j = 0, pos, num;

    clrscr();
    printf("Enter String:");
    gets(s1);

    printf("\nEnter position after that you want deletion: ");
    scanf("%d", &pos);

    printf("\nEnter number of characters to delete: ");
    scanf("%d", &num);
```

 Copy code



```

    while (i < pos)
    {
        s2[j] = s1[i];
        i++;
        j++;
    }
}

```

```

while (num > 0)
{
    i++;
    num--;
}

while (s1[i] != NULL)
{
    s2[j] = s1[i];
    i++;
    j++;
}

s2[j] = NULL;
printf("\n\nString after deletion is: %s", s2);
getch();
}

```

## Output:

pgsql

Enter string: Computer

Enter position **after** that you want deletion: 2

Enter number **of** characters **to delete**: 3

String **after** deletion **is**: Comer

## UNIT -3: Stack and Queue

### LINEAR AND NON-LINEAR DATA STRUCTURE:

- **Non-Primitive Data structure** is classified into two categories: linear and non-linear data structure.
- A data structure is said to be **linear** if the data is arranged in linear sequence physically or logically or in both the ways.
  - Example: Array, Stack, Queue, Linked List etc.
- A data structure is said to be **non-linear** if the elements/data are not stored sequentially.
  - Example: Tree, Graph etc.

☞ The main difference between linear and non-linear data structure is:

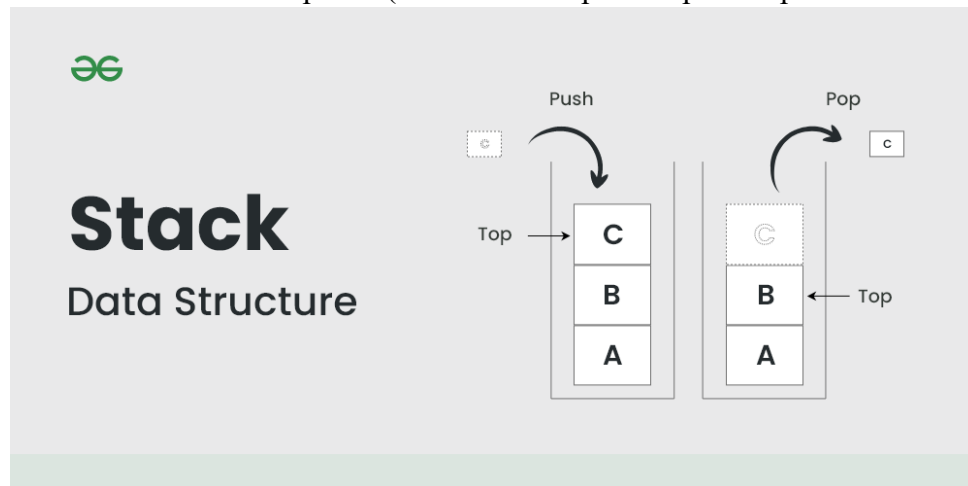
- In **linear fashion**, data are organized sequentially. So, they are easy to implement in computer's memory.
- In **non-linear fashion**, they are difficult to implement. Care should be taken to select data in non-linear data structure.

### 3.2 STACK:

- Stack is one of the most important data structures in computer science.
- **Definition:** A stack is a **non-primitive linear data structure** in which **insertion and deletion operations are performed at only one end**, known as **top of the stack (TOP)**.
- As deletion and insertion operation in stack is done from one end, the **last added element will be removed first**.
- So, stack is also called **Last In First Out (LIFO)**.

☞ **Example of stack:**

Illustration: A stack of plates (where the last plate kept on top is the first one removed).



### Consider a stack of plates

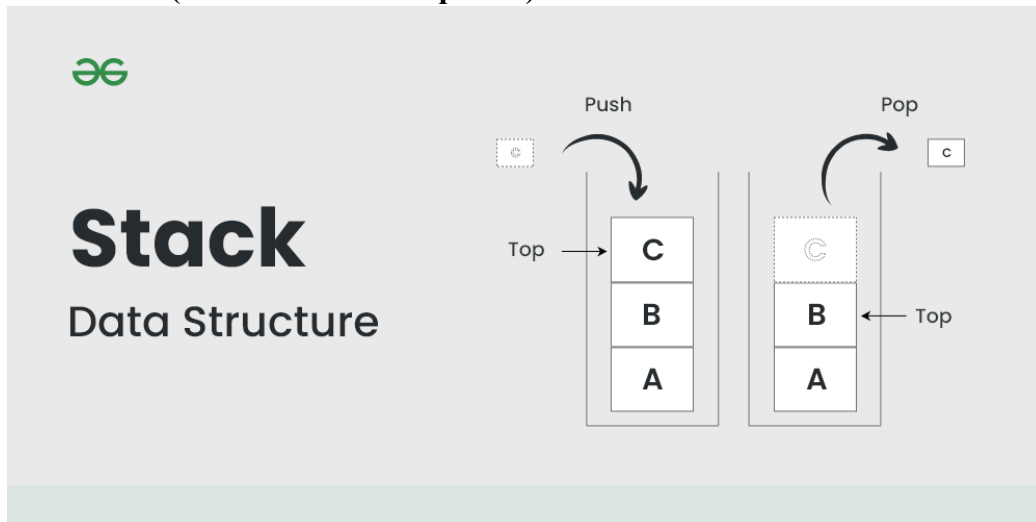
On the counter in a cafeteria shown in the above figure, during the time of dinner, customers take plates from the top of the stack and after then waiter puts the used plates on the top of the stack.

- So a new plate is put on top and the old one is yet at bottom.
- Most **microprocessors** use a stack-based architecture. When a member function is called, the return address and arguments are pushed into a stack, and when the function returns, they are popped from the stack.

### 3.2.1 Characteristics of stack

- The insertion operation in stack is called **PUSH** operation.
- The deletion operation in stack is called **POP** operation.
- The first element of the stack is referred to as **BOTTOM** element.
- The most accessible element in the stack is the **TOP** element (only one end of the stack is accessible).
- Since insertion and deletion are performed only at one end, stack is also known as **Last In First Out (LIFO)**.
- The element inserted last in the stack is first to delete.
- A pointer TOP always keeps track of the top element in the stack.
  - Initially when the stack is empty,  $TOP = 0$ .
  - When the stack contains a single element,  $TOP = 1$ .
  - Each time a new element is inserted, the TOP pointer increases by one.
  - Each time an element is deleted, the TOP pointer decreases by one.

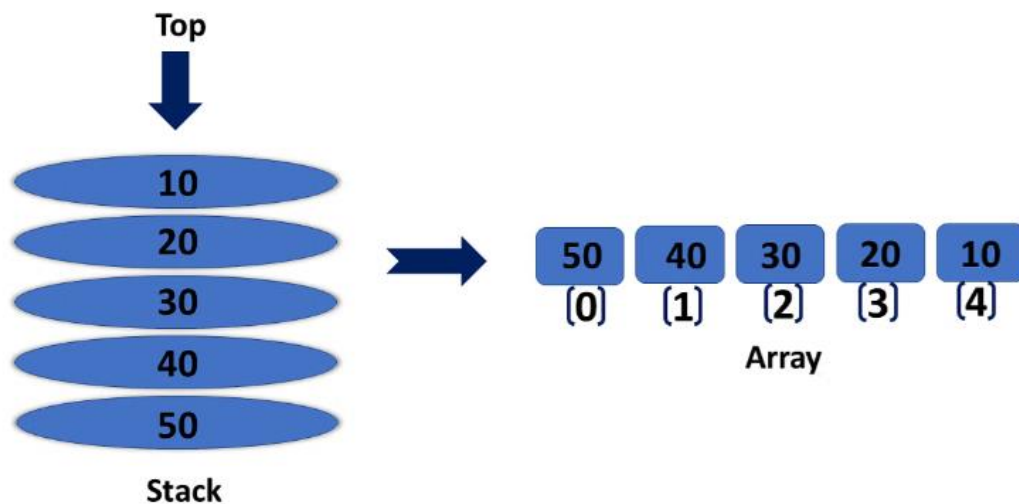
**Illustration (PUSH and POP sequence):**



### 3.2.1 Array Representation of Stack

A stack can be represented using an array in two ways:

- **Vertical Representation of Stack:**
- **Horizontal Representation of Stack:**



### Operations of Stack (PUSH and POP operation)

Basically, two main operations are performed on stack:

1. **PUSH:**

- To insert an element on the TOP of the stack is called **PUSH operation**.
- Pushing an element in the stack inserts a new element at the TOP.
- After every PUSH operation the **TOP** is incremented by one.
- **Condition:**
  - If the stack is full, it is called **Stack Full condition** or **Stack Overflow**.

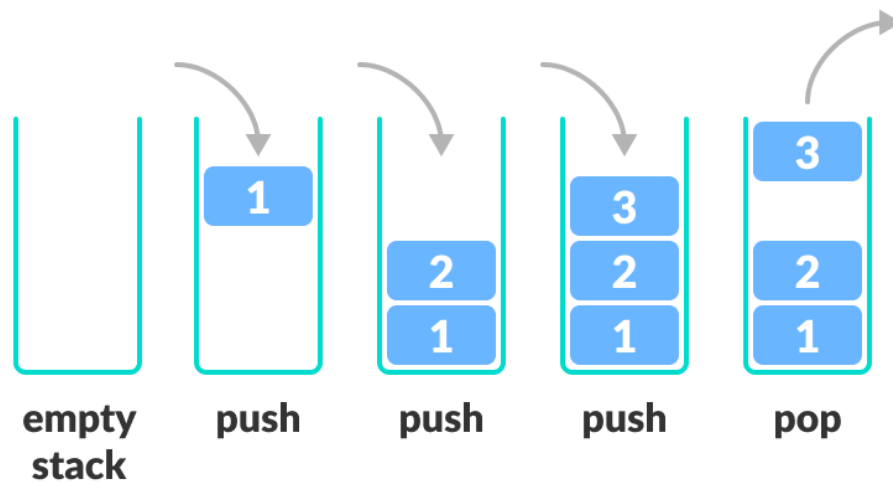
2. **POP:**

- To remove an element from the TOP of the stack is called **POP operation**.
- After every POP operation the **TOP** is decremented by one.
- **Condition:**
  - If there is no element on the stack and POP is performed, this will result in **Stack Underflow**.

### Other operations

Along with the above two main operations, two other operations can also be performed on stack:

1. **Peep:** To retrieve a specific element from the stack.(the peek method just returns the topmost element; however, in the case of a pop method, the topmost element is returned and also that element is deleted from the stack)
2. **Change:** To change the value of a specific element from the stack.



### Algorithm: STACK\_PUSH(S, TOP, X)

This algorithm inserts an element on the top of the stack.

- **S:** representing array for Stack
- **TOP:** a variable which points to the top of the stack
- **X:** the element to be inserted

#### Steps:

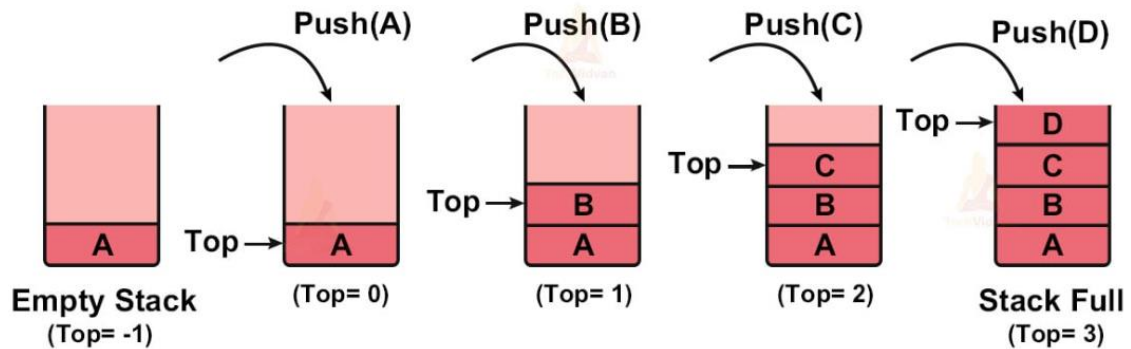
1. **Check for stack overflow**
  - If ( $TOP \geq N$ )
    - Write "STACK OVERFLOW"
    - Return(0)
2. **Increment TOP**
  - $TOP = TOP + 1$
3. **Insert element**
  - $S[TOP] = X$
4. **Finished**
  - Return(0)

#### ★ Description:

In step 1, the algorithm checks for overflow (whether the stack is full). If not, then the TOP pointer is incremented in step 2, and the element is inserted (pushed) in step 3.

#### Example

# Push Operation



## Algorithm: STACK\_POP(S, TOP)

This algorithm removes an element from the top of the stack.

- **S**: representing array for Stack
- **TOP**: a variable which points to the top of the stack
- **Y**: a variable which holds the value of deleted element

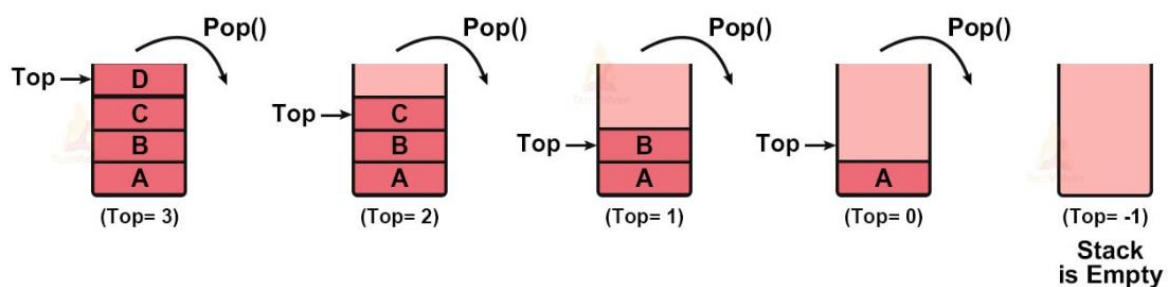
### Steps:

1. **Check for stack underflow**
  - If (TOP = 0)
    - Write "STACK UNDERFLOW"
    - Return(0)
2. **Delete an element**
  - $Y \leftarrow S[TOP]$
3. **Decrement TOP pointer**
  - $TOP \leftarrow TOP - 1$
4. **Return deleted element**
  - Return (Y)

### ★ Description:

In step 1, the algorithm checks for underflow (whether the stack is empty). If not, then the popped element is stored in variable Y and TOP is decremented by 1 in step 3.

# Pop operation



## 3.2.1 Implementation of Stack

Stack can be implemented in two ways:

1. **Static implementation (using Array)**
2. **Dynamic implementation (using Linked List)**

### 3.2.1 Applications of Stack

There are mainly three applications of stacks:

1. **Recursion**
2. **Stack machine**
3. **Polish notation**

---

#### (1) Recursion

- Recursion is an important facility in many programming languages such as C, C++ etc.
- It defines as, *“function calling itself, thus the chain of processes occurs.”*
- There are two types of recursion:
  - (i) Primitive Recursive function
  - (ii) Non-primitive Recursive function
- Stack performs a major role in implementing recursion.

---

#### (2) Stack machines

- It is used for executing reverse polish notation.
- It is also used for stacking local variables.
- Stack machine provides faster execution of polish notation.
- One problem with stack machine is that it has very limited number of registers.

---

#### (3) Polish notation

- The process of writing the operators of an expression either before those operands or after operands are called the Polish Notation.
- Polish notation is used for representation of arithmetic expression.
- There are basically three types of polish notation:
  - (A) Infix
  - (B) Prefix
  - (C) Postfix

### 3.2.2 Infix, Prefix and Postfix forms of expressions

One of the most important applications of stack is a polish notation or polish form.

- The process of writing the operators of an expression either before those operands or after operands is called the *polish notation*.

There are basically three types of polish notation:

1. Infix
2. Prefix
3. Postfix

✓ When the operator exists between two operands then it is known as **Infix** notation.

Example:  $A + B$

✓ When the operator exists before its operands then it is known as **Prefix** notation.

Example:  $+AB$

✓ When the operator exists after its operands then it is known as **Postfix** notation or reverse polish notation.

Example:  $AB+$

- Stack is widely used to evaluate infix notation or prefix or postfix notation.
- If exponential expression (^) is used more than one time in a sequence in any expression then order will be right to left, for any other expression order will be left to right (like +, -, \*, /, %).

The following recursive definition gives all such valid infix expressions:

1. Any single letter variable (a to z) or nonnegative integer is a valid infix expression.
2. If **a** and **b** satisfied in step-1 then (a+b), (a-b), (a\*b) and (a/b) are valid infix expressions.
3. The only valid infix expressions are those defined by step-1 and step-2.

---

### Conversion of algebraic functions (Infix to Postfix)

**General guidelines at the time of converting infix expressions into postfix expressions:**

- Take a single operation at a time (in which one operator and two operands are existing).
- The operator with the highest precedence should be converted first.
- Then converted operation is considered as a single unit in the next operation.
- Always convert the parentheses first.
- If more than two operators are there in expressions having same precedence then operate them as per their association (i.e. +, -, \*, / have left to right).

**Precedence of symbols is as follows:**

| Symbol (operator) | Priority |
|-------------------|----------|
| ( )               | Highest  |
| ^                 |          |
| * / %             |          |
| + -               | Lowest   |

1.  $a + b * c - d / e * f$

r

$= a + b * c - d / e * f$   
 $= a + b c * - d / e * f$   
 $= a b c * + d e / f * -$   
 $= a b c * + d e / f * -$

2.  $(a + b * c) + (d * e / b)$

r

$= (a + b * c) + (d * e / b)$   
 $= (a + b * c) + (d * e / b)$   
 $= a + b c * + d e * / b$   
 $= a b c * + d e * b / +$

**Expression 1** —  $a + b * c - d / e * f$

**Goal:** Convert to postfix.

| Step | Token | Action                                                | Operator Stack | Output (Postfix) |
|------|-------|-------------------------------------------------------|----------------|------------------|
| 0    | —     | Start                                                 | —              | —                |
| 1    | A     | Operand → output                                      | —              | A                |
| 2    | +     | Push (stack empty)                                    | +              | A                |
| 3    | B     | Operand → output                                      | +              | a b              |
| 4    | *     | Higher than + → push                                  | + *            | a b              |
| 5    | C     | Operand → output                                      | + *            | a b c            |
| 6    | -     | * > - → pop *<br>+ = - (left-assoc) → pop +<br>Push - | -              | a b c * +        |
| 7    | D     | Operand → output                                      | -              | a b c * + d      |
| 8    | /     | Higher than - → push                                  | - /            | a b c * + d      |
| 9    | E     | Operand → output                                      | - /            | a b c * + d e    |

| Step | Token | Action                                        | Operator Stack | Output (Postfix)      |
|------|-------|-----------------------------------------------|----------------|-----------------------|
| 10   | *     | / = * (left-assoc) → pop / → output<br>Push * | - *            | a b c * + d e /       |
| 11   | F     | Operand → output                              | - *            | a b c * + d e / f     |
| 12   | —     | End → pop * then -                            | —              | a b c * + d e / f * - |

✓ **Final postfix:**

a b c \* + d e / f \* -

### ⓑ Expression 2 — (a + b \* c) + (d \* e / b)

**Goal:** Convert to postfix.

| Step | Token | Action                                        | Operator Stack | Output (Postfix)      |
|------|-------|-----------------------------------------------|----------------|-----------------------|
| 0    | —     | Start                                         | —              | —                     |
| 1    | (     | Push (                                        | (              | —                     |
| 2    | A     | Operand → output                              | (              | A                     |
| 3    | +     | Push (( on stack)                             | ( +            | A                     |
| 4    | B     | Operand → output                              | ( +            | a b                   |
| 5    | *     | Higher than + → push                          | ( + *          | a b                   |
| 6    | C     | Operand → output                              | ( + *          | a b c                 |
| 7    | )     | Pop until ( → pop *, +, discard (             | —              | a b c * +             |
| 8    | +     | Stack empty → push                            | +              | a b c * +             |
| 9    | (     | Push (                                        | + (            | a b c * +             |
| 10   | D     | Operand → output                              | + (            | a b c * + d           |
| 11   | *     | Push (( on stack)                             | + ( *          | a b c * + d           |
| 12   | E     | Operand → output                              | + ( *          | a b c * + d e         |
| 13   | /     | * = / (left-assoc) → pop * → output<br>Push / | + ( /          | a b c * + d e *       |
| 14   | B     | Operand → output                              | + ( /          | a b c * + d e * b     |
| 15   | )     | Pop until ( → pop /, discard (                | +              | a b c * + d e * b /   |
| 16   | —     | End → pop +                                   | —              | a b c * + d e * b / + |

✓ **Final postfix:**

a b c \* + d e \* b / +

$$3. \quad x * (c + d) + (j + k) * n + m * p$$

r

$$= x * (c + d) + (j + k) * n + m * p$$

$$= x * c + d + j * k + n + m * p$$

$$= x * c + d + j * k + n + m * p$$

$$= x * c + d + j * k + n + m * p$$

$$= x * c + d + j * k + n + m * p$$

$$4. \quad a + b * (c / d) - e$$

r

$$= a + b * (c / d) - e$$

$$= a + b * c / d - e$$

$$= a + b * c / d - e$$

$$= a + b * c / d - e$$

5.  $(a * b) * (c ^ (d + e) - f)$

r

$$\begin{aligned}
 &= (a * b) * (c ^ (d + e) - f) \\
 &= a * b * (c ^ (d + e) - f) \\
 &= a b * c d e + ^ f - *
 \end{aligned}$$

6.  $(a + b) * c - (d - e) * (f + g)$

r

$$\begin{aligned}
 &= (a + b) * c - (d - e) * (f + g) \\
 &= a + b * c - d - e * f + g \\
 &= a b + c * d e - f g + * - \\
 &= a b + c * d e - f g + * -
 \end{aligned}$$

### Algorithm to convert from Infix to Postfix using stack (Reverse Polish Notation)

#### Step 1:

Initialize an empty stack of operators.

#### Step 2:

While no error and not to end of infix expression:

a. Get the next input token (constant, variable, arithmetic operator, left parenthesis, right parenthesis etc.) in the infix expression.

b. If the token is:

i. A left parenthesis: Push it onto the stack.

ii. A right parenthesis: Pop and display stack elements until a left parenthesis is encountered. But do not display the parentheses.

- It is an error if the stack becomes empty with no left parenthesis found.

iii. An operator:

- If the stack is empty or input token has **higher priority** than the top of the stack symbol, push onto the stack.

- If the stack is **not empty** and input token has **lower priority** than top of stack symbol, pop and display the top stack element, then compare input token with the new top of stack.

**Note:** A left parenthesis in the stack is assumed to have lower priority than all operators.

iv. An operand: just display it.

#### Step 3:

When end of infix expression is reached, pop and display stack items until the stack is empty.

**Example: Conversion of infix to postfix expressions using stack****Expression:**  $A * B + C$ 

We will show this trace in table with three columns:

- First column: Current symbol being read.
- Second column: What is on the stack.
- Third column: Current postfix string.

Stack will be written left to right (bottom to top of the stack is left to right).

| Step # | Current Input Symbol | Operator on Stack | Postfix String                                                                              |
|--------|----------------------|-------------------|---------------------------------------------------------------------------------------------|
| 1      | A                    |                   | A                                                                                           |
| 2      | *                    | *                 | A                                                                                           |
| 3      | B                    | *                 | AB                                                                                          |
| 4      | +                    | +                 | AB*                                                                                         |
|        |                      |                   | <i>(+ has lower priority, so top of stack * is popped and displayed, then + is pushed.)</i> |
| 5      | C                    | +                 | AB*C                                                                                        |
| 6      | end of string        |                   | <b>AB*C+</b>                                                                                |
|        |                      |                   | <i>(This is the postfix string which we wanted as result)</i>                               |

**2.  $A * (B + C)$** 

| Step # | Current Input Symbol | Operator on Stack | Postfix String |
|--------|----------------------|-------------------|----------------|
| 1      | A                    |                   | A              |
| 2      | *                    | *                 | A              |
| 3      | (                    | *(                | A              |
| 4      | B                    | *(                | AB             |
| 5      | +                    | *(+               | AB             |
| 6      | C                    | *(+               | ABC            |
| 7      | )                    |                   | ABC+           |
| 8      |                      |                   | <b>ABC+*</b>   |

**3.  $A * (B + C * D) + E$**

| Step # | Current Symbol | Operator on Stack | Postfix String   |
|--------|----------------|-------------------|------------------|
| 1      | A              |                   | A                |
| 2      | *              | *                 | A                |
| 3      | (              | *(                | A                |
| 4      | B              | *(                | AB               |
| 5      | +              | *(+               | AB               |
| 6      | C              | *(+               | ABC              |
| 7      | *              | *(+*              | ABC              |
| 8      | D              | *(+*              | ABCD             |
| 9      | )              | *(                | ABCD*+           |
| 10     | +              | +                 | ABCD*+*          |
| 11     | E              | +                 | ABCD*+*E         |
| 12     |                |                   | <i>ABCD*+*E+</i> |

### 3.2.1 Recursive Functions

- **Recursion** is a problem-solving approach in which a problem is solved using previous solutions by applying the same solution to smaller instances.
- **Recursive function** is a programming technique that allows the programmer to define operations in terms of themselves.

Simply put, the recursive function is a  $\rightarrow$  **function call to itself**.

## Example:

A very simple example that shows recursion is:

```
c

void main()
{
    printf("Hello");
    main();
    getch();
}
```

`main()` function is calling itself, and the program will be in an infinite loop.

## Advantages of Recursion

- Simple to implement.
- Recursion is very useful when the solution of a problem is in **repetitive form**.
- It is used with data structures like: **linked list, stacks, queues**, etc.

---

## Disadvantages of Recursion

- We have to solve sub-problems **recursively**.
- **Care should be taken**, else there is a chance of infinite loop.

---

Different recursive functions are explained below:

### Factorial number

The factorial function can be **recursively defined** as:

```
cpp

Factorial(n) = 1                // when n == 1
              = n * factorial(n-1) // otherwise
```

## C program to find factorial of a given number:

```
c

#include <stdio.h>
#include <conio.h>

long int fact(int);

void main()
{
    int num;
    long int factorial;

    clrscr();
    printf("Enter your number: ");
    scanf("%d", &num);

    factorial = fact(num);
    printf("\n\nFactorial value is: %ld", factorial);

    getch();
}

long int fact(int n)
```

## Continuation of Factorial Program (Recursive)

```
c
long int fact(int n)
{
    if(n == 0)
        return 1;
    else
        return (n * fact(n - 1));
}
```

Output:

```
yaml
```

```
Enter your number: 5
Factorial value is: 120
```



## Fibonacci Series

### Description:

Fibonacci series is defined as a sequence of numbers in which the first two are 1 and 1, or 0 and 1, depending on the selected beginning point of the sequence. Every subsequent number is the sum of the previous two. So, in this series, the  $n^{\text{th}}$  term is the sum of the  $(n-1)^{\text{th}}$  term and  $(n-2)^{\text{th}}$  term.

**Mathematically**, the  $n^{\text{th}}$  term of the Fibonacci series can be represented as:

$$t_n = t_{n-1} + t_{n-2} \quad t_n = t_{n-1} + t_{n-2}$$

**The function of Fibonacci series can be recursively defined as:**

**Take n as input**

```
cpp
```

```
if (n == 0 or n == 1) then fibo(n) = n
else fibo(n) = fibo(n-1) + fibo(n-2)
```

### Step-by-step example:

1. Start with:

$$F(0) = 0, \quad F(1) = 1$$

2. Now calculate further terms using the formula:

- $F(2) = F(1) + F(0) = 1 + 0 = 1$
- $F(3) = F(2) + F(1) = 1 + 1 = 2$
- $F(4) = F(3) + F(2) = 2 + 1 = 3$
- $F(5) = F(4) + F(3) = 3 + 2 = 5$
- $F(6) = F(5) + F(4) = 5 + 3 = 8$

C program for generating Fibonacci series is given below:

```
c

#include <stdio.h>
#include <conio.h>

int fibo(int); // Function declaration

void main() {
    int n, i = 0;
    clrscr();
    printf("Enter the range of series you want: ");
    scanf("%d", &n);
    printf("\nFibonacci Series is:\n");

    while (i < n) {
        printf("%d ", fibo(i));
        i++;
    }
    getch();
}

// Function definition
int fibo(int n) {
    if (n == 0 || n == 1)
        return n;
    else
        return (fibo(n - 1) + fibo(n - 2));
}
```

**Output:**

mathematica

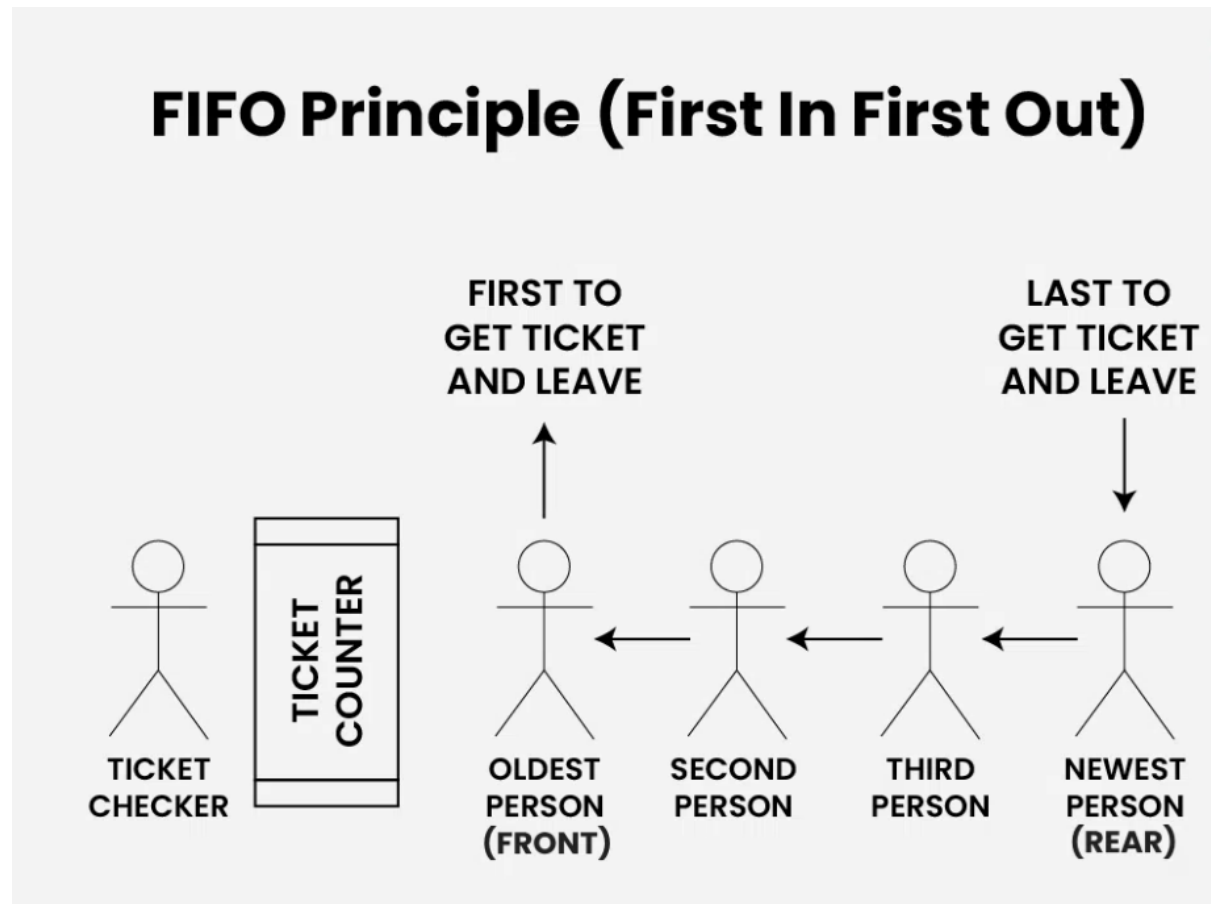
```
Enter the range of series you want: 5
Fibonacci Series is: 0 1 1 2 3
```

### 3.3 QUEUE:

A queue is a non-primitive linear data structure in which insertion operation is performed at one end called **REAR** and deletion operation is performed at another end of the list called **FRONT**.

#### Examples of Queue:

1. A row of students at registration counter.
2. Line of cars waiting to proceed in some direction at a traffic signal, which came first is allowed to come out first.
3. A movie ticket window.



A queue has two pointers, **front pointer** and **rear pointer**, which are pointing to front and rear elements of the queue.



#### 3.3.1 Characteristics of a Queue

- The information in such a list is processed in the **same order** as it was received.
- Since **insertion is performed at one end** and **deletion is performed at another end**, the element which is inserted first is the first to be deleted.

- So, **queue data structure** is also known as **First In First Out (FIFO)** or **First Come First Served (FCFS)** data structure.

### 3.3.2 Operations on Queue (Insertion and Deletion)

Basically, two main operations performed on a queue:

#### 1. INSERT:

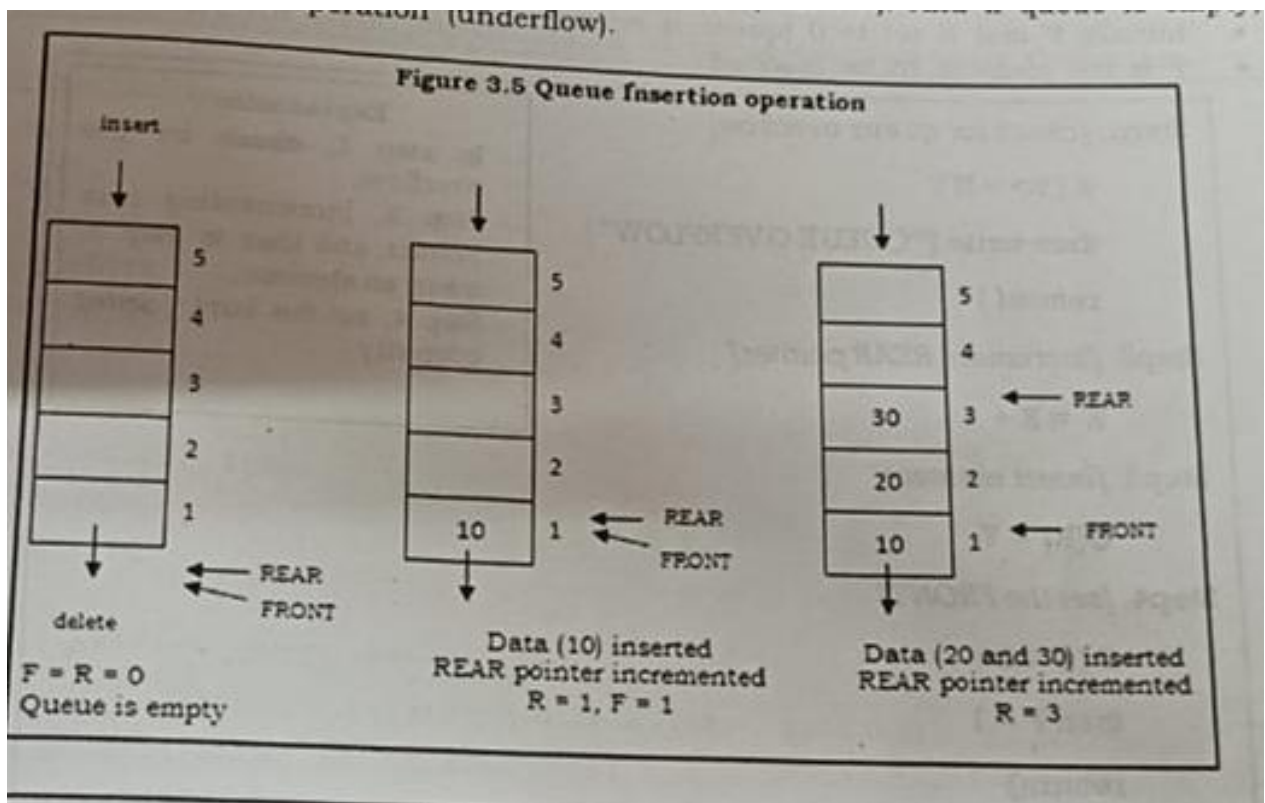
To insert an element at the **REAR end** of the queue is called **insertion** operation.

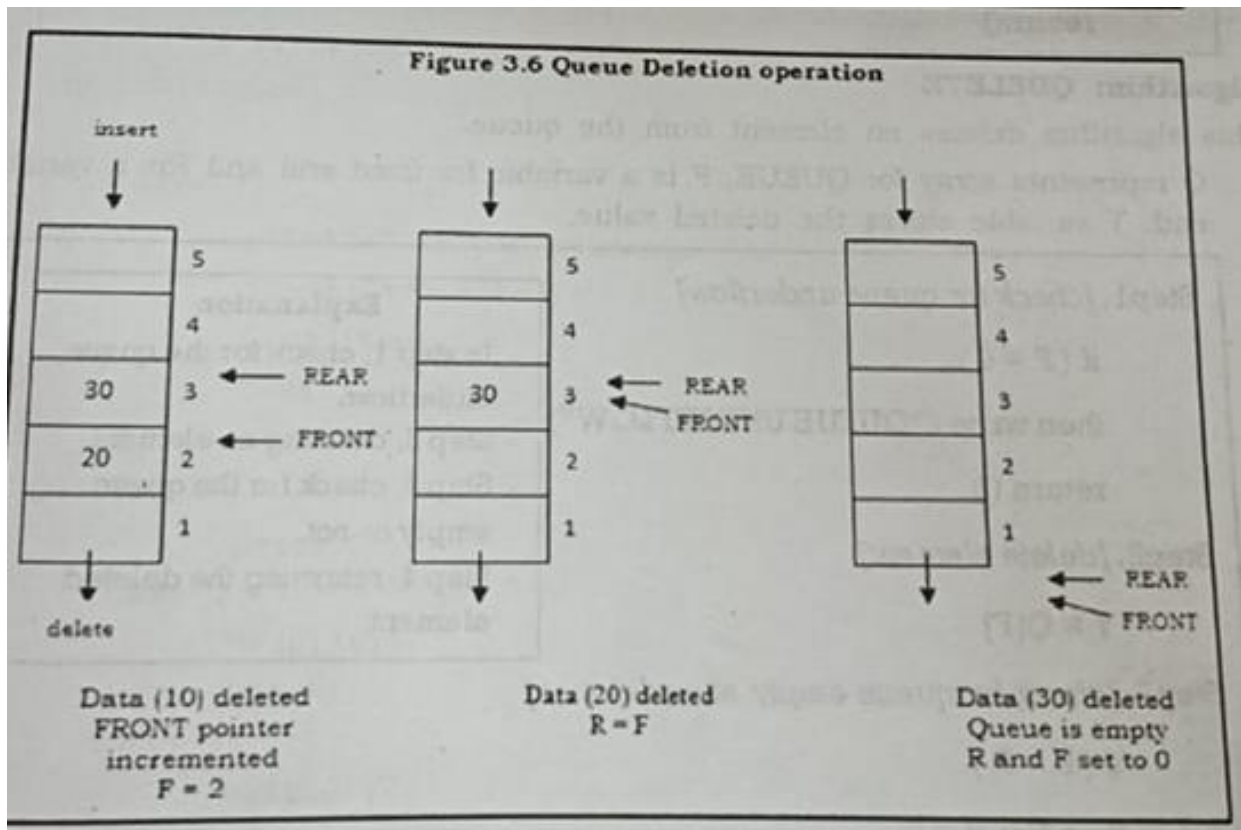
#### 2. DELETE:

To remove an element at the **FRONT end** of the queue is called **deletion** operation.

#### Notes:

- Initially, when the queue is empty, **F** and **R** pointers are at 0.
- When we insert an element at the REAR end:
  - **R pointer is incremented**
  - The element is inserted at REAR end.
  - As many elements as we insert (up to the size of the queue), each time **R pointer is incremented**.
  - When we are removing an element, it is removed at the **FRONT** end.
  - Each time an element is removed, the **F (front) pointer is incremented**.
  - When **F reaches R** and we do deletion operation, both **F and R pointers are set to 0** (i.e., the queue becomes empty).
  - If the queue is **full**, we can't insert more elements (**overflow**).
  - If the queue is **empty**, we can't do deletion operation (**underflow**).





### Algorithm: QINSERT

This algorithm inserts an element into the queue.

- Q represents array for **QUEUE**
- F is a variable for **front end**, R is a variable for **rear end**
- N is the **maximum number of elements**
- Initially F and R are set to 0 (queue is empty)
- Y is the element to be inserted

## Steps:

### Step 1. *[Check for queue overflow]*

```
C  
  
if (R >= N)  
    then write ("QUEUE OVERFLOW")  
    return()
```

### Step 2. *[Increment REAR pointer]*

```
C  
  
R = R + 1
```

### Step 3. *[Insert element]*

```
C  
  
Q[R] = Y
```

### Step 4. *[Set the FRONT]*

```
C  
  
if (F == 0)  
    then F = 1  
return()
```

---

## Explanation Box (QINSERT):

- Step 1: Check for overflow
- Step 2: Increment rear pointer
- Step 3: Insert an element
- Step 4: Set the front pointer properly

## Algorithm: QDELETE

This algorithm deletes an element from the queue.

- **Q** represents array for **QUEUE**
  - **F** is a variable for **front end**, **R** is a variable for **rear end**
  - **Y** stores the **deleted value**
- 

### Steps:

**Step 1.** *[Check for queue underflow]*

```
c

if (F == 0)
    then write ("QUEUE UNDERFLOW")
    return()
```

**Step 2.** *[Delete element]*

```
c

Y = Q[F]
```

**Step 3.** *[Check for queue empty after deletion]*

```
c

if (F == R)
    then F = R = 0
else
    F = F + 1
```

**Step 4.** *[Return deleted element]*

```
c

return (Y)
```

## Explanation Box (QDELETE):

- Step 1: Check for queue underflow
- Step 2: Deleting an element
- Step 3: Check if the queue is empty or not
- Step 4: Return the deleted element

### Limitations of the Simple Queue

If we have free spaces, we can not use these memory spaces. So simple queue results in wastage of memory space.

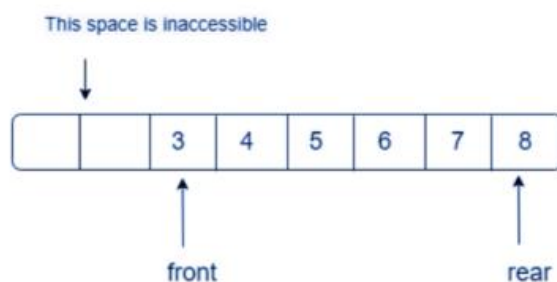
This problem can be solved by using a circular queue.

### 3.4 CONCEPT OF CIRCULAR QUEUE

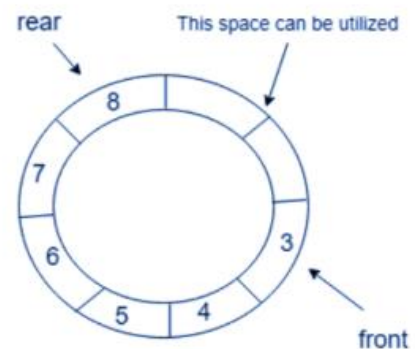
A circular queue is a queue in which data are arranged such that the last position is connected with the first element in the queue.

#### Explanation:

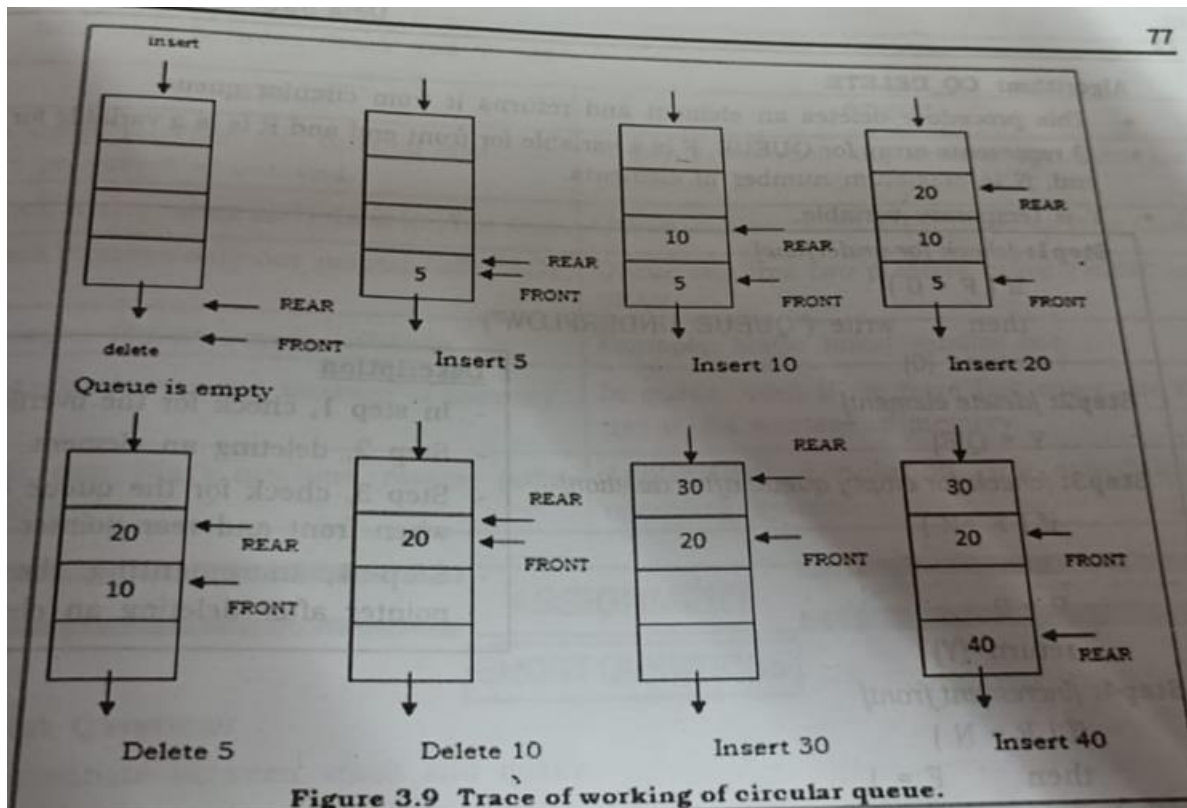
- In a simple queue, when we delete an element, the front pointer moves forward.
- The space previously occupied by the deleted element cannot be reused, which leads to wastage of memory space.
- This issue is solved by a circular queue, where:
  - If the rear or front pointer reaches the end, it wraps around to the first position (circular movement).



Linear Queue



Circular queue



### Algorithm: CQ\_INSERT

This procedure inserts an element in the circular queue.

- **Q** represents the array for the queue.
- **F** is a variable for front end.
- **R** is a variable for rear end.
- **N** is the maximum number of elements.
- **Y** is the element to be inserted.

Steps:

Step 1. [reset rear]

c

```

if (R == N)
    then R = 1
else
    R = R + 1
  
```

**Step 2.** [check for overflow]

```
c

if (F == R)
    then write("QUEUE OVERFLOW")
    return()
```

**Step 3.** [insert element]

```
c

Q[R] = Y
```

**Step 4.** [set the front pointer properly]

```
c

if (F == 0)
    then F = 1
return()
```

**Description Box:**

- **Step 1:** Checks if the **rear pointer** has reached the end, and resets it to 1 if so.
- **Step 2:** Checks for **queue overflow**.
- **Step 3:** Inserts an element.
- **Step 4:** Ensures the **front pointer** is set properly if the queue was initially empty.

### Algorithm: CQ\_DELETE

This procedure deletes an element and returns it from the **circular queue**.

- **Q** represents array for QUEUE
  - **F** is a variable for front end
  - **R** is a variable for rear end
  - **N** is maximum number of elements
  - **Y** is a temporary variable (to store deleted value)
- 

#### Steps:

##### Step 1. [check for underflow]

```
c
if (F == 0)
    then write ("QUEUE UNDERFLOW")
    return (0)
```

**Step 2.** [delete element]

```
c  
  
Y = Q[F]
```

**Step 3.** [check for empty queue after deletion]

```
c  
  
if (F == R)  
    then F = 0  
        R = 0
```

**Step 4.** [increment front]

```
c  
  
if (F != N)  
    then F = F + 1  
else  
    F = 1
```



```
c  
  
return (Y)
```

---

**Description Box:**

- In **Step 1**, check for the queue underflow.
- **Step 2**, deleting an element.
- **Step 3**, check if the queue is now empty.
- **Step 4**, increment the front pointer after deletion.

**3.5 APPLICATIONS OF QUEUE**

- A **queue** is the natural data structure for a system to serve its incoming requests.
- Most **process scheduling algorithms** in operating systems use queues.
- Queues are used to **schedule various jobs, tasks, and processes** for their execution by the **CPU**.
- **Simulation** is another application of queues—it allows experimenting without modifying the original situation.
- **Circular queues** are used for **round robin scheduling** in operating systems.

| Simple Queue (Linear Queue)                                   | Circular Queue                                         |
|---------------------------------------------------------------|--------------------------------------------------------|
| FRONT pointer does not point to the REAR pointer.             | In it, FRONT points to the REAR pointer.               |
| There is a chance of <b>wastage of memory</b> .               | No wastage of memory in circular queue.                |
| We can't insert data even if some free space is available.    | We can insert data as long as free space is available. |
| Problem of overflow occurs <b>frequently</b> .                | Problem of overflow occurs <b>rarely</b> .             |
| Performance is <b>inefficient</b> compared to circular queue. | Works <b>better</b> than simple queue.                 |

| STACK                                                                         | QUEUE                                                                                    |
|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| In stack, insertion and deletion operations are performed at <b>one end</b> . | In queue, insertion and deletion operations are performed at <b>two different ends</b> . |
| Stack is also called <b>LIFO</b> (Last In First Out).                         | Queue is also called <b>FIFO</b> (First In First Out).                                   |
| Stack requires only <b>one pointer</b> called TOP.                            | Queue requires <b>two pointers</b> called FRONT and REAR.                                |
| <b>Example:</b> Pile of tray in cafeteria.                                    | <b>Example:</b> Movie ticket window line.                                                |
| In stack, there is <b>no wastage of memory</b> .                              | In queue, even if we have free space, we can't use it. So <b>wastage of memory</b> .     |
| <b>Application:</b> Stack machine, reverse polish notation.                   | <b>Application:</b> Simulation, job scheduling in operating system.                      |

## UNIT -4: Linked List

### 4.1 POINTERS REVISION:

- Pointer is a variable that stores the address of another variable.
- Pointer is a derived data type.
- Like any variable or constant, you must declare a pointer before using it to store a variable address.
- It is declared using \* operator.

To declare the pointer:

yaml

Syntax

data\_type \*variable\_name

Ex: int \*ptr

- We can also initialize the pointer at the time of declaration, like:

c

```
int n = 10;
int *ptr = &n;    // here, pointer 'ptr' points to the variable n.
```

CSS

n

10 ← Value of the variable

↑

2000 ← Address of the variable

ptr

2000 ← Value of the pointer variable

↑

5000 ← Address of the pointer variable

In the above figure, we can see that the value of the pointer is the address of another variable.

### Linked List

- The \* operator is used for "value at address" notation and & operator is used as "address of" the variable.

- \* is also called **dereferencing operator** or **indirectional operator**.
  - Pointer is very much useful while creating linked list.
- 

#### **Advantages of pointer are**

- It enhances the capability of the programming.
  - It reduces the size of the programming.
  - Pointer is used for creating complex data structures like linked list, trees, and graphs.
  - It increases the execution speed of the program.
  - It allows dynamic memory allocation (DMA).
- 

#### **Disadvantage of pointers are**

- Pointer variable is slower than normal variable.
- It is complex to understand.
- It is difficult to debug the program of pointer.
- Programmer needs to take more care at the time of using pointer.

## Print value and address of variable with and without using pointer

```
c

#include <stdio.h>
#include <conio.h>

void main()
{
    int i, *ptr;
    clrscr();
    i = 10;
    ptr = &i; // pointer initialize

    printf("\nThe Value of variable i is : %d", i);
    // Without using pointer

    printf("\nThe Value of variable i is : %d", *ptr);
    // With use of pointer

    printf("\nThe Address of variable i is : %u", &i);
    // Without using pointer

    printf("\n\nThe Address of variable i is : %u", ptr);
    // With use of pointer

    getch();
}
```

## Output

```
less

The Value of variable i is : 10
The Value of variable i is : 10
The Address of variable i is : 25545
The Address of variable i is : 25545
```

## Revision of Structure:

The text at the bottom briefly explains structures in C programming:

- **Structure:** A structure is a collection of logically related data items of different data types, known by a single name.
- To access the members of a structure, a structure variable must be created.

### Syntax (Prototype) of a Structure:

A structure in C is defined using the `struct` keyword. The basic syntax for a structure is:

```
c
struct structure_name
{
    data_type var1;
    data_type var2;
    // more members
};
```

### Declaration of structure variables

```
c
struct structure_name structure_varname1, structure_varname2, ... ;
```

We can create or declare structure variables in 3 different ways:

- With template itself
- Anywhere in the program
- Array of structures

By using the **dot operator (.)** with a structure variable, we can access the structure members.

```

#include <stdio.h>
#include <conio.h>

struct student
{
    int roll;
    char name[10]; // Structure members
};

void main()
{
    struct student s1; // Structure variable s1 is created

    clrscr();
    printf("Enter Roll no: ");
    scanf("%d", &s1.roll);

    printf("\nEnter name: ");
    scanf("%s", s1.name);

    printf("\n\nRoll no is: %d and name is: %s", s1.roll, s1.name);

    getch();
}

```

#### ✦ Explanation:

- struct student defines a template with members roll and name.
- s1 is a variable of type struct student.
- Using **dot operator (.)** → s1.roll, s1.name → to access members.

#### Output:

```

pgsql

Enter Roll no: 1
Enter name: BHAGYA
Roll no is: 1 and name is: BHAGYA

```

### 4.3 REVISION OF STRUCTURE USING POINTERS

- The way we can have a pointer pointing to an integer, float etc., similarly we can have a **pointer pointing to a structure**.
- The pointer to a structure is used to store the **address of the structure variable**.

Syntax to create pointer to structure:

```
c Copy code
struct structure_name *structure_pointer_variable;
```

- Generally, the `->` operator is used when we access structure members using a pointer to structure.

Example of 'structure using pointer':

```
c
#include <stdio.h>
#include <conio.h>

struct test
{
    int a;    // structure member
    int b;    // structure member
} i;         // structure variable is created with template

void main()
{
    int ans;
    struct test *ptr;    // pointer to structure

    clrscr();
    ptr = &i;

    printf("Enter two values: ");
    scanf("%d%d", &ptr->a, &ptr->b);    // access structure members using pointer

    ans = ptr->a + ptr->b;
    printf("\n\nAnswer is %d", ans);

    getch();
}
```

## Output:

```
pgsql
```

```
Enter two values: 5 10
```

```
Answer is 15
```

### ★ Explanation:

- `struct test` defines two members: `a` and `b`.
- `ptr` is a pointer to the structure.
- `ptr = &i;` assigns the address of `i` to the pointer.
- Accessing members with pointer uses the **arrow operator** (`->`).

## 4.4 DYNAMIC MEMORY ALLOCATION (DMA)

Allocation of memory is an important part of program execution.

There are two types of memory allocations:

### 1. Static Memory Allocation

- Memory is allocated **at compile time**.
- Once allocated, it **cannot be changed** during program execution.
- Example: arrays.
- Limitation: If size is underestimated → overflow; if overestimated → wastage of memory.

### 2. Dynamic Memory Allocation (DMA)

- Memory is allocated **at runtime**.
- Very useful when we don't know in advance how much memory is required.
- Allocates memory from the **heap area**.
- Provides flexibility to manipulate storage according to requirements.
- Extensively used in **linked list, trees, graphs**, etc.
- Advantages:
  - Efficient use of memory.
  - No wastage of memory.
  - Flexible and manageable.
- Disadvantages:
  - Slightly slower due to runtime allocation.
  - Programmer must free unused memory to avoid **memory leaks**.

## Functions used in Dynamic Memory Allocation

| Function | Use of function                                                                 |
|----------|---------------------------------------------------------------------------------|
| malloc() | Allocates requested size of bytes and returns a pointer to first byte of memory |

| Function  | Use of function                                                                 |
|-----------|---------------------------------------------------------------------------------|
| calloc()  | Allocates space for array elements, initializes them to zero, returns a pointer |
| free()    | Releases previously allocated memory                                            |
| realloc() | Changes the size of previously allocated memory                                 |

### malloc() function

- **Full form:** memory allocation.
- Syntax:

```
c
pointer_variable = (target_type*) malloc(number_of_bytes);
```

- Example:

```
c
int *p;
p = (int*) malloc(sizeof(int));
```

- Allocates requested size of bytes and returns the pointer to the first byte.
- Returns `NULL` if memory allocation fails.

### calloc() function

- **Full form:** contiguous allocation.

Syntax:

```
c
pointer_variable = (target_type*) calloc(num_elements, size_of_each_element);
```

### Difference between malloc() and calloc()

- The only difference is:
  - malloc() allocates a single block of memory.
  - calloc() allocates multiple blocks of memory, **each initialized to zero.**

## Syntax for `calloc()`

c

```
pointer_variable = (target_type *)calloc(n, number_of_bytes);
```

### ◆ Example:

c

```
int *p;  
p = (int*)calloc(25, sizeof(int));
```

This allocates contiguous space in memory for an array of **25 elements** of type `int` .  
Each element (size = 2 bytes) is initialized to **zero**.

## `free()` function

- Dynamically allocated memory (using `malloc()` or `calloc()` ) is **not freed automatically**.
- You must explicitly use `free()` to release the memory.

### Syntax:

c

```
free(pointer-variable);
```

This statement frees the space allocated in the memory pointed by the pointer.

## `realloc()` function

- Full form: **re-allocation**.
- Used to resize already allocated memory (increase or decrease size).

### Syntax:

c

```
pointer-variable = realloc(pointer-variable, newsize);
```

Here, the pointer variable is reallocated with the new size.

## Example of DMA (using `malloc()` function)

C program to create memory for int, char, and float variable at run time:

```
c

#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

void main()
{
    int *intVar;
    char *charVar;
    float *floatVar;

    clrscr();

    intVar = (int*) malloc(sizeof(int));
    charVar = (char*) malloc(sizeof(char));
    floatVar = (float*) malloc(sizeof(float));

    printf("Enter integer value: ");
    scanf("%d", intVar);

    printf("\nEnter character value: ");
    scanf(" %c", charVar);

    printf("\nEnter float value: ");
    scanf("%f", floatVar);

    printf("\nInput values are: %d, %c, %.2f", *intVar, *charVar, *floatVar);

    free(intVar);    // free allocated memory
    free(charVar);
    free(floatVar);

    getch();
}
```

## Output

```
sql
```

```
Enter integer value: 10
```

```
Enter character value: A
```

```
Enter float value: 3.50
```

```
Input values are: 10, A, 3.50
```

### Advantages of DMA

1. Memory allocation at run time.
2. No wastage of memory.
3. Faster execution speed.
4. Data can be rearranged efficiently.
5. Memory blocks which are no longer required can be freed.
6. Mainly used for creating complex data structures (like linked list, tree, etc.).
7. In DMA, program gets memory **directly from the operating system** (not from the compiler).

### Comparison: Static vs Dynamic Memory Allocation

| Static memory allocation                                                                    | Dynamic memory allocation                                                                         |
|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Memory allocation at compile time.                                                          | Memory allocation at run time.                                                                    |
| There should be wastage of memory.                                                          | No wastage of memory.                                                                             |
| Used when required amount of memory is known in advance.                                    | Used when memory utilization is not known in advance.                                             |
| Slower execution speed than DMA.                                                            | Faster execution speed.                                                                           |
| Data rearrangement is not efficient.                                                        | Data rearrangement is efficient.                                                                  |
| <b>Example of Static Allocation:</b><br><pre>c int a[5]; // allocates 10 bytes (5 int</pre> | <b>Example of Dynamic Allocation:</b><br><pre>c int *ptr; ptr = (int*) malloc(sizeof(int));</pre> |

## 4.5 LINKED LIST PRESENTATION

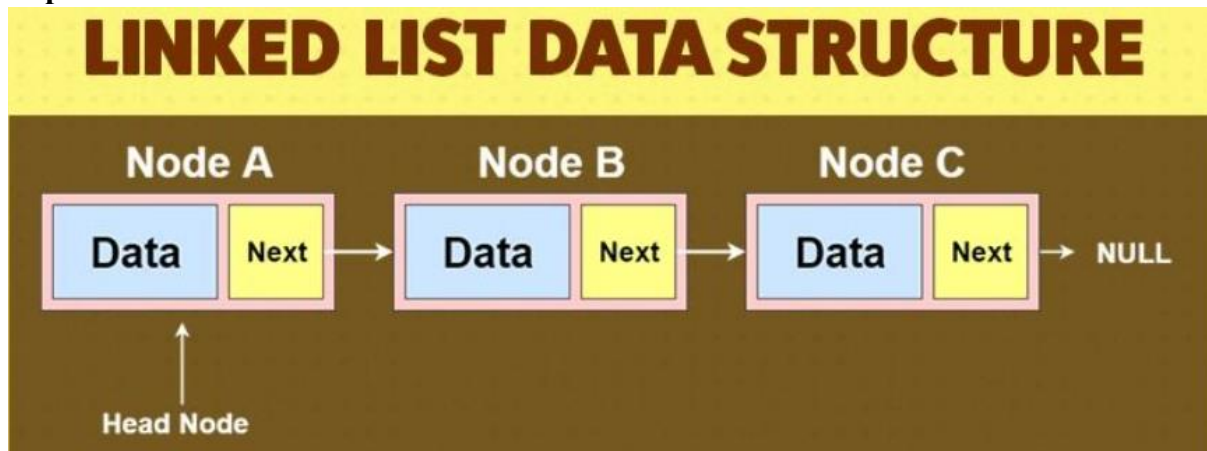
A **Linked List** is a dynamic linear data structure that consists of groups of nodes.

Each **node** contains two fields:

1. **DATA field** → stores the actual data.

2. **LINK field** → stores the pointer to the next node.

### Representation of Linked List



(Think of it like a train with compartments: each coach = node, chain = link, engine = FIRST pointer.)

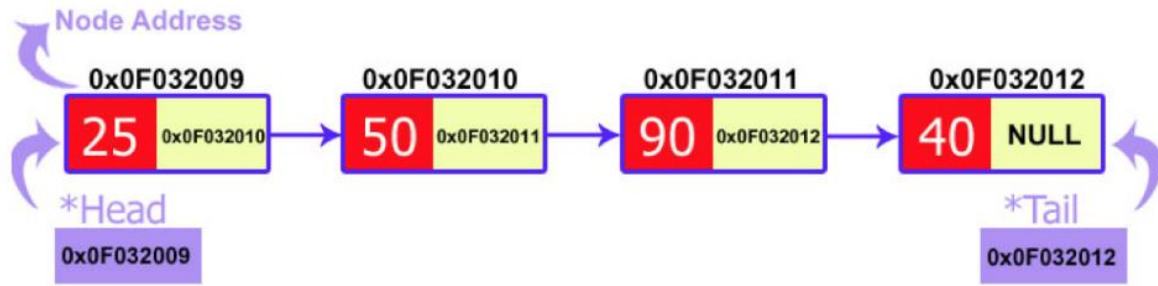
### Important Points:

- In a linked list, elements are logically adjacent but not necessarily physically adjacent in memory (unlike arrays).
- FIRST is a pointer that stores the address of the first node.
- The LINK field of the last node stores NULL (indicating end of list).
- Each element (node) consists of:
  - DATA (actual value)
  - LINK (address of the next node)

### C Structure for Linked List Node

```
c
struct node {
    int DATA;
    struct node *LINK; // self-referential pointer
};
```

- Here, **LINK** is a pointer pointing to another node of the same structure type.
- This makes linked list a **self-referential structure**.
- Each node contains link OR pointer to the next node.
- A linked list is a looking list (this might be a typo — likely meant to say "linked list").



### Advantages of Linked List

- It is a dynamic allocation of memory and saves memory space.
- Insertion and deletion operations are easy and fast in it. Because it does not need shifting of elements in insertion and deletion.
- Various data structures like stacks and queues are easily implemented using a linked list.
- Complex data structures like tree and graph can be implemented using linked list.
- Linked list can reduce access time without memory overhead.
- Linked list can be extended or reduced as per requirements.

### Disadvantages of Linked List

- Address calculation is very difficult.
- Searching and sorting is time consuming and complex.
- No element can be accessed randomly; it has to access each node sequentially.
- Node in a linked list must be read in order from the beginning as linked lists are inherently sequential access.
- Reverse Traversing is difficult in linked list.

### Difference between Array and Linked list

| Array                                                                                            | Linked list                                                                                  |
|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Array is static in size.                                                                         | Linked list is dynamic in size.                                                              |
| In array, elements are stored in contiguous memory location or consecutive manner in the memory. | In a linked list, new elements can be stored anywhere in the memory.                         |
| Allocation of memory at compile time. So in case of array, it is static memory allocation.       | Allocation of memory at runtime. So in case of linked list, it is dynamic memory allocation. |
| Insertion and deletion operations are complex and take more time.                                | Insertion and Deletion operations are fast in linked list.                                   |
| Types of arrays are: single dimensional or multi dimensional array.                              | Types of linked list are: singly linked list, circular linked list, and doubly linked list.  |
| Searching is easy and takes less time.                                                           | Searching is time consuming.                                                                 |
| No need of pointer variable.                                                                     | Require pointer variable which occupy extra memory space.                                    |

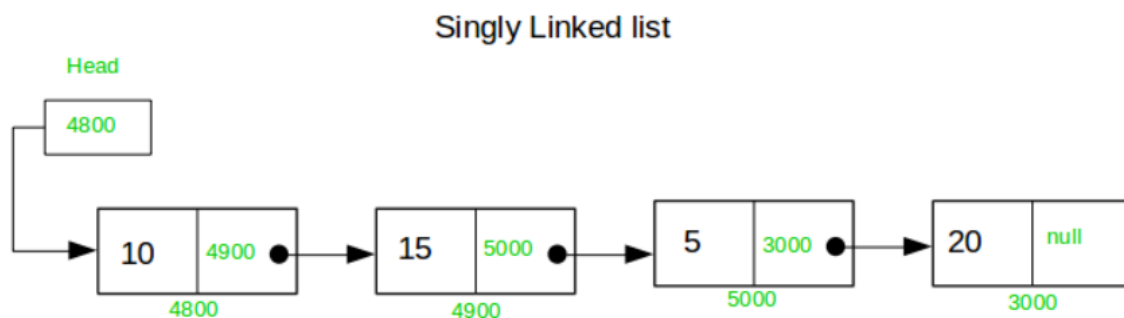
| Array                                             | Linked list                                                                                 |
|---------------------------------------------------|---------------------------------------------------------------------------------------------|
| Random access is allowed.                         | Random access is not allowed. We must have to traverse the linked list from the first node. |
| Array gets memory allocated in the Stack section. | Linked list gets memory allocated in Heap section.                                          |

#### 4.6 TYPES OF LINKED LIST:

Following are the types of linked list:

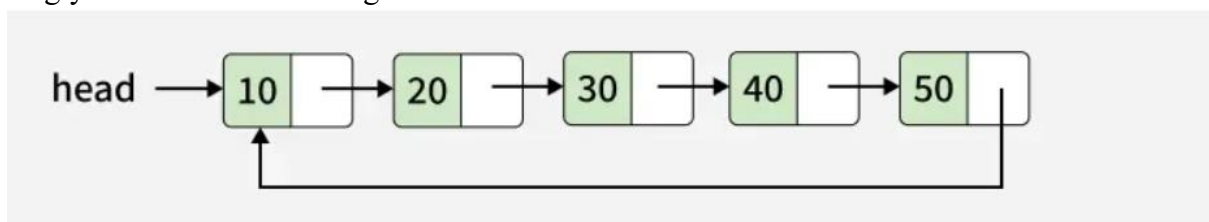
##### 1. Singly Linked List

In singly linked list, there is one link per node. Each node contains two fields, namely DATA and LINK. INFO / DATA field contains the data and LINK field contains link or pointer to next node. First node is a pointer which points to the first node and the link field in the last node is NULL. Figure of singly linked list is shown below:



##### 2. Singly Circular Linked List

A linked list in which the last node contains a link or pointer to the first node is known as circular linked list. So, first node and last node are adjacent logically. Representation of singly circular linked list is given below:



##### 3. Doubly Linked List

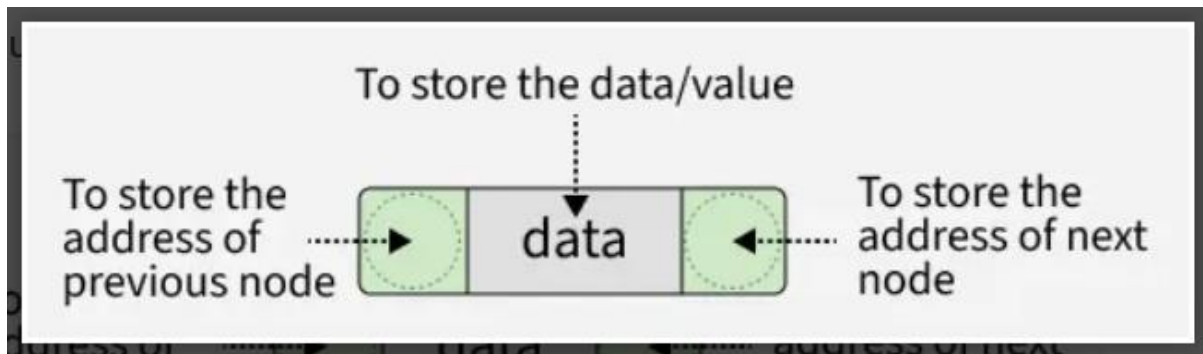
A linked list in which each node contains three fields:

1. DATA (it contains actual data),
2. LPTR (left pointer points to the predecessor node),
3. RPTR (right pointer points to the successor node)

So, the definition of the doubly linked list can be written as →

**"A list in which each node contains two links one to the predecessor of the node and one to the successor of the node"** is known as doubly linked list or two-way chains.

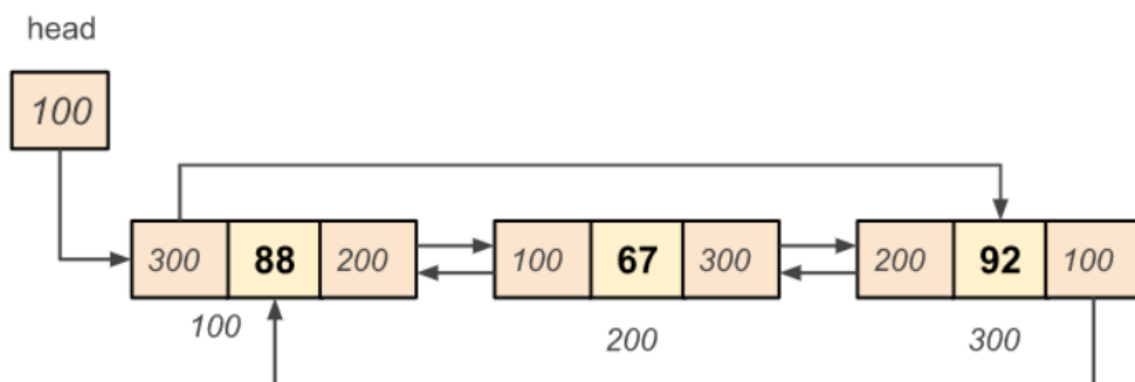
Representation of doubly linked list is shown in below figure:



#### 4. Circular Doubly Linked List

Circular doubly linked list has properties of both doubly linked list and circular linked list. A list in which every node is linked with next and previous node and the right pointer of last node links to first node and the left pointer of first node links to last node.

Representation of circular doubly linked list is shown in below figure:



#### 4.7 BASIC OPERATIONS ON SINGLY LINKED LIST

Different operations that can be performed on linked list are:

- **Create a linked list:**  
This operation is used to create a new node as and when required.
- **Traversing a linked list:**  
It is a process of accessing all nodes of the linked list once.
- **Insert a node at beginning (at first position):**  
Inserting a new node at the first position of the linked list.
- **Insert a node at end:**  
Inserting a new node at the last position of the linked list.
- **Insert a node at any position:**  
Inserting a new node at a specific position in the linked list.
- **Insert a node in ordered linked list:**  
Inserting a new node in the linked list which is ordered either in ascending or descending.
- **Delete a node from beginning:**  
This operation is used to delete a node from the first position in the linked list.
- **Delete a node from end:**  
This operation is used to delete a node from the last position in the linked list.
- **Delete a node (at any position):**  
Deleting a node at the specific address from the linked list.

- **Searching a node (at any position):**  
Search a specific node from the given linked list.
- **Counting a node:**  
The process of counting number of nodes in given linked list.

### 7.1 Insert a new node at the beginning of the linked list

- When we want to insert a new node, first we have to check whether a free node is available in the memory or not.
- The detailed algorithm to insert a node at beginning of the linked list with description is given below:

#### Algorithm: INSERT\_START\_SINGLY

This algorithm inserts a new node NEW with value X at the beginning of the linked list.

- FIRST is a pointer to the first element of the linked list which contains DATA and LINK fields.
- NEW is a pointer variable which denotes the new node.

#### Algorithm (continued): INSERT\_START\_SINGLY

##### Step 1: (Obtain new node from memory)

```
c
NEW ← NODE           // obtain a new node from memory using DMA
NEW = (struct NODE*) malloc(sizeof(struct NODE))

if (NEW == NULL)
    print("NOT ENOUGH MEMORY")
    return()
```

### Step 2: *(Initialize new node)*

```
c  
  
DATA(NEW) = X  
LINK(NEW) = NULL
```

---

### Step 3: *(Check for empty list)*

```
c  
  
if (FIRST == NULL)  
    return (NEW)
```

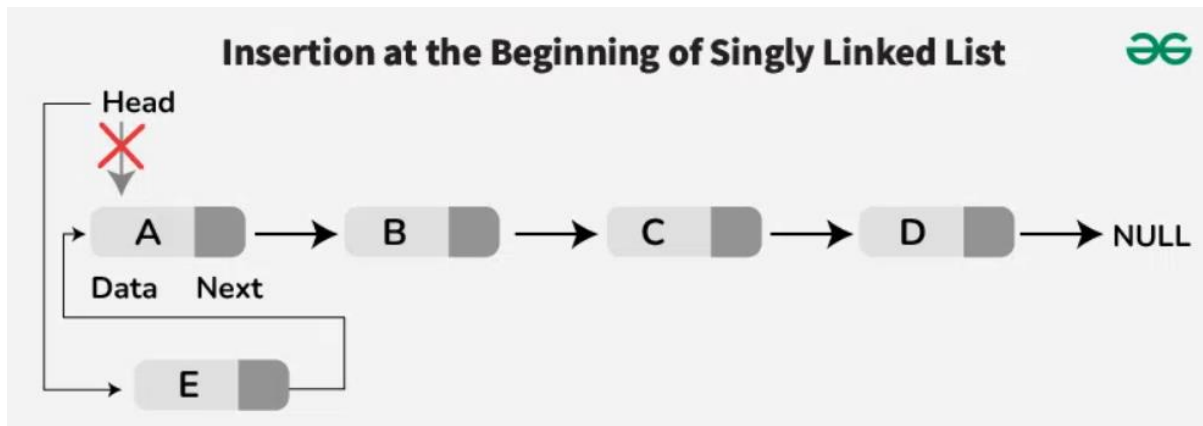
### Step 4: *(When list is not empty, insert the new node at beginning)*

```
c  
  
LINK(NEW) = FIRST  
FIRST = NEW  
return (FIRST)
```

#### **Description:**

- In the first step, checking for the free memory for a new node. If enough memory is available, then obtain a free node using DMA (malloc() function).
- After initializing the new node in step 2, in step 3 we are checking if the list is empty or not.
- If the list is not empty, in step 4 the new node is inserted at the beginning of the linked list.

#### **Example**

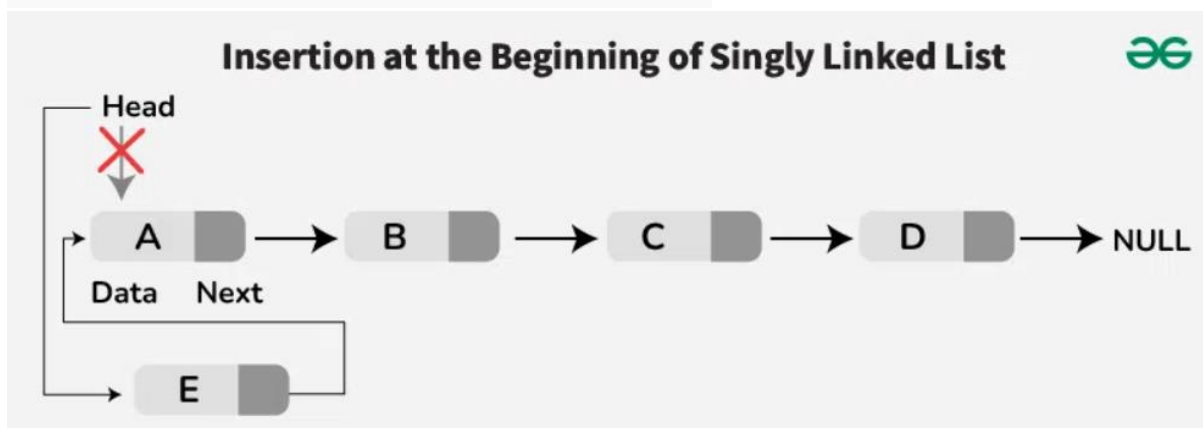


When we get a new node from available memory, it is ready to be inserted at the beginning of the linked list.

When we write:

C

```
DATA(NEW) = X,
LINK(NEW) = FIRST,
FIRST = NEW
```



Now, our new linked list is ready with the new node inserted at the starting position.

### To Insert a Node at the End of the Linked List

#### Algorithm: INSERT\_END\_SINGLY

This algorithm inserts a new node NEW with value X at the end of a singly linked list.

- FIRST is a pointer to the first element of the linked list which contains DATA and LINK fields.
- NEW and SAVE are pointer variables.
- NEW denotes the new node.

### Step 1: *(Obtain new node from memory)*

```
c
// Obtain a new node from memory using DMA
NEW ← NODE
NEW = (struct NODE*) malloc(sizeof(struct NODE))

if (NEW == NULL)
    print("NOT ENOUGH MEMORY")
    return()
```

 Copy code

### Step 2: *(Initialize new node)*

```
c
DATA(NEW) = X
LINK(NEW) = NULL
```

 Copy code

### Step 3: *(Check for empty list)*

```
c
if (FIRST == NULL)
    return (NEW)
```

### Step 4: *(Initialize SAVE and search for end of the list)*

```
c
SAVE = FIRST
repeat while (LINK(SAVE) ≠ NULL)
    SAVE = LINK(SAVE)
```

### Step 5: *(Set LINK field of last node to NEW)*

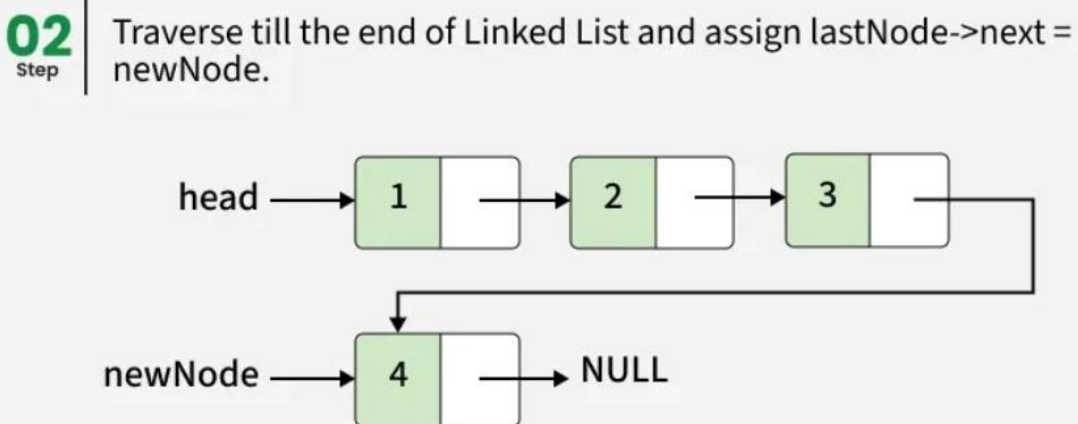
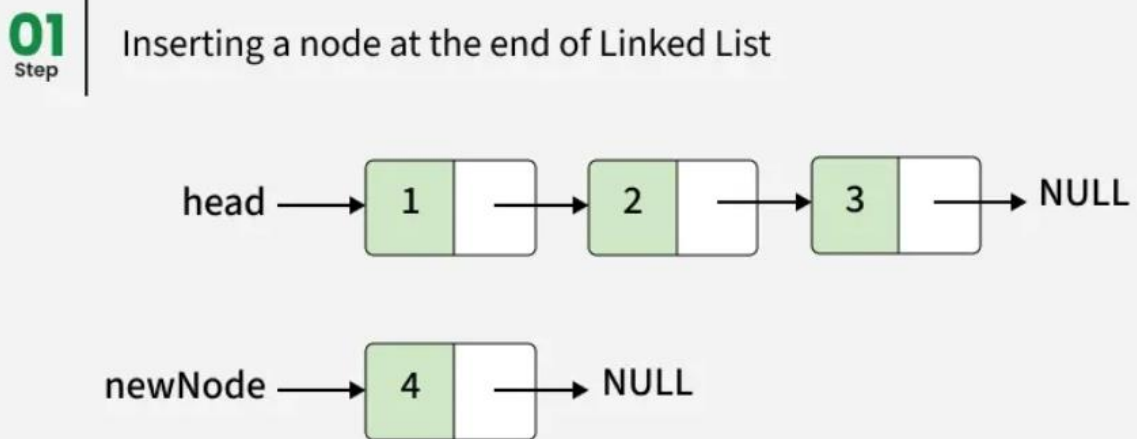
```
c
LINK(SAVE) = NEW
return (FIRST)
```

## Description

- In the first step, we check for free memory for the new node. If enough memory is available, then obtain a free node using DMA (malloc() function).
- In step 2, the new node is initialized.
- In step 3, we check whether the list is empty.
- In step 4, we initialize the SAVE pointer and search for the end of the list.
- When the last node is reached, in step 5, the NEW node is inserted at the end.

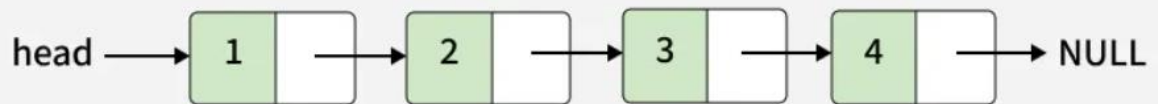
## Figurative Description

- Up to step 2, we have seen the algorithm. In step 4 we write:  
 $\text{SAVE} \leftarrow \text{FIRST}$ , so SAVE points to the first node.
- In step 5, we traverse the linked list until the last node (where the LINK part is NULL).
- At that time, the situation looks like this:



**03**  
Step

Linked List after insertion at the end.



#### 4.7.3 Insert a node before the given node

We can insert a new node in the linked list before any given node.

Algorithm: INSERT\_POS\_BEFORE\_SINGLY

This procedure inserts a new node NEW into the linked list before the node which is at address ADD.

- FIRST is a pointer to the first element of the linked list which contains DATA and LINK.
- NEW and SAVE are temporary pointer variables. NEW denotes the new node.
- PRED is a pointer pointing to the predecessor node.

### Step 1: (obtain new node from memory)

objectivec

```
NEW ← NODE           // obtain a new node from memory using DMA
[ NEW = (struct NODE*) malloc(sizeof(struct NODE)) ]
if (NEW = NULL)
    then print ("NOT ENOUGH MEMORY")
    return()
```

### Step 2: (initialize new node)

sql

```
DATA(NEW) = X
LINK(NEW) = NULL
```

### Step 3: (check for empty list)

sql

```
if (FIRST = NULL)
    then return (NEW)
```



### Step 4: (if only one node is there in list)

sql

```
if (LINK(FIRST) = NULL OR ADD = FIRST)
    then LINK(NEW) = FIRST
    FIRST = NEW
    return (FIRST)
```

### Step 5: (initialize SAVE and search the list until desired node ADD found)

r

```
SAVE = FIRST
repeat while (LINK(SAVE) ≠ NULL AND SAVE ≠ ADD)
    PRED = SAVE
    SAVE = LINK(SAVE)
```

## Step 6: (Insert node when we find targeted node ADD)

sql

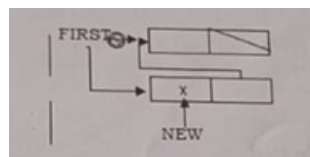
```
LINK(PRED) = NEW
LINK(NEW) = SAVE    // OR    LINK(NEW) = ADD
return (FIRST)
```

### Description

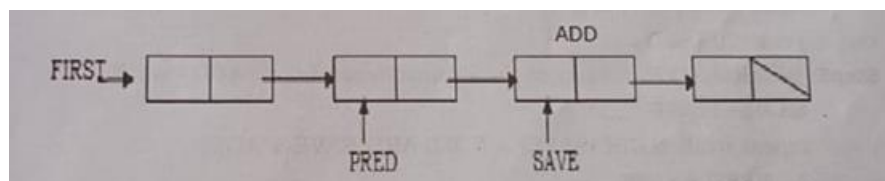
- In first step, checking for the free memory for new node. If enough memory is available then obtain a free node using DMA (malloc() function).
- In step 2, initializing new node. Then in step 3 we are checking for the empty list.
- In step 4, we are checking whether only one is there in the list.
- In step 5, initialize SAVE pointer and search for target node ADD where we want to insert NEW node and in step 6, insert NEW node at proper place.

### Figurative description

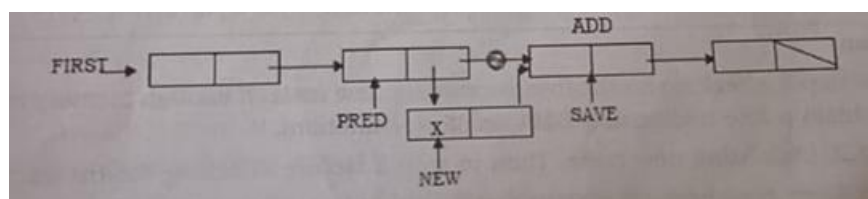
- We already know what happens up to step 3.
- In step 4, if we have only one node or the X node is the first node, then NEW node is inserted at the starting position of the list. Like:



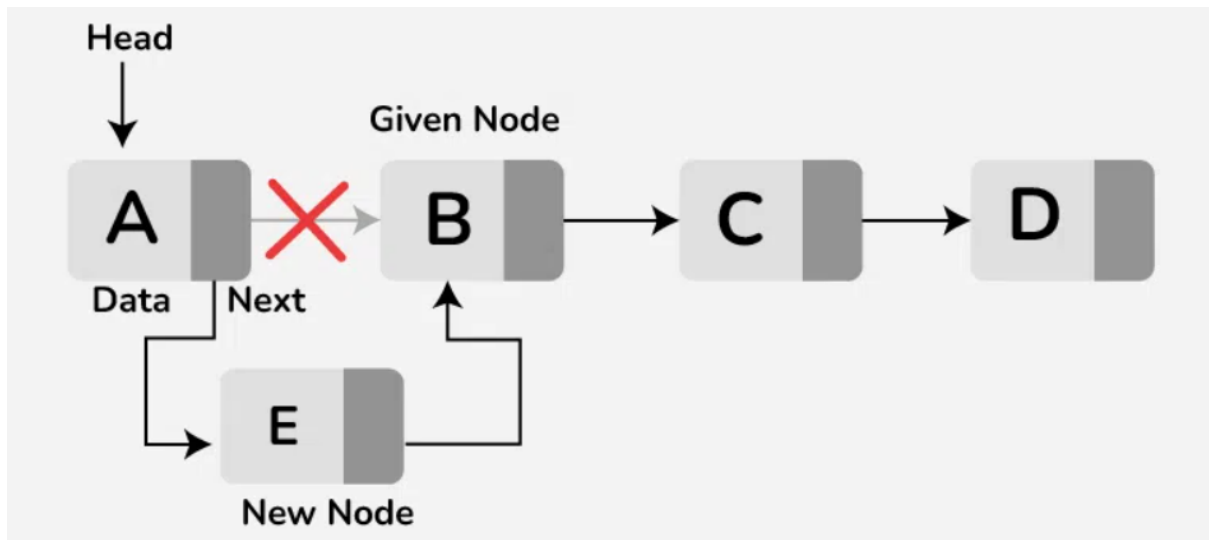
- In step 5, using two temporary pointers PRED and SAVE, we are finding our targeted node X up to the end of the list. Suppose the node ADD is there at position 3, so the situation of PRED and SAVE pointer will be like:



- After finding the targeted node, in step 6 we are writing:  
 $LINK(PRED) \leftarrow NEW$  and  $LINK(NEW) \leftarrow SAVE$ .  
At that time the NEW node is inserted at the specified position ahead of node X. Like:



Now our linked list is ready with a NEW node inserted at the position specified by ADD.



#### 4.7.4 Insert a node after given node

We can insert a new node in the linked list after any given node.

---

##### Algorithm: INSERT\_POS\_AFTER\_SINGLY

This procedure inserts a new node NEW into linked list after node which is specified by address ADD.

- FIRST is a pointer to the first element of linked list which contains DATA and LINK fields.
- NEW and SAVE are temporary pointer variables. NEW denotes new node.

##### Step 1: [Obtain new node from memory]

```

c
NEW <- NODE    // obtain a new node from memory using DMA
[ NEW = (struct NODE*) malloc(sizeof(struct NODE)) ]
if (NEW == NULL)
    then print ("NOT ENOUGH MEMORY")
    return ()
  
```

##### Step 2: [Initialize new node]

```

c
DATA(NEW) = X
LINK(NEW) = NULL
  
```

### Step 3: [Check for empty list]

```
c

if (FIRST == NULL)
    then return (NEW)
```

---

### Step 4: [If only one node is there in list]

```
c

if (LINK(FIRST) == NULL)
    then LINK(FIRST) = NEW
        LINK(NEW) = NULL
        return (FIRST)
```

### Step 5: [Initialize SAVE and search the list until desired node ADD found]

```
c

SAVE = FIRST
repeat while (LINK(SAVE) ≠ NULL AND SAVE ≠ ADD)
    PRED = SAVE
    SAVE = LINK(SAVE)
```

---

### Step 6: [Insert node when we find targeted node ADD]

```
c

LINK(NEW) = LINK(SAVE) // OR LINK(NEW) = LINK(ADD)
LINK(SAVE) = NEW      // OR LINK(ADD) = NEW
return (FIRST)
```

#### Description:

- In step 1, checking for the free memory for new node. If enough memory is available, then obtain a free node using DMA (malloc() function).
- In step 2, initializing new node. Then in step 3, we are checking for the empty list.
- In step 4, we are checking whether only one node is there in the list.

- In step 5, initialize SAVE pointer and search for target node ADD where we want to insert NEW node.
- In step 6, insert NEW node at proper place.

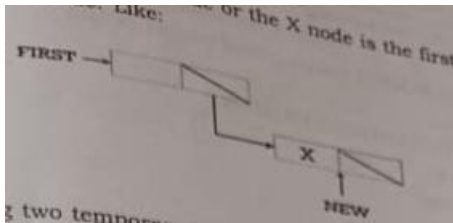
### Figurative Description

We already know what happens up to step 3.

In step 4, if we have only one node or the X node is the first node, then NEW node is inserted after the first node. Like:

CSS

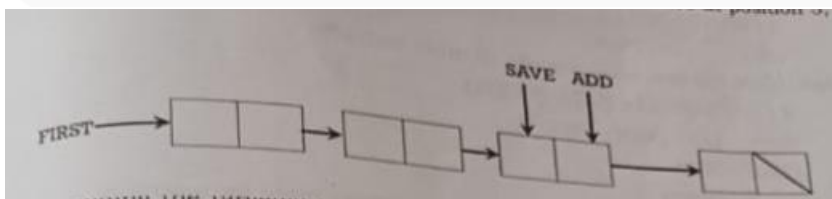
FIRST → [ ] → [X] → NEW



In step 5, using two temporary pointers PRED and SAVE, we are finding our targeted node X up to the end of the list. Suppose the node ADD is there at position 3, so the situation of SAVE pointer will be like:

CSS

FIRST → [ ] → [ ] → [SAVE/ADD] → [ ]

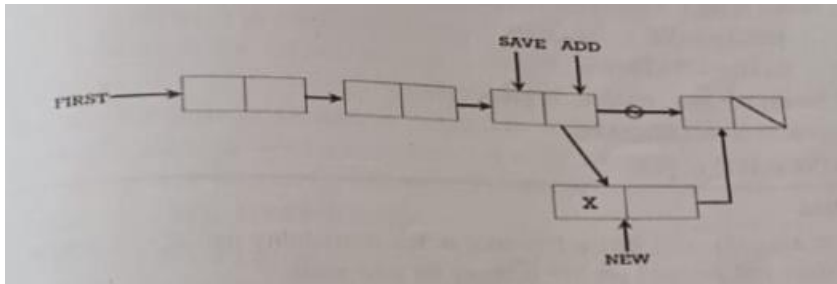


After finding the targeted node, in step 6 we are writing:

C

LINK(NEW) = LINK(SAVE)

LINK(SAVE) = NEW



At that time, the NEW node is inserted at the specified position after node X. Like:

CSS

FIRST → [ ] → [ ] → [SAVE/ADD] → NEW → [ ]

Now our linked list is ready with a NEW node inserted at the position specified by ADD.

#### 4.7.5 Insert a node in sorted (ordered) linked list

- In sorted linked list, all the data in the list are arranged in a particular order (either ascending or descending). It is also called ordered linked list.
- We have to find the proper position where the new node is to be inserted.
- For that, we have to compare the value in the DATA field of each node.

---

#### Algorithm: INSERT\_ORD\_SINGLY

This algorithm inserts a new node in ordered (sorted) linked list at the proper position.

- X is a value of the NEW node to be inserted.
- FIRST is a pointer pointing to the first node of the linked list.

NEW and SAVE are temporary pointer variables. NEW denotes new node.

### Step 1: [Obtain new node from memory]

C

```
NEW ← NODE    // obtain a new node from memory using DMA
[ NEW = (struct NODE*) malloc(sizeof(struct NODE)) ]
if (NEW == NULL)
    then print("NOT ENOUGH MEMORY")
    return()
```

---

### Step 2: [Initialize new node]

C

```
DATA(NEW) = X
LINK(NEW) = NULL
```

### Step 3: [Check for empty list]

C

```
if (FIRST == NULL)
    then return (NEW)
```

---

### Step 4: [Does the new node precede all other nodes?]

C

```
if (DATA(NEW) ≤ DATA(FIRST))
    then LINK(NEW) = FIRST
    FIRST = NEW
    return (FIRST)
```

### Step 5: [Initialize SAVE pointer and search the proper place for insertion]

C

```
SAVE = FIRST
repeat while (LINK(SAVE) ≠ NULL AND DATA(LINK(SAVE)) ≠ DATA(NEW))
    PRED = SAVE
    SAVE = LINK(SAVE)
```

### Step 6: [Insert the new node at its proper place]

C

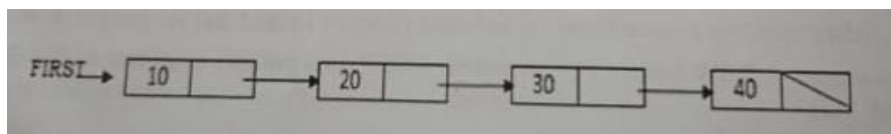
```
LINK(NEW) = LINK(SAVE)
LINK(SAVE) = NEW
```

#### Description

- In step 1: Check for free node in the availability list. If unavailable, availability list is underflow and memory can't be allocated for the new node.
- In step 2: Get a free node and initialize the fields.
- In step 3: Check whether the list is empty.
- In step 5: Initialize SAVE pointer and search for the proper place of insertion.
- In step 6: Insert NEW node at its proper place.

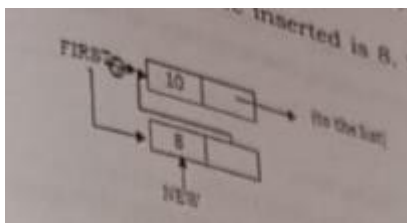
#### Figurative Description

- Suppose we have the ordered linked list as given below:



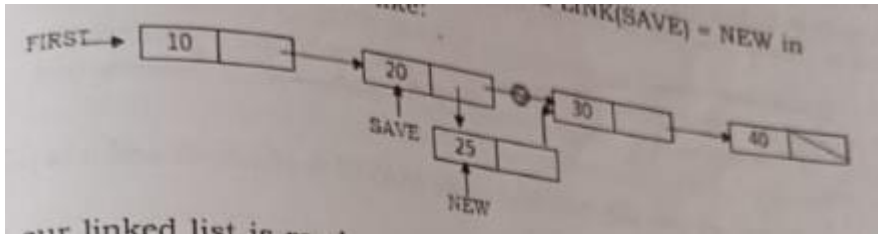
We already know what happens up to step 2. In step 3, if the list is empty then the **NEW node** will be the list.

- Suppose the value of the node to be inserted is 8.  
So it will be inserted at the first position in the list. Like:



- Suppose we want to insert a node having value **25**.  
In step 5 we are finding the position to insert a node in the list.
- When we write:  
 $\text{LINK}(\text{NEW}) = \text{LINK}(\text{SAVE})$  and  
 $\text{LINK}(\text{SAVE}) = \text{NEW}$  in step 6.

The situation looks like:



- Now our linked list is ready with a node **25** inserted at its proper position in ordered linked list.

#### 4.7.6 Delete a node from the starting position (first node) of the linked list

- Whenever we want to delete a node, we first have to check whether the list is empty or not.  
Because if the list is empty, it is an **underflow situation**, and we can't perform delete operation.
- When a node is deleted, it is inserted back into the availability list — meaning we must **release or free the memory** occupied by the deleted node.  
Using `free()` function, we can free the occupied memory.

---

#### Algorithm: DELETE\_FIRST\_SINGLY

This procedure deletes a node from the beginning of a **singly linked list**.

- `FIRST` is a pointer to the first node of linked list.
- `SAVE` is a temporary pointer variable which is to be deleted.

##### Step 1: [Checking for empty list]

```
c
if (FIRST == NULL)
    then print("LIST IS EMPTY AND UNDERFLOW")
    return()
```

---

##### Step 2: [Initialize SAVE pointer]

```
c
SAVE = FIRST
```

### Step 3: [Delete first node and free the space of deleted node]

c

```
if (LINK(FIRST) == NULL) // when only one node is there in list
```

c

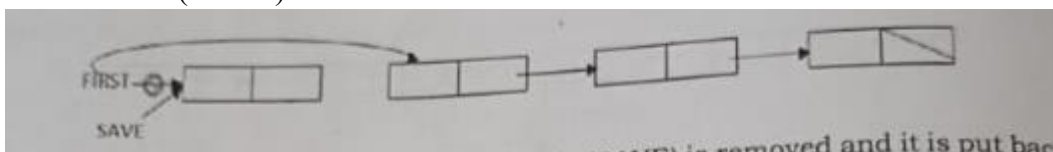
```
then
    FIRST = NULL
    free(SAVE)
    return(FIRST)
else
    FIRST = LINK(FIRST) // in case of more than one node
    free(SAVE)           // freeing the memory of deleted node
    return(FIRST)
```

#### Description:

- **Step 1:** Check for the list underflow.
- **Step 2:** Initialize SAVE pointer to the first node.
- **Step 3:** Delete the first node and release (or free) the memory of deleted node (SAVE).

#### Figurative Description:

- After **step 2** is executed, SAVE pointer is pointing to FIRST, and in **step 3**, if there is only one node in linked list, it is removed and put into availability list.
- If there are more than one node in the linked list, the situation after  $FIRST \leftarrow LINK(FIRST)$  will be:



- From above figure, we can see that the first node (SAVE) is removed and is put back in the memory using free(SAVE) statement.

#### ◆ 4.7.7 Delete a node from the end (last node) of the linked list

- When we want to delete a node at the **end**, we first need to reach the end of the linked list.
- We have already seen the logic for reaching the end of the list (using **SAVE** and **PRED** pointers).

#### Algorithm: DELETE\_END\_SINGLY

This procedure deletes a node from the **end** of a **singly linked list**.

- FIRST is a pointer to the first node of the linked list.
  - SAVE and PRED are temporary pointer variables.  
PRED keeps track of the predecessor node.  
SAVE points to the node to be deleted.
- 

### Step 1: [Checking for empty list]

C

```
if (FIRST == NULL)
    then print("LIST IS EMPTY AND UNDERFLOW")
        return()
```

---

### Step 2: [Initialize SAVE pointer]

C

```
SAVE = FIRST
```

---

### Step 3: [Search target node]

C

```
if (LINK(FIRST) == NULL)    // if only one node in list
    then
        FIRST = NULL
        free(FIRST)
else
    repeat while (LINK(SAVE) ≠ NULL)
```

c

```
PRED = SAVE  
SAVE = LINK(SAVE)
```

Step 4: [when we find the last node, delete it and free the space of deleted node]

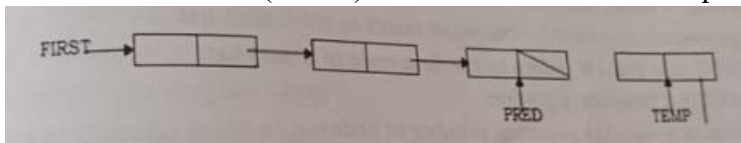
```
LINK(PRED) = NULL    // remove the last node  
free(SAVE)           // freeing the memory of deleted node  
return(FIRST)
```

#### Description:

- **Step 1:** Check for underflow.
- **Step 2:** SAVE variable is pointing to FIRST node.
- **Step 3:** Search for target node.
  - If only one node is there, it is removed.
  - Else use SAVE and PRED pointers to reach the last node.
- **Step 4:** Delete the last node and free the memory of deleted node (SAVE).

#### Figurative Description:

- After step 2, SAVE is pointing to FIRST.
- In step 3, if there's only one node, it's removed and memory is freed (using free(SAVE)).
- If there are multiple nodes, use SAVE and PRED to reach the **last node**, where:
  - SAVE points to the last node.
  - PRED points to the second-last node.
- When  $\text{LINK}(\text{PRED}) \leftarrow \text{NULL}$  is written in step 4, SAVE node is deleted.



- So now, the **last node** is removed and returned to the **availability list** using free(SAVE).

#### 4.7.8 Search a node in the singly linked list

##### Algorithm: SEARCH\_SINGLY

This algorithm searches the given node (element) in the linked list.

- FIRST is a pointer to the first node of the linked list.
- X is the element which we want to search.
- SAVE is a temporary pointer variable.
- POS is the variable showing the **position of the searched node**.

```

Step 1: [check for empty list]
    if (FIRST == NULL)
        then print("LIST IS EMPTY AND UNDERFLOW")
        return()

Step 2: [initialize SAVE pointer and set POS at first node]
    POS = 1
    SAVE = FIRST

Step 3: [Search target node]
    repeat while (DATA(SAVE) ≠ X AND LINK(SAVE) ≠ NULL)
        SAVE = LINK(SAVE)
        POS = POS + 1

    if (DATA(SAVE) == X)
        then
            print("Node is found at")
            print(POS)
            return()
    else
        print("Node is not found in the list")
        return()

```



#### Description:

- In **Step 1**, we check if the list is empty.
- In **Step 2**, initialize the POS variable and SAVE pointer. Set POS = 1.
- In **Step 3**, search for the target node. If found, display its position. Otherwise, show not found.

---

#### ◆ 4.7.9 Count the number of nodes in the linked list

##### Algorithm: COUNT\_SINGLY

This procedure counts the number of nodes in the linked list.

- FIRST is a pointer to the first node of the list.
- SAVE is a temporary pointer.
- count is a variable used to count the number of nodes.

c

Step 1: [check for empty list]

```
if (FIRST == NULL)
    then print("LIST IS EMPTY")
    return()
```

Step 2: [initialize SAVE pointer and set count variable to 1]

```
count = 1
SAVE = FIRST
```

Step 3: [process the linked list until end of the list and count nodes]

```
repeat while (LINK(SAVE) ≠ NULL)
    SAVE = LINK(SAVE)
    count = count + 1
```

[finished]

```
return(count)
```

#### Description:

- In **Step 1**, check for the empty linked list.
- **Step 2**: Initialize count variable and SAVE pointer.
- **Step 3**: Traverse until the end of the list and count the number of nodes.

#### CONCEPT OF CIRCULAR LINKED LIST

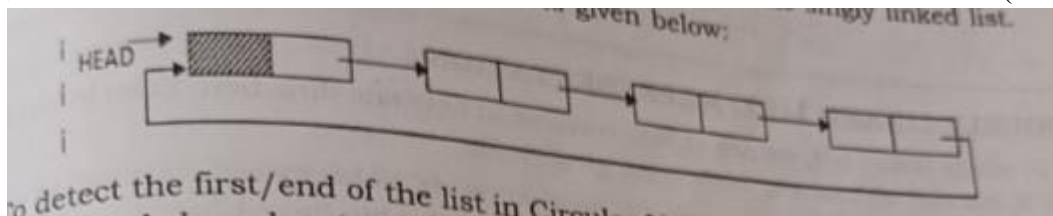
A **linked list** in which the last node contains a link (or pointer) to the **first node** is known as a **circular linked list**.

So, the first node and the last node are adjacent.

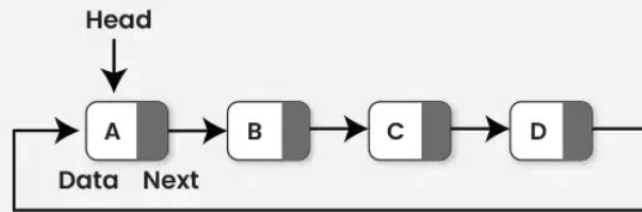
- The **link field** of the last node does **not** contain NULL like a singly linked list.

#### Representation:

✦ An illustration is shown where the last node links back to the HEAD node (the start node).



# Circular Linked List



**To detect the first/end of the list in a Circular Linked List:**

- A special node called **HEAD** is used (as shown in the diagram).

**Types of Circular Linked List:**

1. **Singly circular linked list**
2. **Doubly circular linked list**

**Operations on Circular Linked List:**

- **Traversal:** Access every node once in a systematic manner.
- **Insert a node:** Insert a new node in the circular linked list.
- **Delete a node:** Delete a node from the circular linked list.
- **Print:** Display the circular linked list.

✓ **Advantages of Circular Linked List**

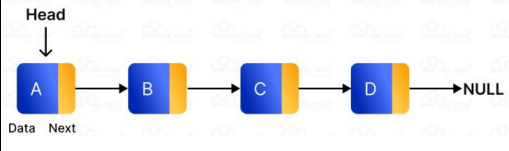
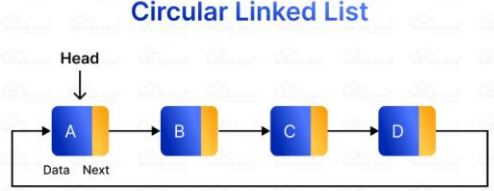
- Every node in the list is accessible from a given node. (Unlike singly linked list where access is only from the first node.)
- In doubly linked list, moving from last to first requires full traversal, but circular linked list can save time.
- Can be used to implement **circular queues**.

✗ **Disadvantages of Circular Linked List**

- If care is not taken during processing, it may result in an **infinite loop**.
- **Backward traversing is not possible**.
- A **HEAD node** is required to indicate the start or end of the list.

## 4.9 DIFFERENCE BETWEEN SINGLY LINKED LIST AND CIRCULAR LINKED LIST

| Singly Linked List                                                              | Circular Linked List                                                                                    |
|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| In this list, the last node has the NULL value. Not adjacent to the first node. | In this list, the last node is adjacent to the first node (link of last node points to the first node). |
| Every node is not accessible like circular linked list.                         | Every node is accessible from the given node.                                                           |
| No chance to get into infinite loop.                                            | There is a chance of getting into infinite loop.                                                        |

| Singly Linked List                                                                | Circular Linked List                                                                             |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| This list should be empty.                                                        | This list never be empty due to HEAD node, so no need to check for empty list in its operations. |
|  |                |

#### 4.10 DOUBLY LINKED LIST: REPRESENTATION

- In **singly linked list**, we can traverse in only one direction.  
Even in **circular linked list**, we can access any node from a given node.
- But in both singly and circular linked lists, we **can't move to the previous node**, as **backward traversal is not possible**.  
This is possible in case of a **doubly linked list**.

##### Definition:

A **doubly linked list** is “a list in which each node contains two links — one to the **predecessor** of the node and one to the **successor** of the node.”

Also known as:

**Doubly linked list** or **two-way chains**

##### Structure of Node in Doubly Linked List:

Each node contains **three fields**:

- DATA** – Data part
- LPTR** – Left pointer (points to the predecessor node)
- RPTR** – Right pointer (points to the successor node)

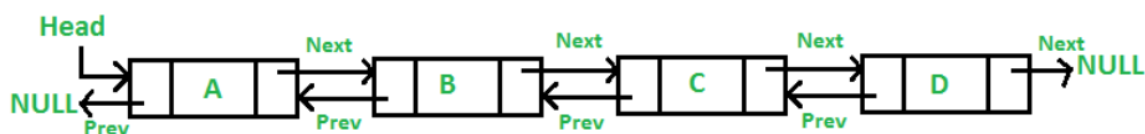
##### Node Representation:

[ LPTR | DATA | RPTR ]

##### Diagram:

A visual representation of a **doubly linked list** is shown, with:

- L → pointing to the **leftmost node**
- R → pointing to the **rightmost node**



##### Explanation:

- L is a pointer that points to the **leftmost node** in the list.
- R is a pointer that points to the **rightmost node** in the list.
- LPTR**: Left pointer – points to the **predecessor node** of that node.
- RPTR**: Right pointer – points to the **successor node** of that node.

- **DATA:** Field contains the **actual data**.
- **NULL pointer (LPTR)** of **leftmost node** and **RPTR** of **rightmost node** are set to NULL to indicate **end of the list** in each direction.

✅ **C Structure Declaration:**

```
c
struct node {
    int DATA;
    struct node *LPTR; // left pointer to point to the previous node
    struct node *RPTR; // right pointer to point to the next node
};
```

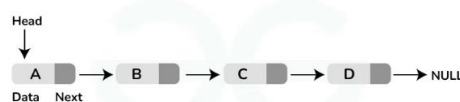
✅ **Advantages of Doubly Linked List**

- Allows traversal **in both directions** (forward and backward).
- **Faster** operations compared to singly linked list in some cases.
- **Easy to reverse** the linked list.
- We can access **any node from a given node**.
- **Program execution time is reduced**.
- Useful for representing **hierarchical structures** like trees.

⚠ **Disadvantages of Doubly Linked List**

- **More space per node** required because of the extra pointer.
- **Insertion** takes more time compared to singly linked list since more pointers need to be set.

#### 4.11 DIFFERENCE BETWEEN SINGLY LINKED LIST AND DOUBLY LINKED LIST

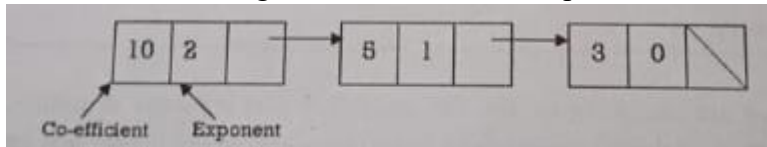
|                    | Singly Linked List                                                                  | Doubly Linked List                                                                   |
|--------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Traversing         | Traversing in only one direction (forward)                                          | Traversing in both directions                                                        |
| Each node contains | Each node contains one link field                                                   | Each node contains two link fields                                                   |
| Deletion           | Deletion operation requires more memory                                             | Deletion operation requires less memory                                              |
| Speed to implement | Speed to implement is slower                                                        | Speed to implement is faster                                                         |
|                    |  |  |

#### 4.12 APPLICATIONS OF LINKED LIST:

Linked List is very useful dynamic data structure. Linked list is used in a wide variety of applications such as:

(1) Polynomial representation

- Arithmetic polynomial manipulations are performed by linked list.
- Addition and subtraction operations of polynomial are easily implemented by linked list.
- Compare with multiplication, division and modulo operation.
- For example, if we want to represent the polynomial expression  $10X^2 + 5X + 3$  using linked list, it can be represented like:



(2) Symbol table construction

- Symbol table is a dictionary which is an important part of compiler containing names and their associated values.
- Linked list is used in constructing a symbol table.

(3) Multiple arithmetic precision

- In construction of multiple arithmetic precision and representation of sparse matrices, linked list is used.

(4) Hash list is also used to create data structures like stack, queues, trees and graphs.

(5) Linked table uses linked list for collision resolution.

(6) Linked allocation of files.

(7) Used in back and forward button in web browser, music player etc.

(8) Redo and undo functionality.

(9) The most recently used section in applications is represented by the Doubly Linked List.

## UNIT -5: Trees

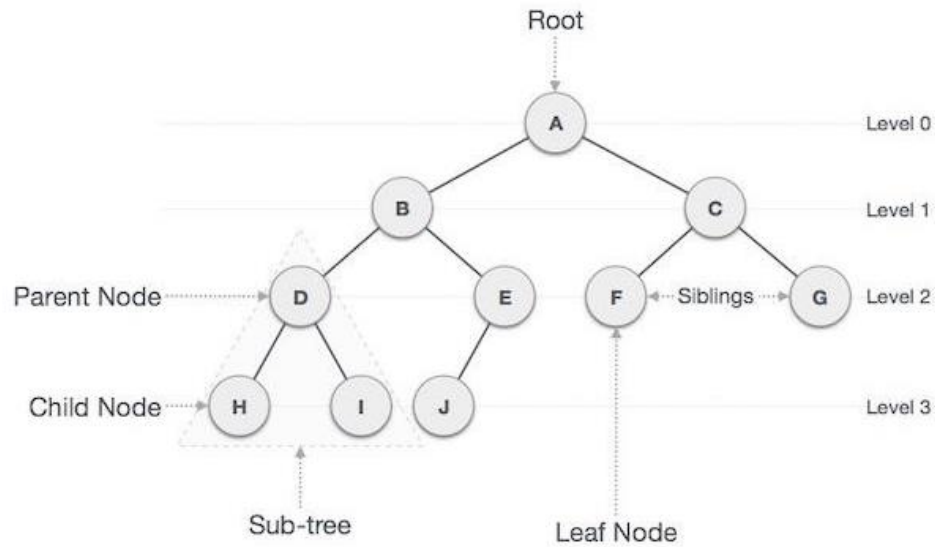
### 5.1 NON-LINEAR DATA STRUCTURE: TREE & GRAPH

Non-linear data structure is a kind of data structure in which the data values are **not** arranged in linear fashion. Tree, graph, table and set are examples of non-linear data structure.

---

#### 5.1.1 Tree :

- A tree is non-linear and a hierarchical data structure consisting of a collection of nodes, such that each node of the tree stores a value and a list of references to other nodes.
- Tree data structure is mainly used to represent hierarchical relationship between elements.
- Basically, a **Tree is an acyclic graph**.



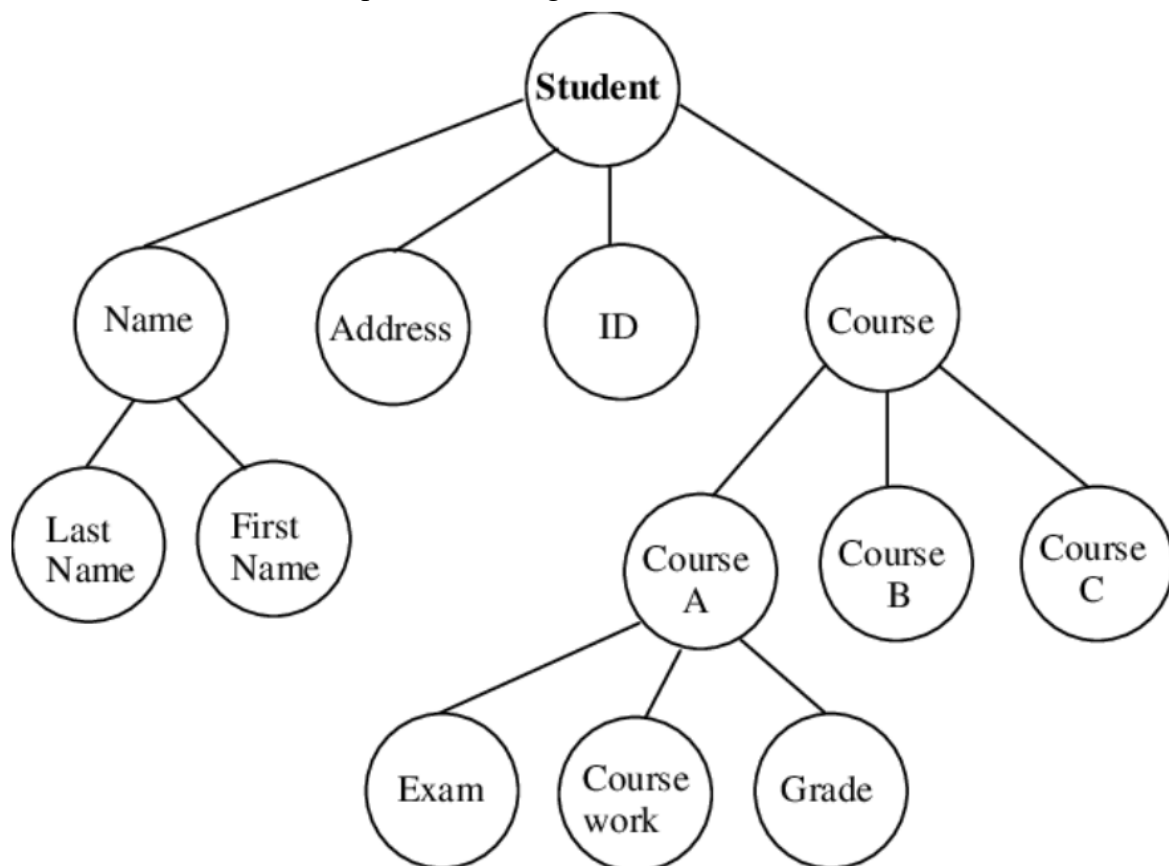
Tree data structure contains:

- **Root node** (Top most node)
- **Sub nodes**
- **Edge (Link / relationship between nodes)**

Here each node contains some data (may be of any type).

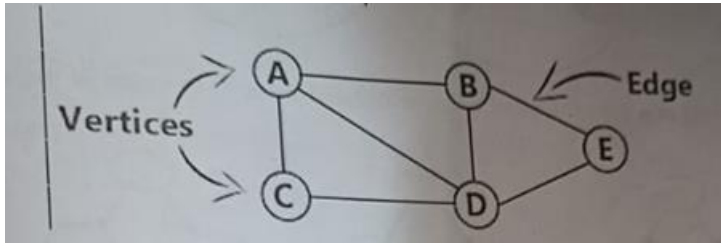
**Example of tree:**

Here the student details are represented using tree data structure.



**5.1.2 Graph:**

- Graph (G) is a non-linear data structure which consists of a non-empty set of nodes (V) and a set of edges (E), and a mapping from set of edges E to a set of pair of elements of V.
- Simply, a graph is represented as:  $G=(V,E)$



#### Example:

- $V=\{A,B,C,D,E\}$   $V = \{A, B, C, D, E\}$   $V=\{A,B,C,D,E\} \rightarrow$  Set of Vertices (Nodes)
- $E=\{AB,AC,AD,BD,BE,DE\}$   $E = \{AB, AC, AD, BD, BE, DE\}$   $E=\{AB,AC,AD,BD,BE,DE\} \rightarrow$  Set of Edges

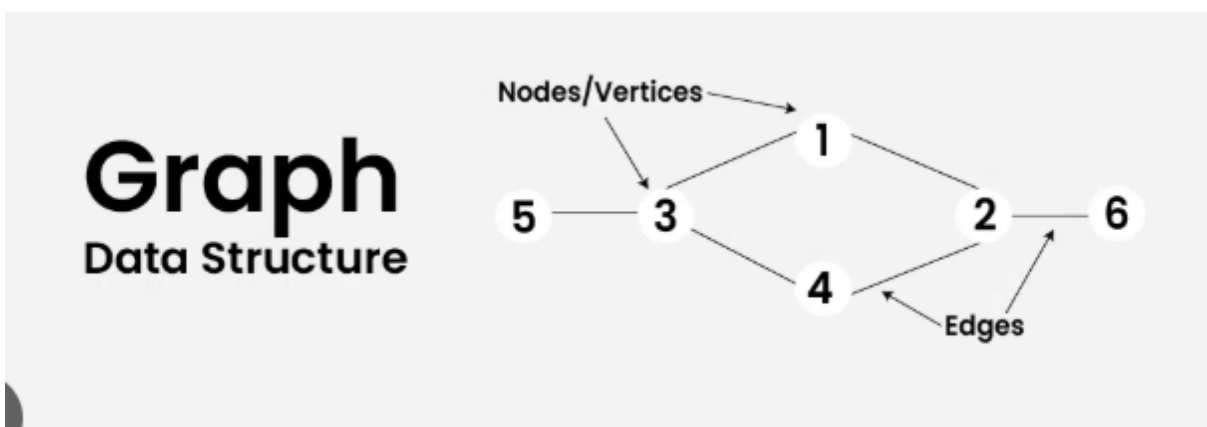
#### Types of Graph:

- ✓ Undirected Graph
- ✓ Directed Graph
- ✓ Weighted Graph
- ✓ Cyclic Graph
- ✓ Acyclic Graph

#### Applications of Graph:

Graph is a very useful data structure in computer science. Various applications of graphs are:

- Graphs are used to represent flow of control in computers.
- Graphs are used in social networking sites where users act as nodes and connection between them acts as edges.
- Graphs are used as resource allocation graphs in an operating system.
- Graphs are used in Google Maps to find the shortest route.
- Graphs are used to represent network of communication.



## 5.2 BASIC TERMS OF A TREE

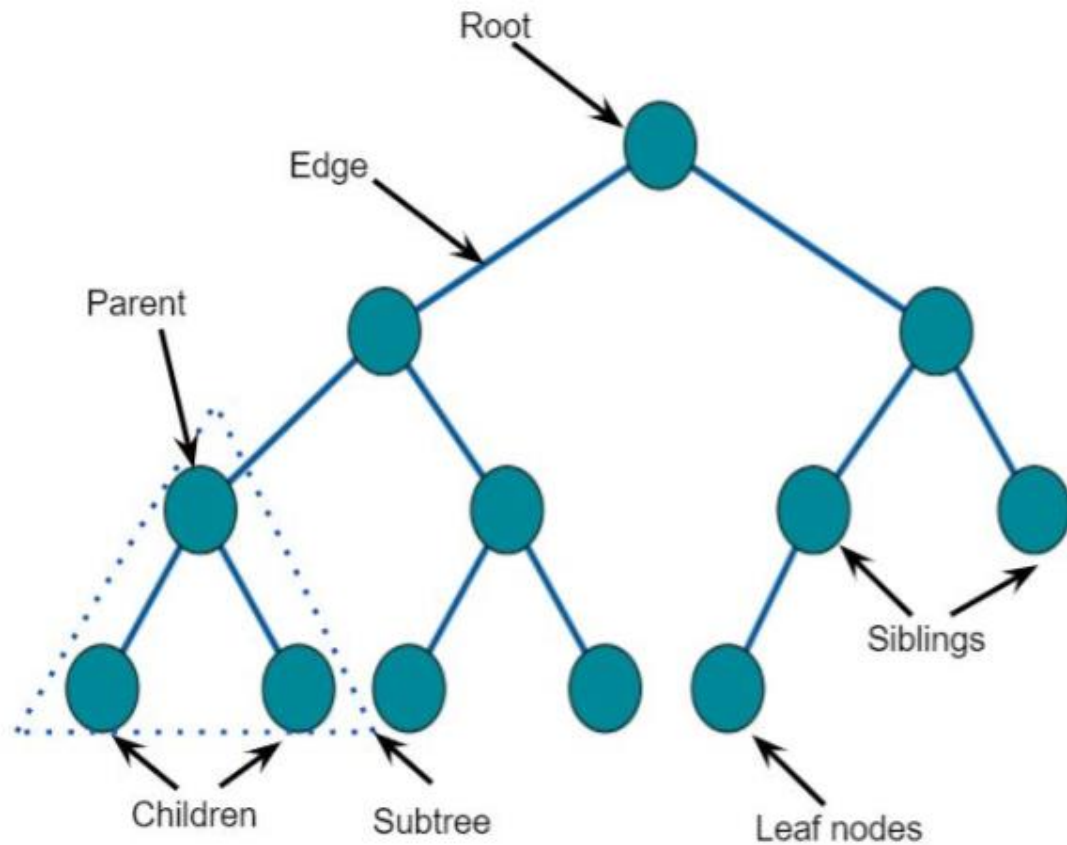
### 1. General Tree

- A tree is defined as a finite set of one or more nodes such that:
  - There is a special node called the root node **R**.

- The remaining nodes are divided into  $n \geq 0$  disjoint sets **T1, T2, ..., Tn**, where each of these sets are the tree.
- **T1, T2, ..., Tn** are called the **subtrees** of the root.

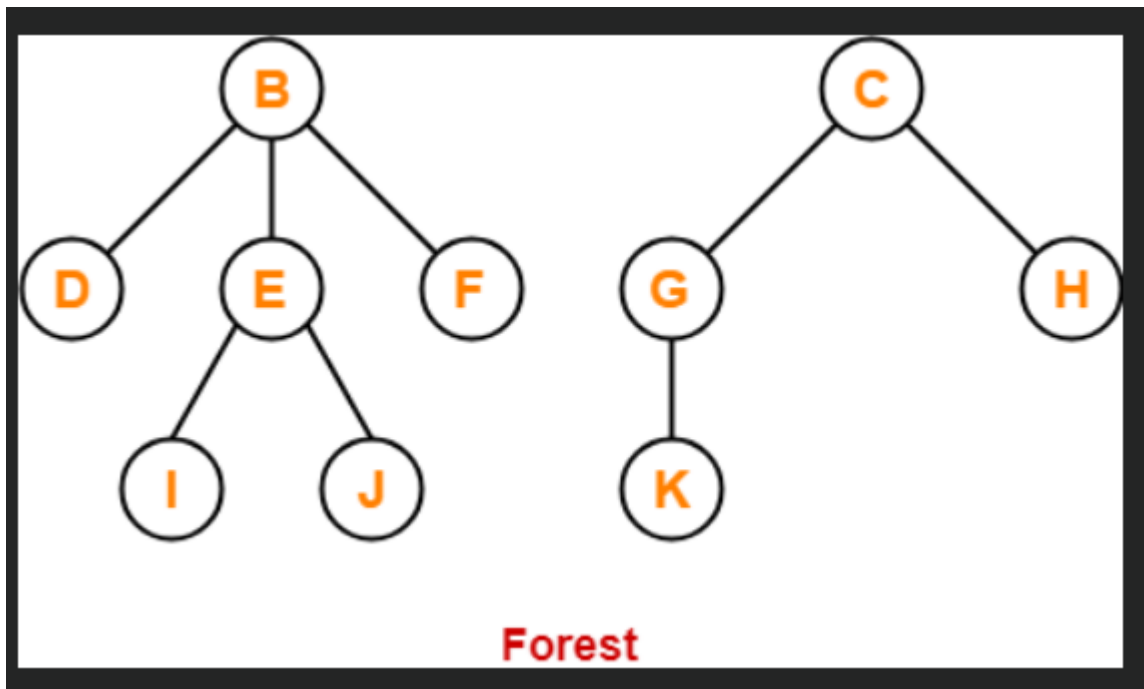
#### Terms:

- **Root Node** – The topmost node of the tree.
- **Edge** – The connection between two nodes.
- **Left Sub Tree and Right Sub Tree** – The subtrees on left and right of the root.



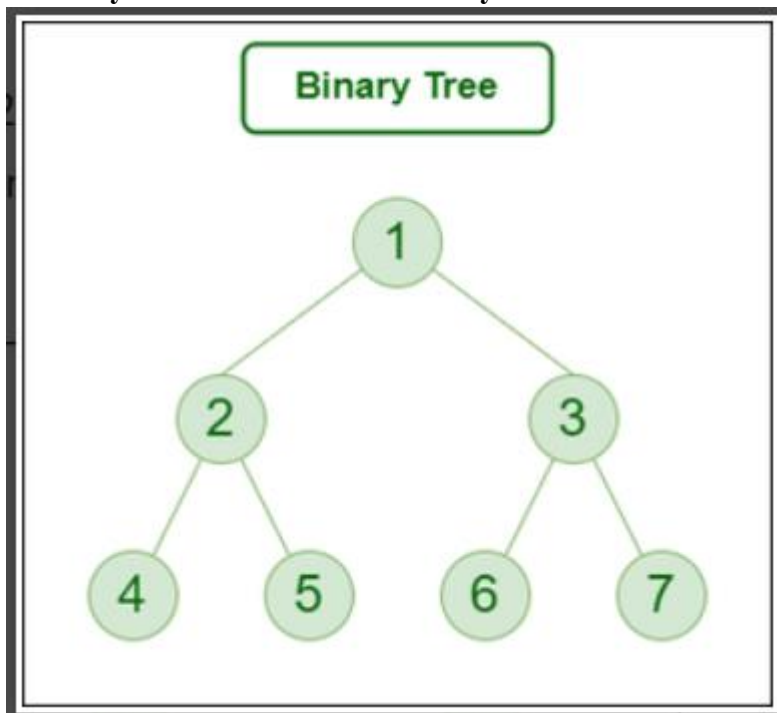
#### 2. Forest

- A **forest** is a set of disjoint trees or isolated trees.
- It is the **union of disjoint trees**.



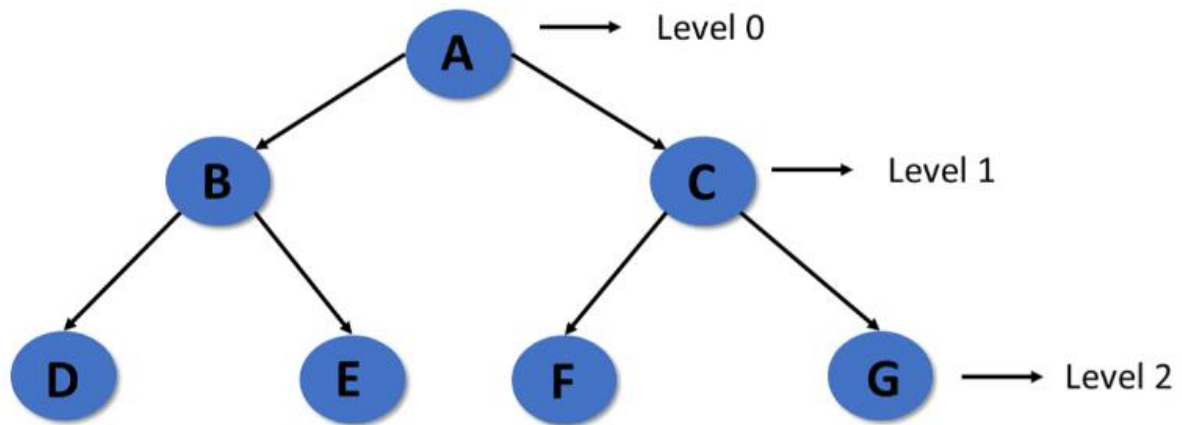
### 3. Binary Tree

- A **binary tree** is a tree in which **every node can have a maximum of two children**.



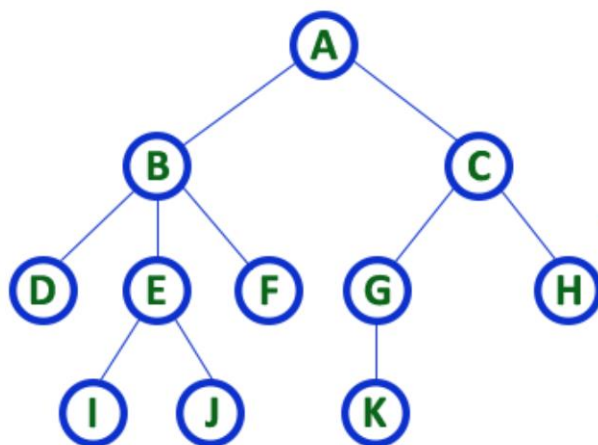
### 4. Level of the Tree

- In a tree, the **root node is at Level 0**.
- The children of root node are at **Level 1**, and the children of those nodes are at **Level 2**, and so on.
- Each step from top to bottom is called a **level**.
- The level count starts with **0** and increments by one at each level (step).



### 5. degree

- In the tree data structure, the total number of children of a node is called the degree of the node.
- The highest degree of the node among all the nodes in a tree is called the Degree of Tree.



Here **Degree of B is 3**

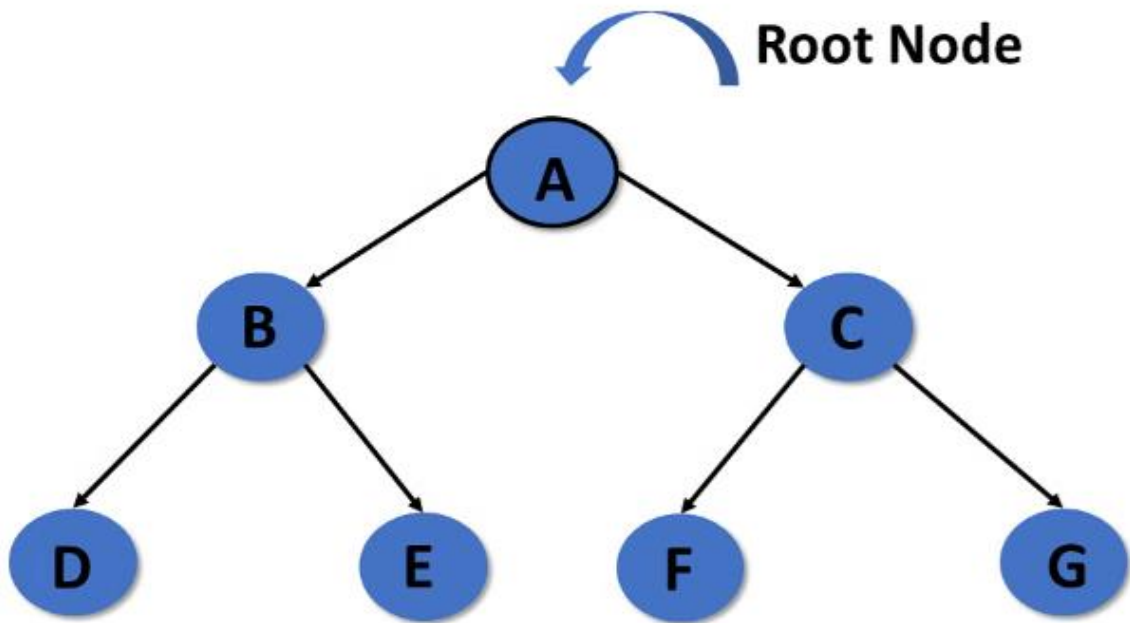
Here **Degree of A is 2**

Here **Degree of F is 0**

- In any tree, '**Degree**' of a node is total number of children it has.

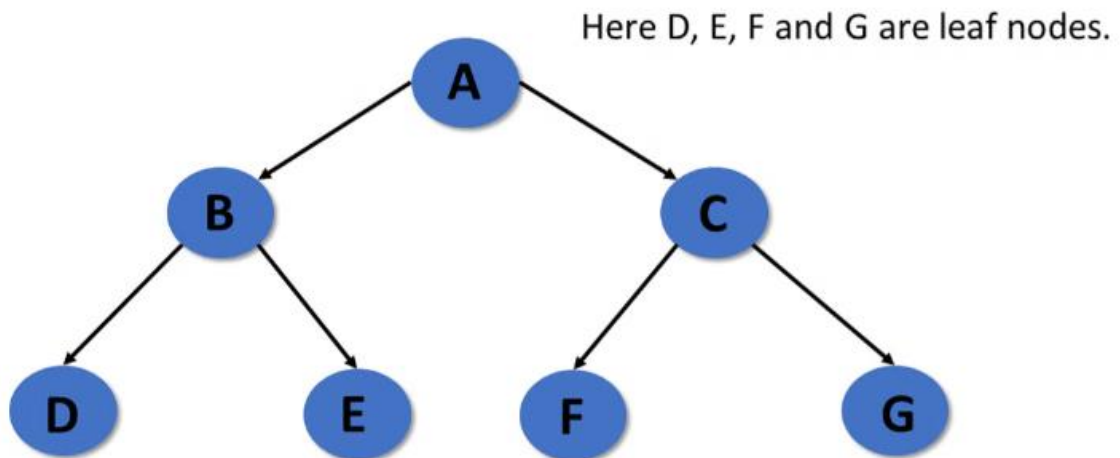
### 7. Root node

- In a directed tree, a **node which has in-degree 0** is called the **root node**.
- In a tree, there is only one root node.
- We can say the **root node is the origin of the tree**.
- Example: In the diagram shown, the root has **Out degree 2** and **In degree 0**.



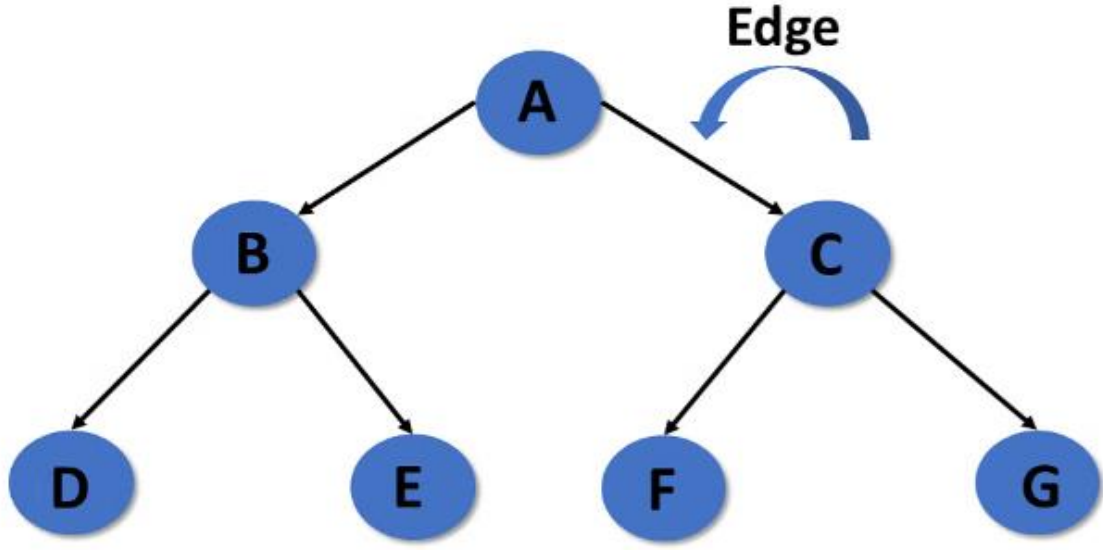
#### 8. Leaf Node (Terminal Node):

In a directed tree, a node which has out-degree 0 is called the **leaf node** or **terminal node**.



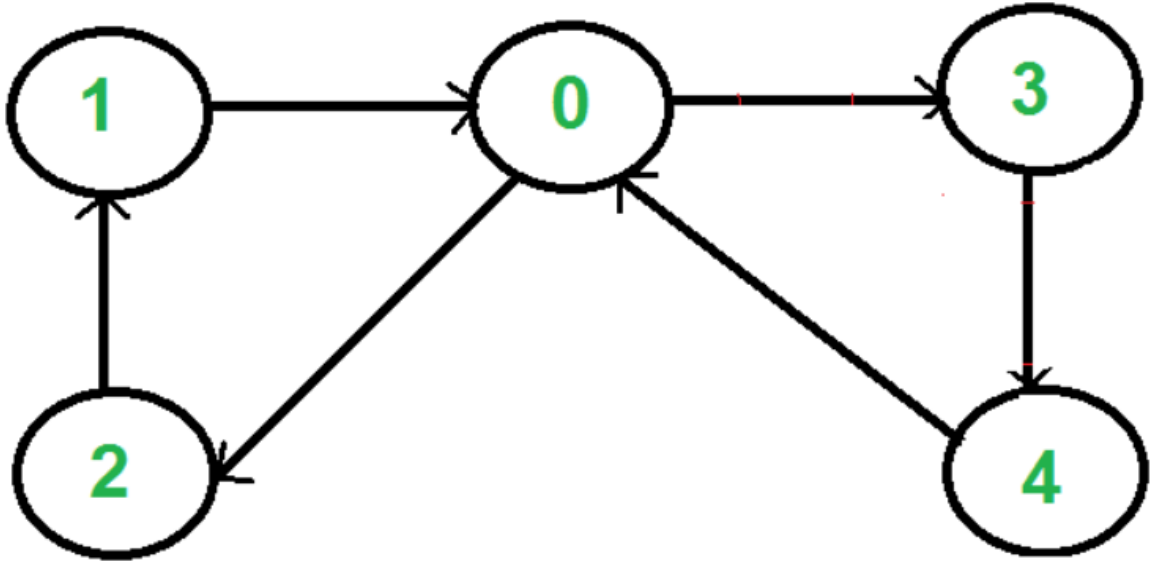
#### 9. Edge:

- In a tree, the connecting link between any two nodes is called an **edge**.  
In a tree with '**N**' number of nodes, there will be a maximum of '**N-1**' number of edges.



#### 10. Directed Edge:

In a tree, if direction is shown on the edge, it is called a **directed edge**.  
In a **directed tree**, all the edges are directed.

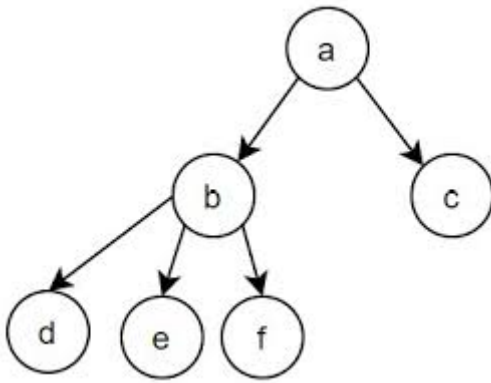


#### 11. Directed Tree:

A **directed tree** is an acyclic diagram which has one node called the **root node** with **in-degree 0**, while all other nodes have **in-degree 1**.  
(Gujarati Translation Present)

(Tree માં Edges પર direction દર્શાવવામાં આવેલ છે તેવાં tree ને directed tree કહેવાય.)

In a tree, all the edges are directed (an edge with direction is called **Directed edge**).

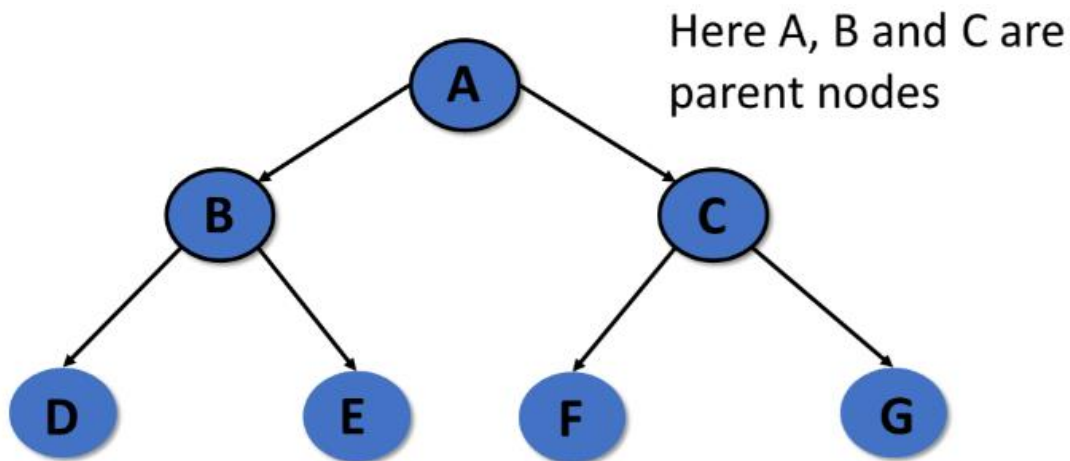


### 12. Parent:

In a tree, the node which is a **predecessor** of any node is called a **parent node**.

Parent node can also be defined as:

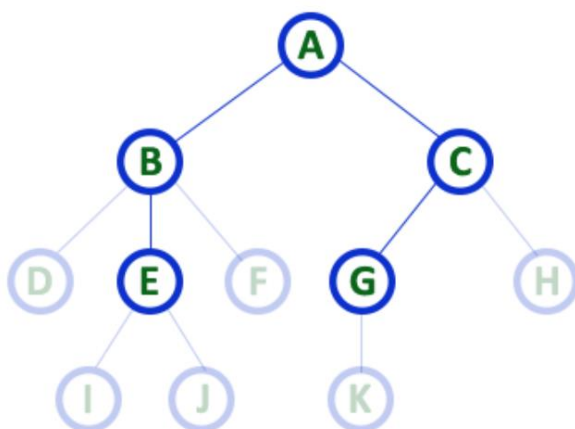
“The node which has child / children”.



### 13. Child:

In a tree, the node which is a **descendant** of any node is called a **child node**.

In simple words, the node which has a link from its parent node is called a **child node**.

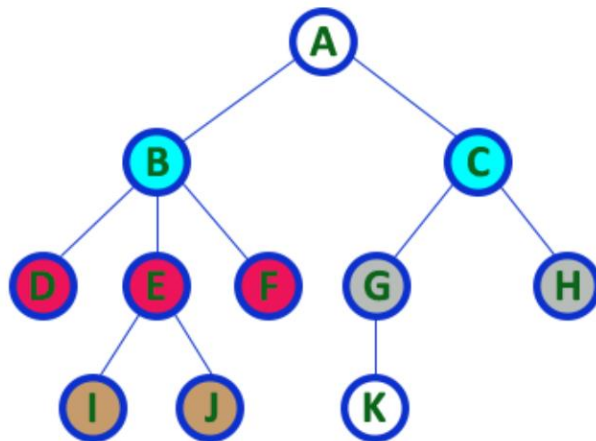


Here A, B, C, E & G are **Parent** nodes

- In any tree the node which has child / children is called 'Parent'
- A node which is predecessor of any other node is called 'Parent'

### 14. Siblings:

All the **children of the same parent node** are known as **sibling nodes**.



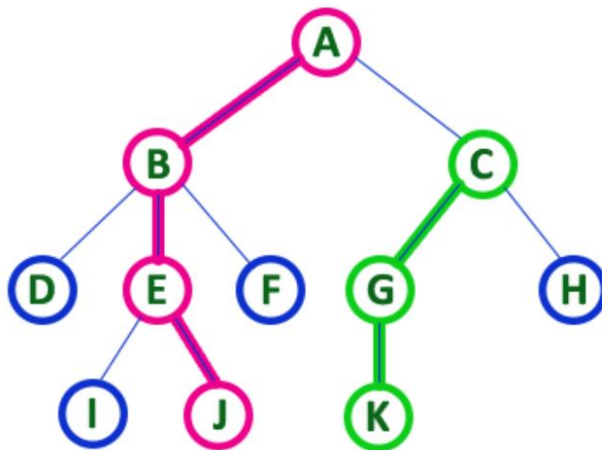
Here B & C are Siblings  
 Here D E & F are Siblings  
 Here G & H are Siblings  
 Here I & J are Siblings

- In any tree the nodes which has same Parent are called 'Siblings'
- The children of a Parent are called 'Siblings'

### 15. Path:

A **sequence of continuous edges** is called a **path** of the tree, OR a set of connecting edges from one node to another.

(Length of the path is the total number of nodes in the path)



- In any tree, 'Path' is a sequence of nodes and edges between two nodes.

Here, 'Path' between A & J is

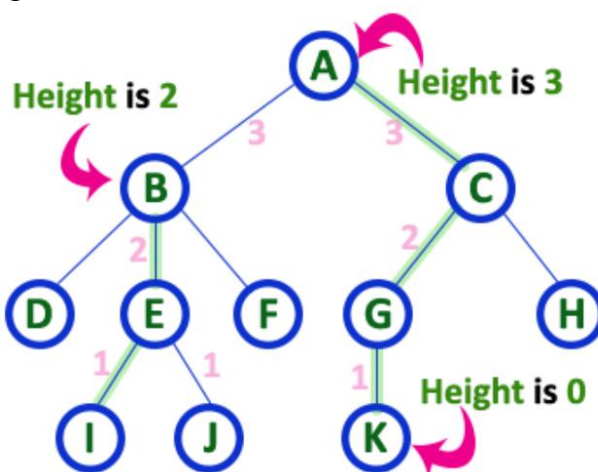
A - B - E - J

Here, 'Path' between C & K is

C - G - K

### 16. Height:

Number of levels or the length of the longest path from the root to any other node is the height of the tree.



Here Height of tree is 3

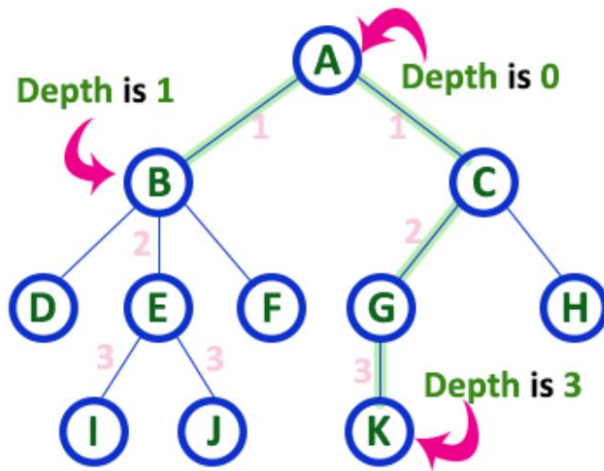
- In any tree, 'Height of Node' is total number of Edges from leaf to that node in longest path.
- In any tree, 'Height of Tree' is the height of the root node.

### 17. Depth:

In a tree data structure, the **total number of edges** from root node to a particular node is called the **depth** of that node.

In simple words, in a tree, the total number of edges from root node to a leaf node in the longest path is said to be the **depth of the tree**.

Depth of the root node is '0'.



Here Depth of tree is 3

- In any tree, 'Depth of Node' is total number of Edges from root to that node.

- In any tree, 'Depth of Tree' is total number of edges from root to leaf in the longest path.

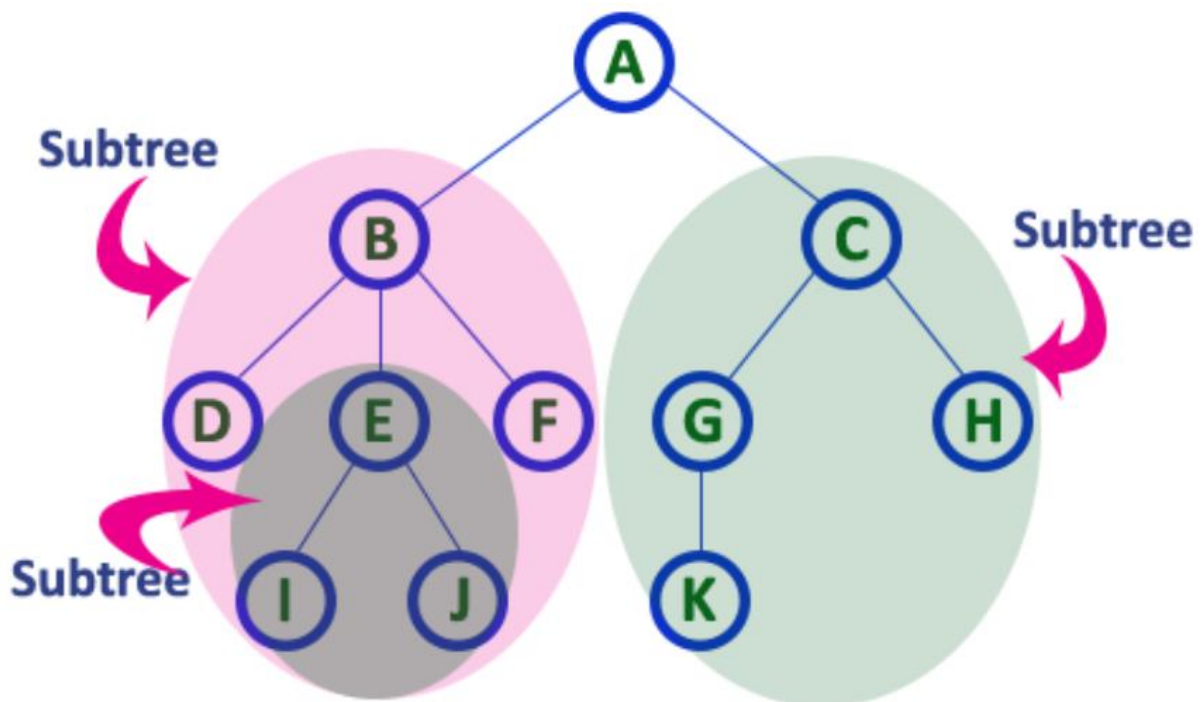
### 18. Null Tree:

A tree with **no node** is called a **Null tree** or **Empty tree**.

### 19. Subtree:

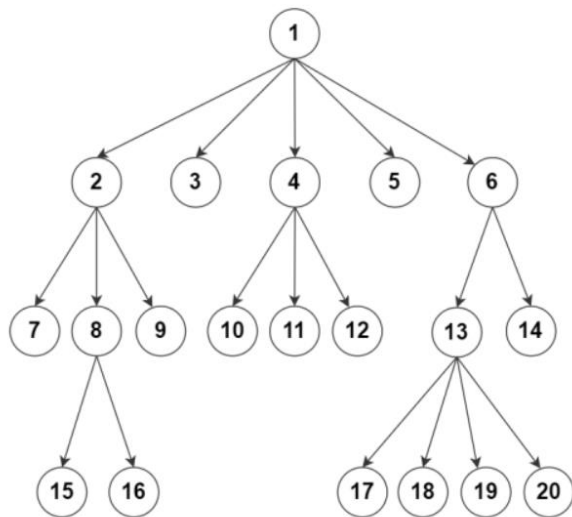
In a tree, each child from a node forms a **subtree** recursively.

Every child node will form a subtree on its parent node.



### 20. M-ary Tree:

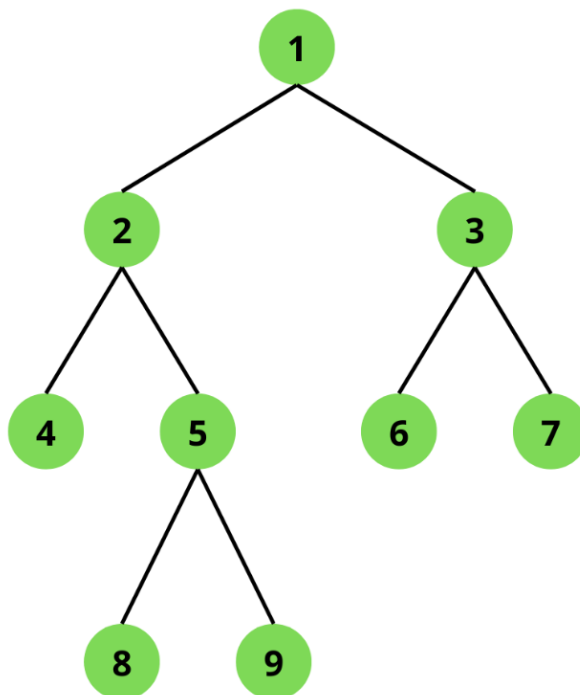
If in a directed tree, the **outdegree of every node is less than or equal to M**, then the tree is called an **M-ary tree**.



## 21. Strict Binary Tree:

A binary tree in which **every node has either two or zero number of children** is called a **Strictly Binary Tree**.

- A strict Binary Tree with **N leaves** always contains  **$(2 \times N - 1)$  Nodes**.



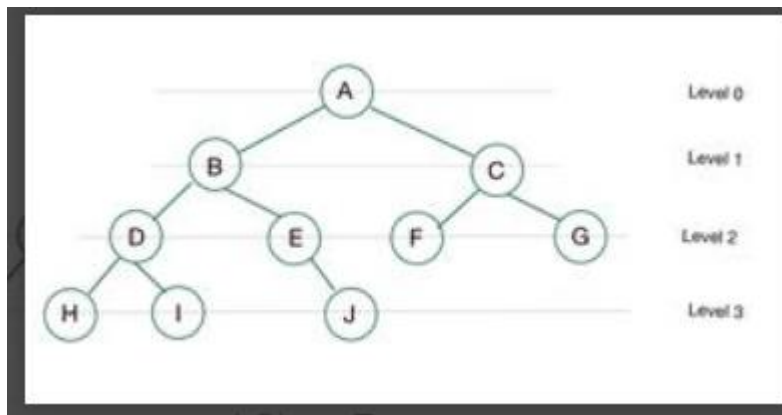
## 22. Complete Binary Tree (Full Binary Tree):

A binary tree in which:

- **Every internal node has exactly two children**, and
  - **All leaf nodes are at the same level**,
- is called a **complete binary tree**.

**OR,**

If the **outdegree of every node is exactly equal to 2 or 0** and **the number of nodes at level  $i$  is  $2^i$** , then the tree is called a **full or complete binary tree**.



### 5.3 Binary Search Tree (BST)

#### Definition:

A tree is called a Binary Search Tree (BST) if each node has at most two branches (left and right), and it should follow these characteristics:

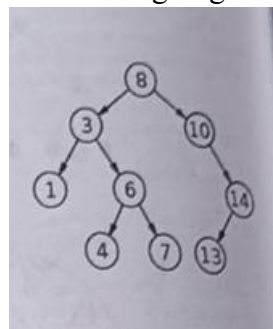
- All nodes to the left of any node have values less than that node's value.
- All nodes to the right of any node have values greater than that node's value.

#### Operations on Binary Search Tree:

- Insertion
- Deletion
- Searching
- Traversal

#### Example of a Binary Search Tree:

- Node 8 is the root node, and it has a left child (node 3) and a right child (node 10).
- Node 3, 6, 10, and 14 are non-terminal (internal) nodes, whereas nodes 1, 4, 7, and 13 are terminal or leaf nodes.
- The structure shows the values on the left side of the root node being smaller than the root node, and the right side values being larger.



#### 5.3.1 Insertion Operation on Binary Search Tree:

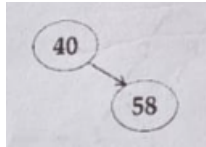
You are provided with an example where a BST is constructed using the following values: 40, 58, 50, 30, 55, 10.

#### Steps:

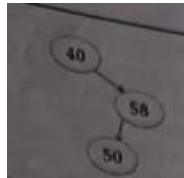
1. **Inserting node 40:** It is inserted as the root node.



2. **Inserting node 58:** It's greater than 40, so it's inserted to the right of 40.

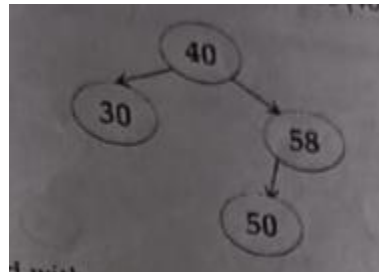


3. **Inserting node 50:** It's greater than 40 but less than 58, so it is inserted to the left of 58.



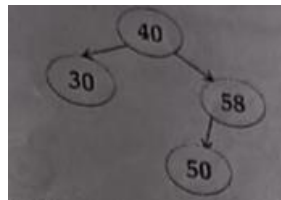
4. **Inserting node 30:**

- First, node 30 is compared with the root node (40). Since 30 is smaller than 40, it will be inserted on the **left** side of the root node.



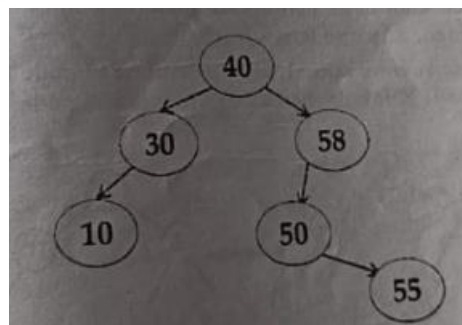
5. **Inserting node 55:**

- First, node 55 is compared with the root node (40). Since 55 is greater than 40, we move to the right side (58).
- Now, node 55 is compared with 58. Since 55 is smaller than 58 and greater than 50, it is inserted to the **right** of node 50.



6. **Inserting node 10:**

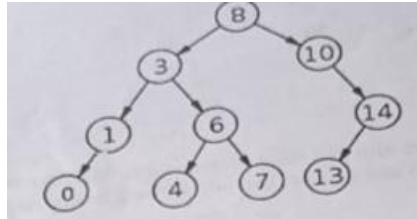
- First, node 10 is compared with the root node (40). Since 10 is smaller than 40, we move to the left side (30).
- Next, node 10 is compared with 30. Since 10 is smaller than 30, it will be inserted to the **left** of node 30.



## Deletion Operation on Binary Search Tree

### Hint:

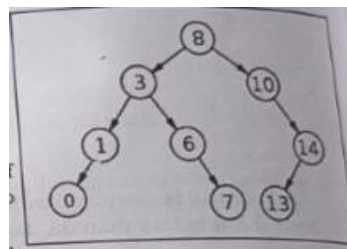
- The node in the BST is compared starting from the root. If the value is less than the root, it will go to the left subtree; if it is greater, it will go to the right subtree. This comparison helps locate the node to be deleted.



**There are three possibilities when we want to delete a node from a Binary Search Tree:**

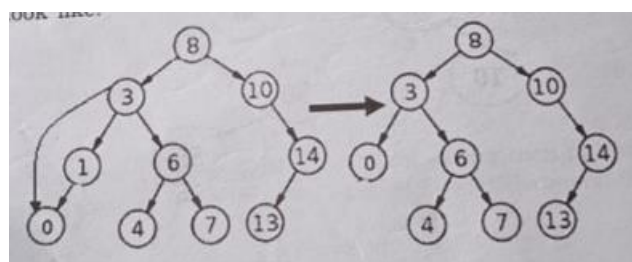
### 1. Deleting a Leaf Node:

- If the node to be deleted has no children, it is a leaf node. In this case, you can simply delete the node directly.
- Example: If we want to delete node 4 from the tree shown in the figure, we will delete it directly since it is a leaf node.
- The tree will look like this after deletion:



### 2. Deleting a Node with One Child:

- If the node to be deleted has **one child**, you replace the node to be deleted with its child. This means connecting the parent of the deleted node directly to the child.
- Example:** If node 1 is to be deleted, it has one child (3). So, instead of deleting node 1 directly, we connect node 3 (the child of node 1) to its parent (node 8). The resulting tree looks like:



### Deleting a Node with Two Children:

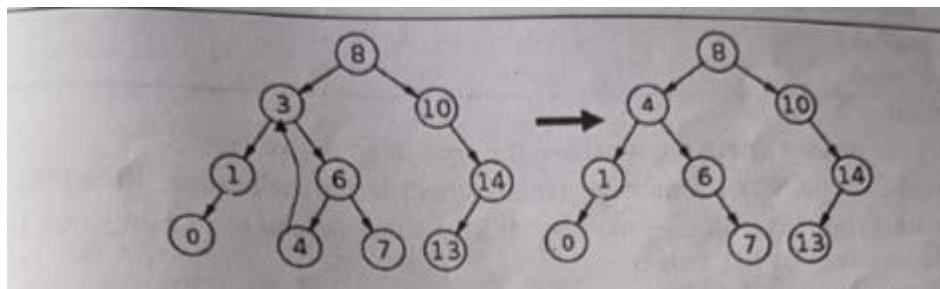
When the node to be deleted has **both left and right subtrees**, you need to follow these steps:

1. **Find the "in-order successor"** of the node to be deleted.
2. **Append the right subtree** of the in-order successor to its parent (this is essentially linking the right subtree of the successor node to the parent of the successor).
3. **Replace the node to be deleted** with its in-order successor.

**Example:**

- In the provided tree, if we want to delete **node 3**, which has both a left and right subtree, we:
  - Find the **in-order successor** of node 3 (node 4 in this case).
  - Replace node 3 with node 4 and link the left subtree of node 4 (if any) to its parent.

The tree will look like:



**In-order successor:** Node 4 (since in-order traversal visits nodes in ascending order).

### 5.3.3 Searching a Node in Binary Search Tree (BST):

When searching for a key in a BST, follow this algorithm:

- **Start with the root node** and compare the key with the root value.
  - **If the key matches the root node**, the search is complete.
  - **If the key is smaller** than the root node, search in the **left subtree**.
  - **If the key is greater** than the root node, search in the **right subtree**.

Continue the process by recursively comparing the key with the nodes as you traverse down the tree until you find the target node.

**Algorithm: BST\_SEARCH**

This algorithm searches for the **KEY node** in the BST:

- **T:** A pointer to the root node.
- **LPTR:** Left pointer.
- **RPTR:** Right pointer.

### Algorithm for Searching a Node in Binary Search Tree (BST)

#### Step 1: [check for empty tree]

```
c
if (T == NULL)
    then write ("Tree is empty")
    return (0)
```

#### Step 2: [Compare the KEY with root node]

```
c
if (KEY == DATA(T))
    then return (T)
```

#### Step 3: [search in left or right sub tree]

```
c
if (KEY < DATA(T))
    then call BST_SEARCH(LPTR(T), KEY)
else
    call BST_SEARCH(RPTR(T), KEY)
```



#### Step 4: [finished]

```
c
return()
```

#### Description:

- In step 1, we are checking whether the tree is empty or not.
- In step 2, if our **KEY** (element to be searched) is the **root value**, then it is found.
- If not, find the **KEY** value in the tree.
- If **KEY** value is less than root value, then find the **KEY** recursively in **left sub tree**.
- If **KEY** value is greater than root value, then find the **KEY** recursively in **right sub tree**.

#### 5.4 TREE TRAVERSAL

As we have seen that traversal is a method of processing every node in the tree exactly once in a systematic manner.

There are three ways for binary search tree traversing:

---

##### Pre-order traversal (Root – Left – Right)

- Process the **root node first**.

- Traverse the **left sub tree** of root node in pre-order.
  - Traverse the **right sub tree** of root node in pre-order.
- 

#### **In-order traversal (Left – Root – Right)**

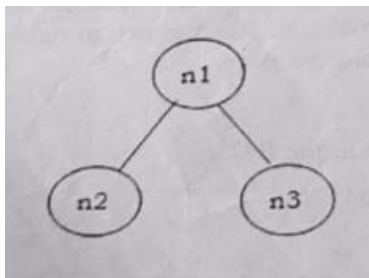
- Traverse the **left sub tree** of root node in in-order.
  - Process the **root node**.
  - Traverse the **right sub tree** of root node in in-order.
- 

#### **Post-order traversal (Left – Right – Root)**

- Traverse the **left sub tree** of root node in post-order.
- Traverse the **right sub tree** of root node in post-order.
- Process the **root node**.

#### **Example:**

- Suppose there are only three nodes in the tree having the following arrangement.  
Find tree traversal (pre-order, in-order and post-order):



#### **Output:**

- Pre-Order: n1 n2 n3
- In-Order: n2 n1 n3
- Post-Order: n2 n3 n1

#### **Algorithm: PREORDER**

- This function traverses the tree in pre-order.  
**T** is a pointer which points to the root node of the tree.  
**LPTR** and **RPTR** denote the left pointer and right pointer of the particular node.

### Step1: [process the root node]

```
c

if (T != NULL)
    then write (DATA(T))
else
    write ("EMPTY TREE")
return()
```

---

### Step2: [process the left sub tree]

```
c

if (LPTR(T) != NULL)
    then call PREORDER(LPTR(T))
```

### Step3: [process the right sub tree]

```
c

if (RPTR(T) != NULL)
    then call PREORDER(RPTR(T))
```

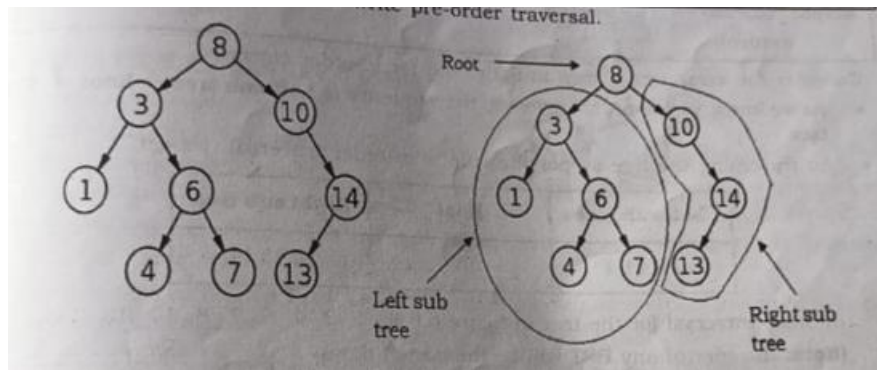
---

### Step4: [finished]

```
c

return()
```

### Example



**Fig. 6.6**

**Explanation:**

- As we know, in pre-order traversal, the sequence is: **Root → Left sub tree → Right sub tree**
- So first node will be accessed is **node 8**.
- Now in left sub tree (3,1,6,4,7) we again follow the same rule (**Root → Left → Right**). So we get left sub tree order: **3 1 6 4 7**.
- In right sub tree (10,14,13) we again follow the same rule and get: **10 14 13**.

**Therefore, Pre-order sequence is:**

8 3 1 6 4 7 10 14 13

| Root | Left sub tree | Right sub tree |
|------|---------------|----------------|
| 8    | 3 1 6 4 7     | 10 14 13       |

**Algorithm: INORDER**

This function traverses the tree in in-order.

**T** is a pointer which points to the root node of the tree.

**LPTR** and **RPTR** denotes to the left pointer and right pointer of the particular node.

### Step1: [check for empty tree]

```
c

if (T == NULL)
    then write ("EMPTY TREE")
    return()
```

---

### Step2: [process the left sub tree]

```
c

if (LPTR(T) != NULL)
    then call INORDER(LPTR(T))
```

---

### Step3: [process the root node]

```
c

write (DATA(T))
```



### Step4: [process the right sub tree]

```
c

if (RPTR(T) != NULL)
    then call INORDER(RPTR(T))
```

---

### Step5: [finished]

```
c

return()
```

**Example:**

Consider the same tree shown in figure for in-order traversal.

**In order sequence is:**

**Left sub tree → Root → Right sub tree**

| Left sub tree | Root | Right sub tree |
|---------------|------|----------------|
| 1 3 4 6 7     | 8    | 10 13 14       |

**In-order traversal for the tree is:**

1 3 4 6 7 8 10 13 14

**Note:** In-order of any BST will be the sorted data.

**Algorithm: POSTORDER**

This function traverses the tree in post-order.

**T** is a pointer which points to the root node of the tree.

**LPTR** and **RPTR** denotes to the left pointer and right pointer of the particular node.

**Step1: [check for empty tree]**

c

```
if (T == NULL)
    then write ("EMPTY TREE")
    return()
```

### Step2: [process the left sub tree]

```
c  
  
if (LPTR(T) != NULL)  
    then call POSTORDER(LPTR(T))
```

### Step3: [process the right sub tree]

```
c  
  
if (RPTR(T) != NULL)  
    then call POSTORDER(RPTR(T))
```

### Step4: [process the root node]

```
c  
  
write (DATA(T))
```



### Step5: [finished]

```
c  
  
return()
```

#### Example:

Consider the same tree shown in figure 6.6 for post-order traversal.

**Sequence is:** Left sub tree → Right sub tree → Root

| Left sub tree | Right sub tree | Root |
|---------------|----------------|------|
| 1 4 7 6 3     | 13 14 10       | 8    |

**Post-order traversal for the tree in figure 6.6 is:**

1 4 7 6 3 13 14 10 8

---

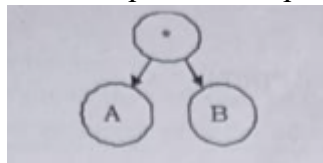
## 5.5 APPLICATION OF BINARY TREES

**The main applications of binary trees are as follows:**

- **Storing naturally hierarchical data:** File structure / Directory Structure
  - **Searching:** We can search for any element in BST (Binary Search Tree) in  $O(\log N)$  time which is much faster than searching in other data structures like Linked List, Arrays, etc.
  - **Priority Queue:** Heap is a tree data structure which is implemented using arrays and used to implement priority queues.
  - **B-Tree and B+Tree:** B-Tree and B+Tree are the tree data structures used to implement indexing in databases.
  - **Routing table:** The tree data structure is also used to store the data in routing tables in the routers.
  - **Manipulation of arithmetic expressions**
  - **Construction of symbol table**
  - **Syntax analysis**
  - **Manipulation of hierarchical data**
- 

### **Manipulation of arithmetic expressions**

- Binary tree is used to represent formulas in prefix or postfix notations.
- In case of infix formula as binary tree, where a node has an operator, its left and right child are the nodes with operands.
  - For example: infix expression  $A * B$  can be shown using a binary tree.



### **Construction of symbol table**

- Symbol table is a dictionary which is an important part of compiler containing names and their associated values.
  - Symbol table keeps track of scope and binding of information.
  - Tree is used in symbol table operations i.e. insertion and look up.
  - Binary tree is used because of two reasons:
    1. First reason → symbol table entries are in lexicographic order so this can be implemented using BST.
    2. Second → because of tree, there is a need of few pointers.
- 

### **Syntax analysis**

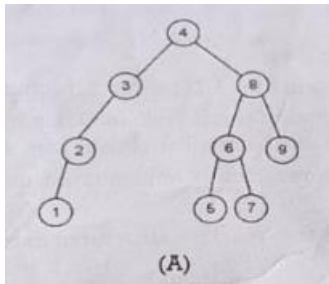
- Tree is used in grammars of syntax analysis or parsing.
- 

### **Manipulation of hierarchical data**

- Trees are used in manipulating hierarchical data as well used in making information easy to search.

### **Some solved examples of tree traversal**

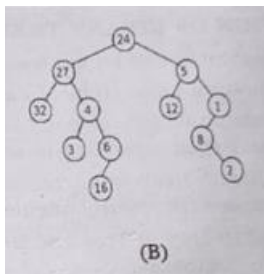
#### **Tree Traversals**



For tree A:

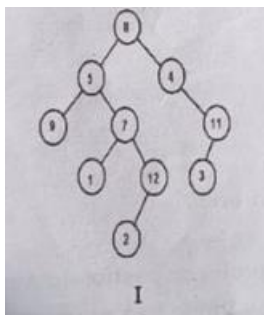
- Pre-order : 4 3 2 1 8 6 5 7 9  
(R → L → Ri)
  - In-order : 1 2 3 4 5 6 7 8 9  
(L → R → Ri)
  - Post-order : 1 2 3 5 7 6 9 8 4  
(L → Ri → R)
- 

For tree B:



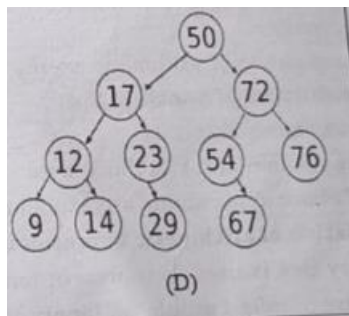
- Pre-order : 24 27 32 4 3 6 16 5 12 1 8 2
  - In-order : 32 27 3 4 6 16 5 12 1 8 2 24
  - Post-order : 32 3 16 6 4 27 12 2 8 1 5 24
- 

For tree C:



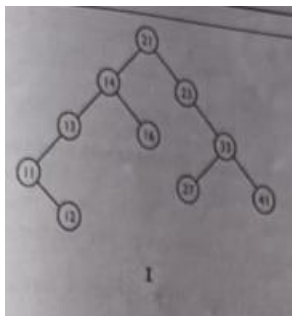
- Pre-order : 8 5 9 7 1 12 2 4 11 3
  - In-order : 9 5 1 7 2 12 8 4 3 11
  - Post-order : 9 1 2 12 7 5 3 11 4 8
- 

For tree D:



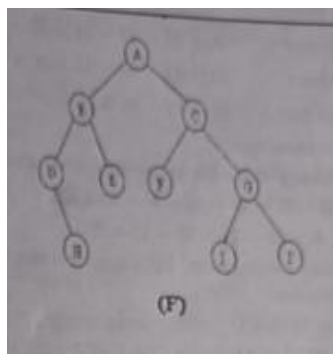
- Pre-order : 50 17 12 9 14 23 29 72 54 67 76
- In-order : 9 12 14 17 23 29 50 54 67 72 76
- Post-order : 9 14 12 29 23 17 67 54 76 72 50

For tree E:



- Pre-order : 21 14 13 11 12 16 23 33 27 41
- In-order : 11 12 13 14 16 21 23 27 33 41
- Post-order : 12 11 13 16 14 27 41 33 23 21

For tree F:



- Pre-order: A - B - D - H - E - F - C - G - I - J
- In-order: D - H - B - E - A - F - C - G - I - J
- Post-order: H - D - E - B - F - I - J - G - C - A

## UNIT -6: Sorting and Hashing

### SORTING METHODS:

**Sorting** is an operation of arranging the given data either in ascending or descending order. The sorting is performed according to the key value of each record. There are a number of methods available to perform sorting. Each one is different from others and each has its own advantages and disadvantages.

In general, the running time of each method is different (i.e., the complexity of an algorithm). Also, the main task in sorting is basically comparison and interchanging the position of elements.

---

### Classification of Sorting Methods:

1. **Stable Sort:** Stable sorting algorithms maintain the relative order of records with equal keys.

- **Example:** Merge Sort, Insertion Sort, and Bubble Sort.
- **Others:** Quicksort, Heapsort, and Selection Sort are unstable.

#### Example (Stable Sorting vs Unstable Sorting)

- Initial list: 8 5 9 8 3 5 9
  - After Stable Sort: 5 8 9 3 5 8 9
  - After Unstable Sort: 5 8 9 3 5 9 8
2. **Comparison Sort:** In which sorting is performed by comparing different elements using comparison operators (like "less than or equal to" or a three-way comparison), determining which elements should occur first in the final sorted list.
    - **Examples:** Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort, Heap Sort, etc.
  3. **Internal Sort:** In which all the data to be sorted is stored in main memory while sorting in progress.
    - **Examples of Internal Sort:** Bubble Sort, Quick Sort, Selection Sort, Insertion Sort, Shell Sort.

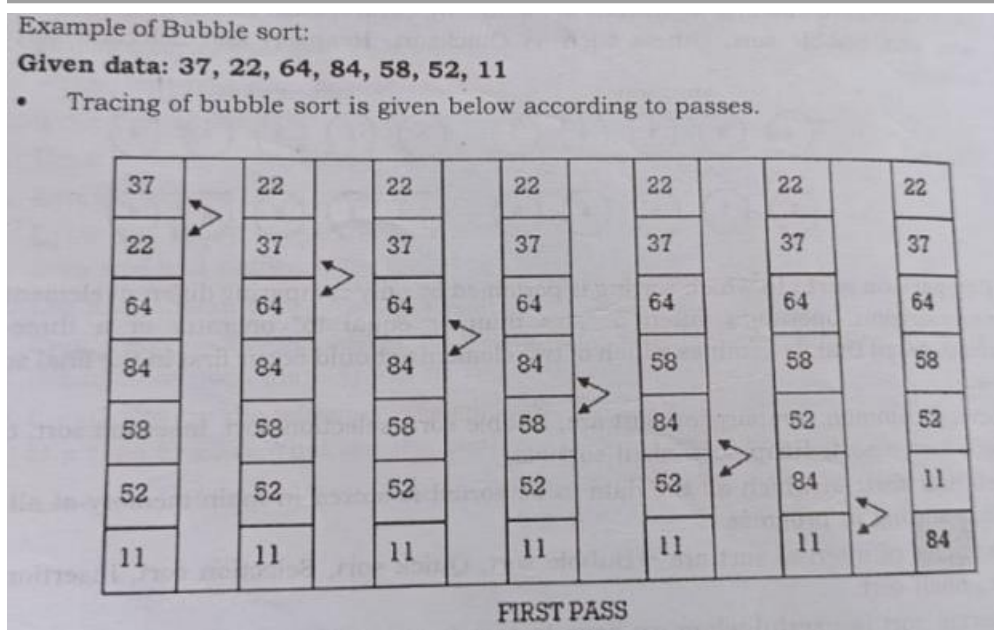
**Internal Sort** is useful when we have to sort fewer numbers of elements.

4. **External Sort:** In which all the data to be sorted is stored outside main memory (like on disk) and only loaded into main memory in small chunks.

#### 6.1.1 Bubble Sort

- **Bubble sort** is the easiest and frequently used sorting algorithm among all other algorithms.
- It is easy to understand and simple to implement.
- In the process of bubble sort, it makes multiple passes through a list. Bubble sort focuses on successive adjacent pairs of elements in the array, compares them, and either swaps them or not.
- The first element is compared with the second, if the first element is greater than the second, it will be swapped and further compared with the third element and again if it is greater than the third, then swapped. If the first element is not greater than the third element, the third element is compared with the fourth one and if greater, then swapped. Thus, comparison and swapping is made up to the end of the list.

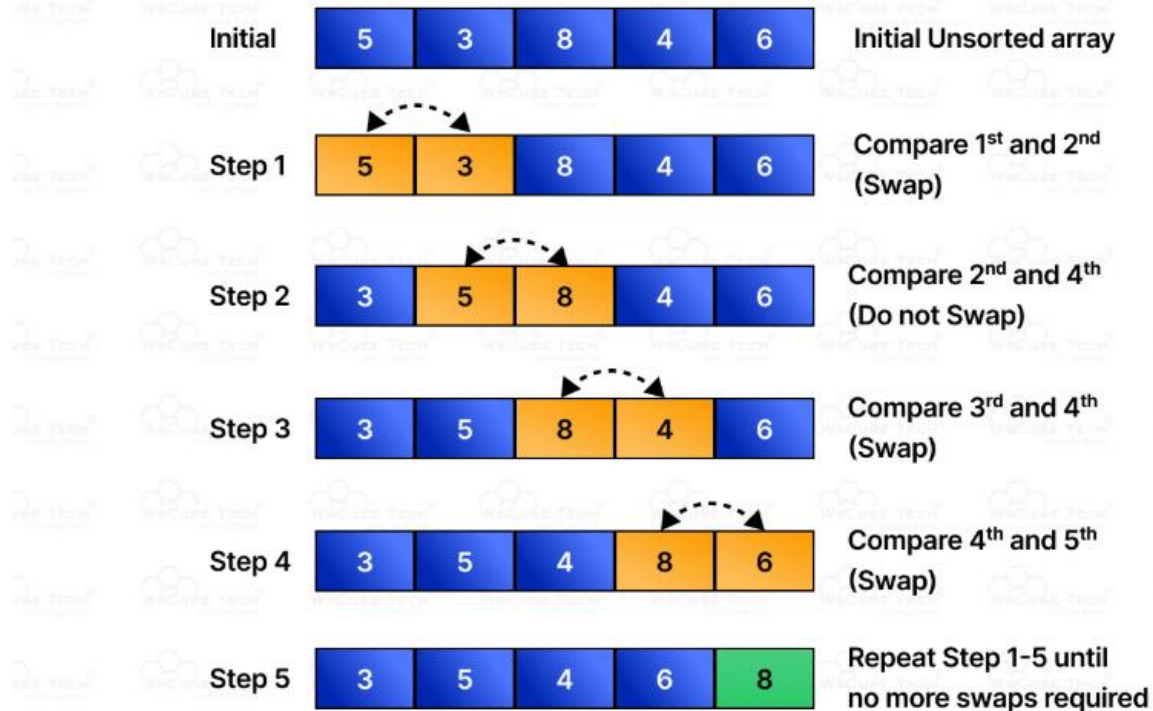
- Finally, at the end of the first pass, the largest element is placed at its proper position (last position).
- Each pass through the list places the next largest value in its proper place. In essence, each item "bubbles" up to the location where it belongs.
- **Maximum number of passes required** is  $n-1$ , where  $n$  is the total number of elements.
- The main disadvantage of this method is that it works efficiently for smaller lists but **not working well** when there is a large number of items in the list.



In the **first pass**, the comparison is made between adjacent elements and swapping is done. The largest element (**84**) is placed in its proper position.

This section explains how the **Bubble Sort** algorithm works through successive comparisons and swaps. The example illustrates the sorting process by showing the data after each comparison and swap, highlighting the largest element being placed correctly in the first pass.

## Bubble Sort Example



### Algorithm: BUBBLE SORT

- **K** is a vector of **N** elements.
- **PASS** denotes the pass counter and **LAST** denotes the last unsorted element.
- **I** is an index variable.
- **EXCH** is a variable counting number of exchanges made on any pass.

**Step1. [initialize]**

**LAST = N**

**Step2. [loop on pass index]**

Repeat thru step 5 for **PASS = 1, 2, 3, ..., N-1**

**Step3. [initialize exchange counter]**

**EXCH = 0**

**Step4. [perform pairwise comparison on unsorted elements]**

Repeat for **I = 0, 1, 2, ..., LAST - 1**

if **K[I] > K[I+1]**

then **K[I]  $\leftrightarrow$  K[I+1]**

// swapping of elements

**EXCH = EXCH + 1**

**Step5. [check whether exchanges made?]**

if **EXCH = 0**

then return(**K**)

// SORTING

**COMPLETED**

else **LAST = LAST - 1**  
**LIST**

**// REDUCE SIZE OF UNSORTED**

### Selection Sort

The content you've provided discusses the **Selection Sort** algorithm. Here's a summary of the key points:

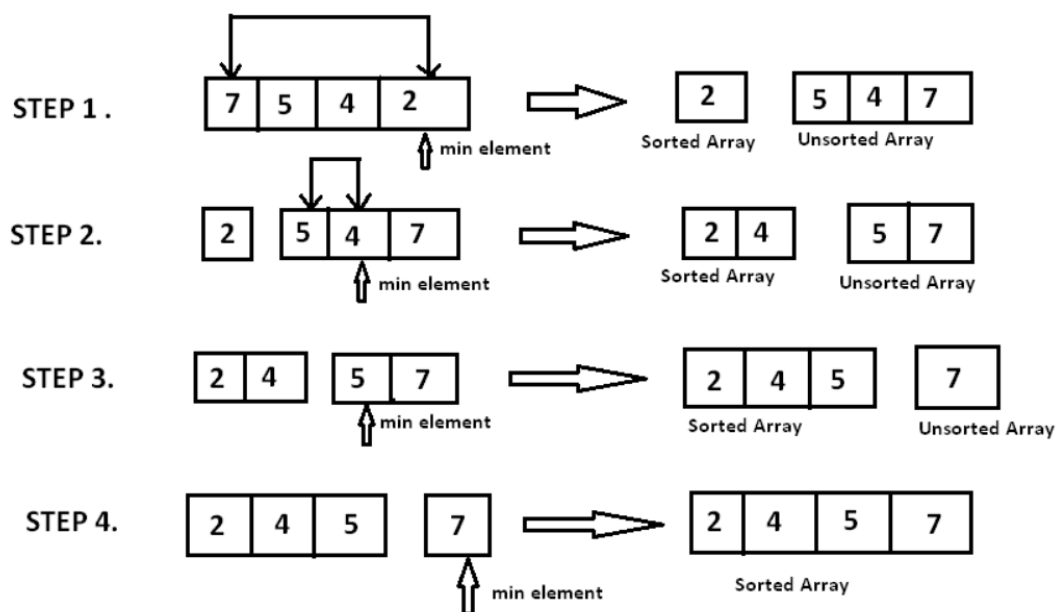
#### Selection Sort Overview:

##### 1. Concept:

- In each step, the algorithm selects the smallest element in the unsorted part of the array and swaps it with the element at the current position.
- This ensures that the smallest elements gradually move towards the front of the array, making it sorted in ascending order.

##### 2. Steps:

- Start by selecting the first element as the "key" value.
- Compare this key value with all other elements to find the smallest value.
- Once the smallest element is found, swap it with the key value.
- Repeat the process for the next element, treating it as the new key, until the entire array is sorted.



#### Selection Sort Algorithm:

The algorithm outlined in the image describes the steps for performing **Selection Sort** on a vector of elements (or an array). Below are the detailed steps of the algorithm:

##### 1. Step 1 (Loop through pass index):

- Loop through passes from  $PASS = 1$  to  $N-1$ , where  $N$  is the number of elements in the array.

##### 2. Step 2 (Initialize minimum index):

- For each pass, initialize  $MIN\_INDEX$  to the value of  $PASS$ .

##### 3. Step 3 (Find the smallest element):

- For each pass, compare the element at  $PASS$  with each element that comes after it.
- If a smaller element is found, update  $MIN\_INDEX$ .

#### 4. Step 4 (Swap elements):

- Once the smallest element is found, swap it with the element at PASS (if MIN\_INDEX is different from PASS).

#### 5. Step 5 (Finish):

- Continue the loop until all elements are sorted.

### Quick Sort Algorithm:

The content also introduces **Quick Sort**, a very efficient sorting algorithm:

#### 1. Overview of Quick Sort:

- Quick Sort is a **divide-and-conquer** algorithm that is fast and efficient, especially for large data sets.
- It works by selecting a **pivot element** and then partitioning the array into two subarrays:
  - One subarray has elements less than the pivot.
  - The other subarray has elements greater than the pivot.
- The pivot is placed in its final sorted position after each partitioning step.

#### 2. Steps of Quick Sort:

- **Pivot Selection:** In each pass, one element is selected as the pivot.
- **Partitioning:** The array is rearranged such that all elements less than the pivot go to the left, and those greater than the pivot go to the right.
- **Recursive Sorting:** The left and right subarrays are recursively sorted.

#### 3. At the End of Each Iteration:

- The array is divided into three parts:
  1. Elements less than the pivot element
  2. The pivot element itself
  3. Elements greater than the pivot element

This quick sort strategy results in fast sorting, especially with large datasets, because it works recursively and efficiently reduces the problem size with each step.

### Example

Consider the following unsorted list of elements...



Define pivot, left & right. Set pivot = 0, left = 1 & right = 7. Here '7' indicates 'size-1'.



Compare List[left] with List[pivot]. If List[left] is greater than List[pivot] then stop left otherwise move left to the next.

Compare List[right] with List[pivot]. If List[right] is smaller than List[pivot] then stop right otherwise move right to the previous.

Repeat the same until left >= right.

If both left & right are stopped but left < right then swap List[left] with List[right] and continue the process.  
If left >= right then swap List[pivot] with List[right].



Compare  $List[left] < List[pivot]$  as it is true increment left by one and repeat the same, left will stop at 8.  
 Compare  $List[right] > List[pivot]$  as it is true decrement right by one and repeat the same, right will stop at 2.



Here left & right both are stopped and left is not greater than right so we need to swap  $List[left]$  and  $List[right]$



Compare  $List[left] < List[pivot]$  as it is true increment left by one and repeat the same, left will stop at 6.  
 Compare  $List[right] > List[pivot]$  as it is true decrement right by one and repeat the same, right will stop at 4.



Here left & right both are stopped and left is greater than right so we need to swap  $List[pivot]$  and  $List[right]$



Here we can observe that all the numbers to the left side of 5 are smaller and right side are greater. That means 5 is placed in its correct position.

Repeat the same process on the left sublist and right sublist to the number 5.



In the left sublist as there are no smaller number than the pivot left will keep on moving to the next and stops at last number. As the  $List[right]$  is smaller, right stops at same position. Now left and right both are equal so we swap pivot with right.



In the right sublist left is greater than the pivot, left will stop at same position. As the  $List[right]$  is greater than  $List[pivot]$ , right moves towards left and stops at pivot number position. Now  $left > right$  so we swap pivot with right. (6 is swap by itself).



Repeat the same recursively on both left and right sublists until all the numbers are sorted.  
 The final sorted list will be as follows...



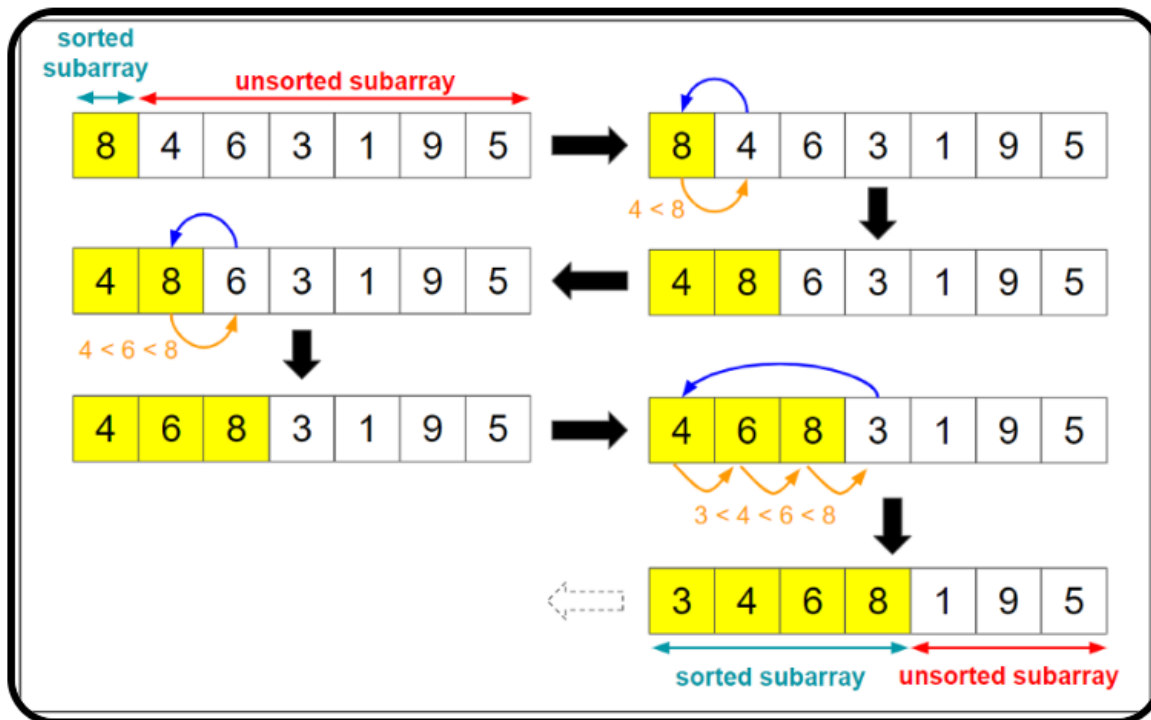
### Insertion Sort:

- It is a simple sorting algorithm where the sorted part of the array is built one entry at a time.

- It is less efficient on large lists compared to more advanced algorithms like quicksort, heap sort, or merge sort.
- Advantages:
  - Simple to implement.
  - Efficient for small datasets.
  - Adaptive, as it becomes more efficient when the data is already sorted.
  - Stable, meaning it does not change the relative order of elements with equal keys.
  - It is online, meaning it can sort a list as it receives it.

Example:

- The page gives a very simple example of insertion sort using a deck of cards. You take the cards one by one and arrange them in order.



### Algorithm: Insertion Sort

Input: An array with N elements.

Process: Compare the current key with previous elements and insert it at the appropriate position.

#### Steps for Insertion Sort (Algorithm):

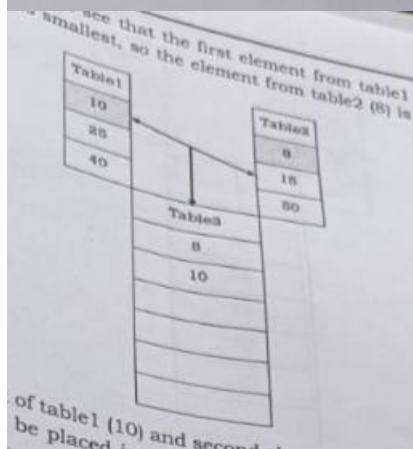
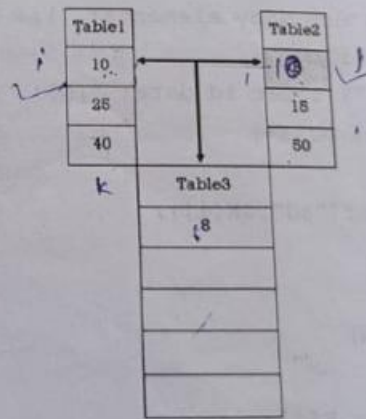
1. Step 1: Initialize
  - Set  $i = 2$  (Pointing to the second element of the array).
2. Step 2: Traverse through the list and compare elements
  - Repeat while  $i \leq N$ :
    - Set  $KEY = K[i]$  (The element to be inserted).
    - Set  $j = i - 1$  (Start comparing with the previous elements).
    - Repeat while  $j \geq 0$  and  $KEY < K[j]$ :
      - Shift  $K[j + 1] = K[j]$  (Move elements one step ahead to make room for the key).

- Decrement  $j$ .
  - Insert the KEY at position  $j + 1$  by setting  $K[j + 1] = \text{KEY}$ .
  - Increment  $i$  to move to the next element.
3. Step 3: Finish
- After all elements are sorted, the loop ends.
  - Return the sorted array  $K$ .

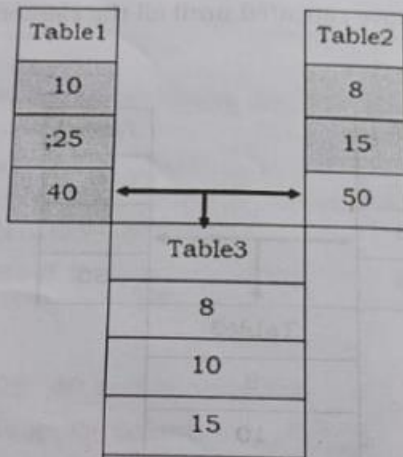
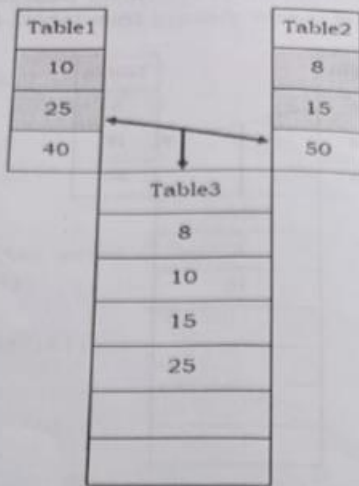
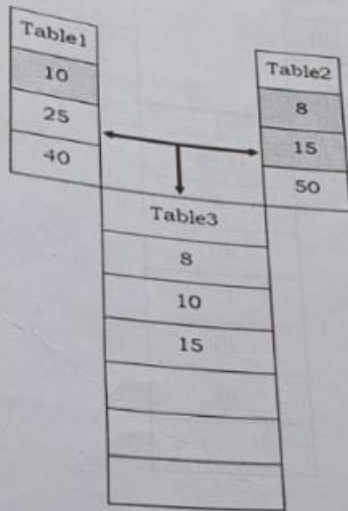
## Merge Sort

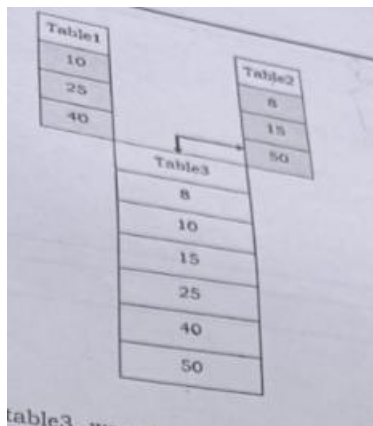
Data in table1 is  $\rightarrow 10 \ 25 \ 40$  and in table2  $\rightarrow 8 \ 15 \ 50$

- Trace of merge sort is given below:



created until all the elemen






---

## Steps for Merge Sort

### Algorithm: MERGE\_SORT

**This algorithm sorts two tables into a third one (both tables must be sorted).**

- List1 is the first list (table) of m elements.
- List2 is the second list (table) of n elements.
- List3 is the third list (table) where the elements are to be sorted.

#### Step 1. [initialize]

- $i = 0$
- $j = 0$
- $k = 0$

#### Step 2. [initialize loop]

- Repeat while  $i < m$  AND  $j < n$ 
  - If  $\text{list1}[i] \neq \text{list2}[j]$  (comparing corresponding elements)
    - Then:  $\text{list3}[k] = \text{list1}[i]$ 
      - $i = i + 1$
      - $k = k + 1$
  - Else:
    - $\text{list3}[k] = \text{list2}[j]$ 
      - $j = j + 1$
      - $k = k + 1$

#### Step 3. [if size of list1 is greater than list2]

- Repeat while  $i < m$

#### Step 4: [Size of List2 is greater than List1]

1. repeat while ( $j < n$ )
2. {
3.  $\text{list3}[k] = \text{list2}[j];$

4.  $k = k + 1;$
5.  $j = j + 1;$
6. }

**Step 5: [Finished]**

7. `return(k);`

### 6.1.6 Radix Sort

This sorting method is used in digital computers. It is also used when we have to sort a large number of elements.

- In this method, **10 buckets** (pocket) of digits (like 0 to 9) are used to store each digit. So, this method is also called **Bucket Sort**.
  - **During the first pass**, we separate the unit digit of the number and place the number into the appropriate pocket based on its unit digit.
  - **After the first pass**, rearrange the elements in **FIFO (QUEUE)** order. (All the pockets 0-9 are operated in FIFO manner.)
  - **During the second pass**, we separate the next higher digit of the number and place it into the appropriate pocket based on its digit.
  - After this, rearrange the elements as above.
  - **The same procedure repeats** until all the elements are sorted.
- 

**Radix Sort** is a linear sorting algorithm (for fixed length digit counts) that sorts elements by processing them digit by digit. It is an efficient sorting algorithm for integers or strings with fixed-size keys.

- It repeatedly distributes the elements into buckets based on each digit's value. This is different from other algorithms like Merge Sort or Quick Sort where we compare elements directly.
- By repeatedly sorting the elements by their significant digits, from the least significant to the most significant, it achieves the final sorted order.

Consider this input

# Array

170

45

75

90

802

24

2

66

Unsorted

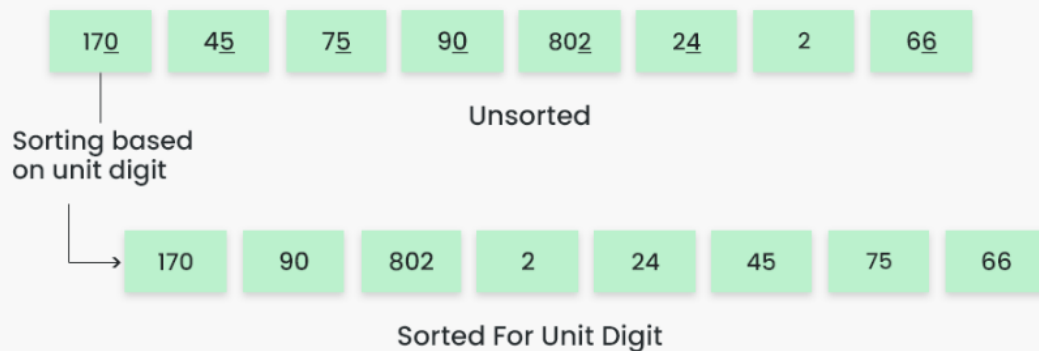
Radix Sort

**Step 1:** Find the largest element in the array, which is 802. It has three digits, so we will iterate three times, once for each significant place.

**Step 2:** Sort the elements based on the unit place digits ( $X=0$ ). We use a stable sorting technique, such as counting sort, to sort the digits at each significant place. It's important to understand that the default implementation of counting sort is unstable i.e. same keys can be in a different order than the input array. To solve this problem, We can iterate the input array in reverse order to build the output array. This strategy helps us to keep the same keys in the same order as they appear in the input array.

**Sorting based on the unit place:**

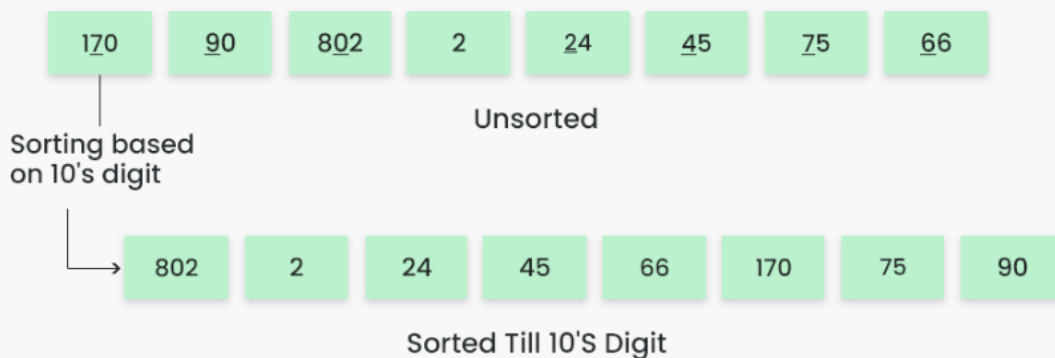
- Perform counting sort on the array based on the unit place digits.
- The sorted array based on the unit place is [170, 90, 802, 2, 24, 45, 75, 66].



**Step 3:** Sort the elements based on the tens place digits.

**Sorting based on the tens place:**

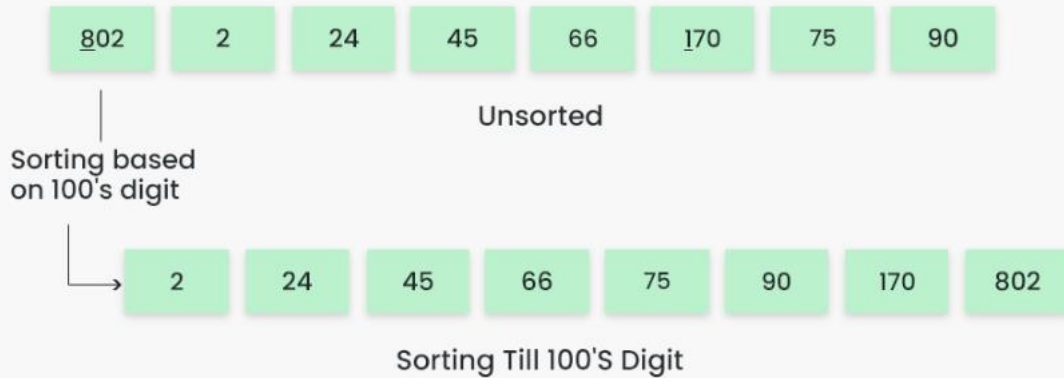
- Perform counting sort on the array based on the tens place digits.
- The sorted array based on the tens place is [802, 2, 24, 45, 66, 170, 75, 90].



**Step 4:** Sort the elements based on the hundreds place digits.

**Sorting based on the hundreds place:**

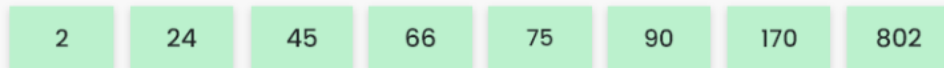
- Perform counting sort on the array based on the hundreds place digits.
- The sorted array based on the hundreds place is [2, 24, 45, 66, 75, 90, 170, 802].



**Step 5:** The array is now sorted in ascending order.

The final sorted array using radix sort is [2, 24, 45, 66, 75, 90, 170, 802].

Array after performing **Radix Sort** for all digits



### Algorithm: RADIX\_SORT

To sort based on each digit in the key:

1. **Initialize the queues.**
2. Repeat for each digit (from the least significant digit to the most significant):
  - $y = a[i]$  (where  $a[i]$  is the current number).
  - $j$  = digit at the particular position of  $y$  (find the digit at the current position).
  - Insert  $y$  into the appropriate queue/pocket based on  $j$ .
3. Combine the queues to form a new linked list.

### Summary of Sorting Methods

**Selection Sort or Bubble Sort:** These are useful when the number of records is small.

**Quick Sort:** Useful when the data is large and keys are short.

**Radix Sort:** Suitable for large data and long keys.

**Merge Sort:** Suitable when keys are uniformly distributed across an interval.

### Analysis of Different Sorting Methods:

| Sorting Method | Best Case | Average Case | Worst Case |
|----------------|-----------|--------------|------------|
| Bubble Sort    | $O(n)$    | $O(n^2)$     | $O(n^2)$   |
| Selection Sort | $O(n^2)$  | $O(n^2)$     | $O(n^2)$   |

| Sorting Method | Best Case     | Average Case  | Worst Case    |
|----------------|---------------|---------------|---------------|
| Quick Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      |
| Insertion Sort | $O(n)$        | $O(n^2)$      | $O(n^2)$      |
| Merge Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ |
| Radix Sort     | $O(m+n)$      | $O(m+n)$      | $O(m+n)$      |

Where "m" is the number of digits or the number of bits used in the radix sort.

### Hashing Concepts:

- In search techniques like linear search, binary search, and tree search, the time required to search for an element depends on the total number of elements in the data structure.
- As the number of elements increases, the time required to search for an element also increases linearly.
- Hashing performs searching in constant time and is efficient, as it allows for direct access of records regardless of where they are stored in a file.

### Examples of hashing:

- In universities, each student is assigned a unique roll number for retrieving information.
- In libraries, each book is assigned a unique identifier, helping in tracking its exact location or issue status.

### Amortized Analysis of Hash Table Insertions

#### Choosing the Table Size

When designing a hash table, we must decide on the table size. There is a trade-off between space and time. If the hash table size is large, search time becomes low, but the space required increases.

